

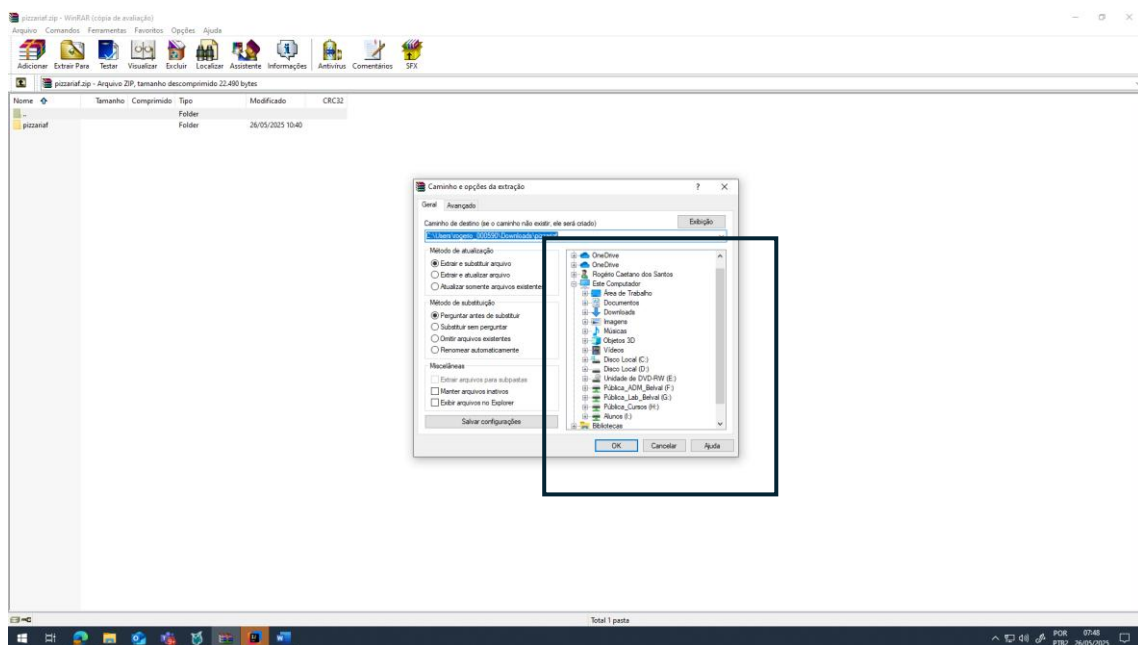
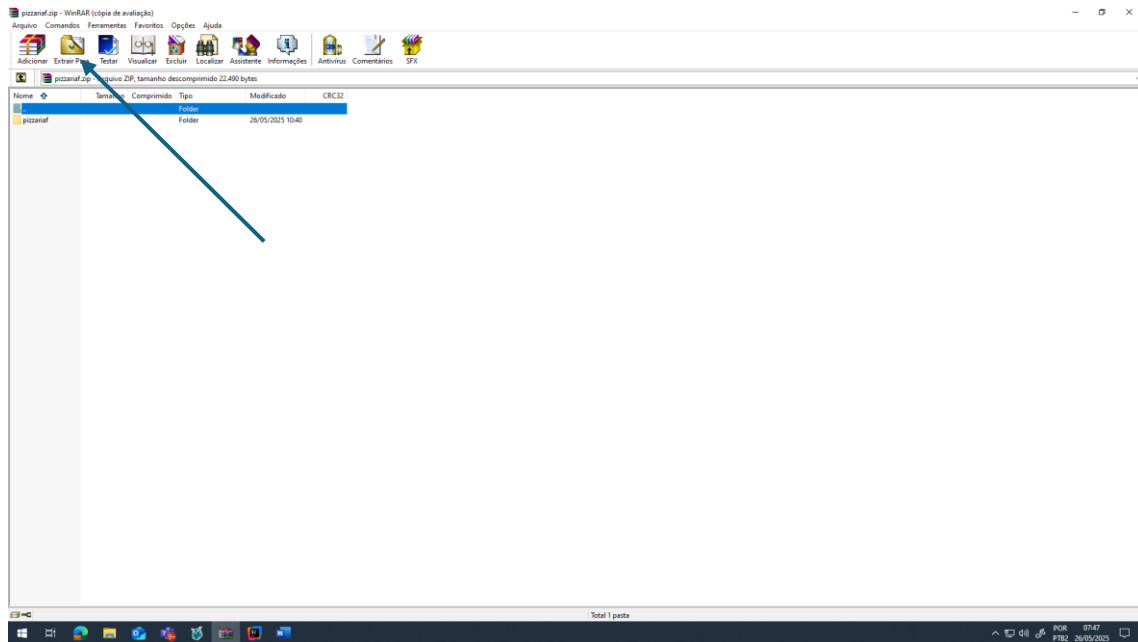
Projeto Pizzaria – Spring Boot

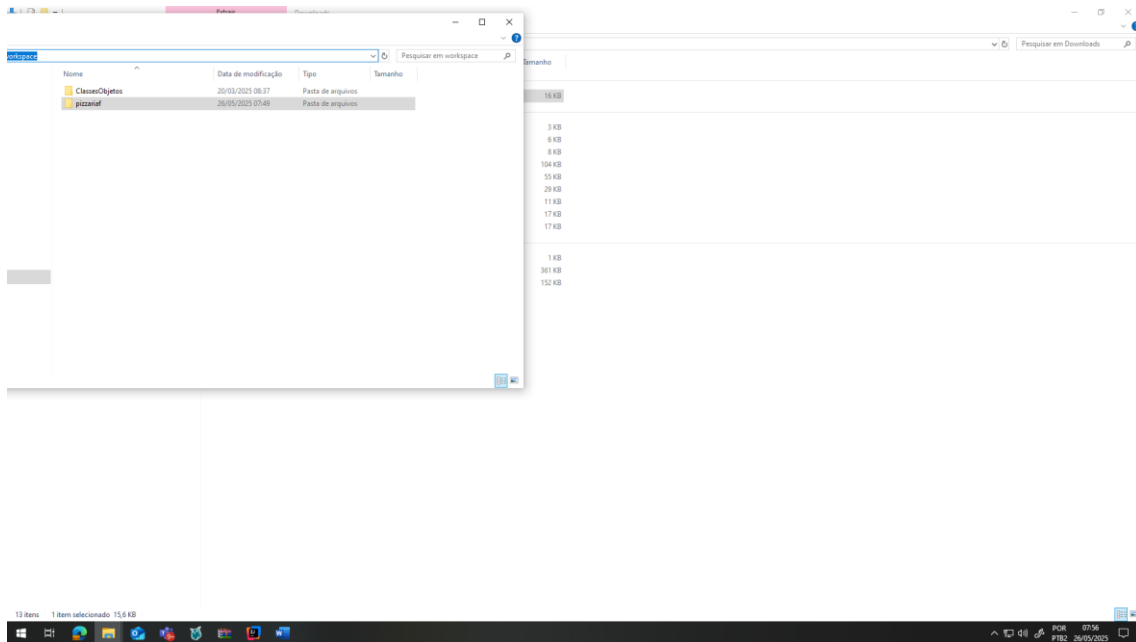
Site: [Spring Initializr](https://start.spring.io)

The screenshot shows the Spring Initializr web interface. On the left, the 'Project' section has 'Maven' selected and circled in yellow. The 'Language' section has 'Java' selected. The 'Spring Boot' section shows version '4.1.1' selected and highlighted with a blue arrow. The 'Project Metadata' section contains fields for Group (com.itb.inf2fm), Artifact (pizzariaf), and Package name (com.itb.inf2fm.pizzariaf). The 'Packaging' section has 'Jar' selected. The 'Configuration' section has 'Properties' selected. The 'Java' section shows version '21' selected. On the right, the 'Dependencies' section lists 'Spring Web' and 'Spring Boot DevTools'. A blue arrow points to the 'ADD DEPENDENCIES... CTRL + B' button. At the bottom, there are buttons for 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a menu icon.

Nota: Dependendo da data em que você acessar o site, a versão do Spring Boot bem como a versão da linguagem Java será diferente. Escolha sempre a versão indicada pelo site.

Descompactar em sua pasta de trabalho

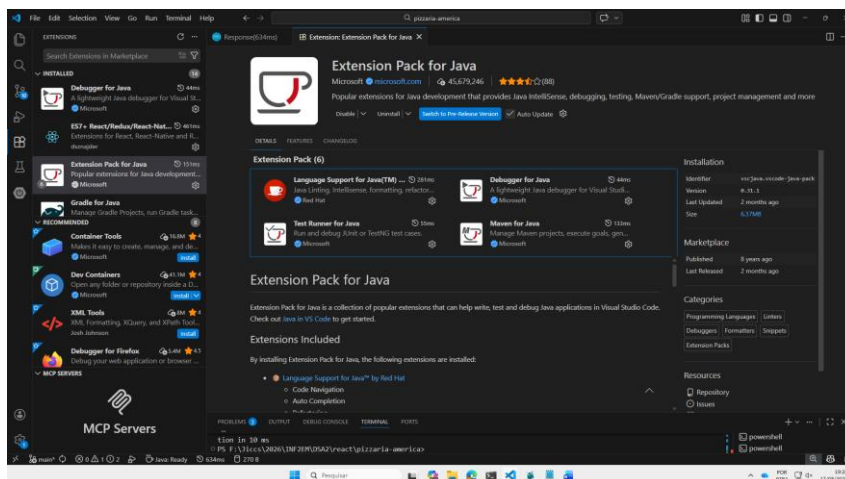


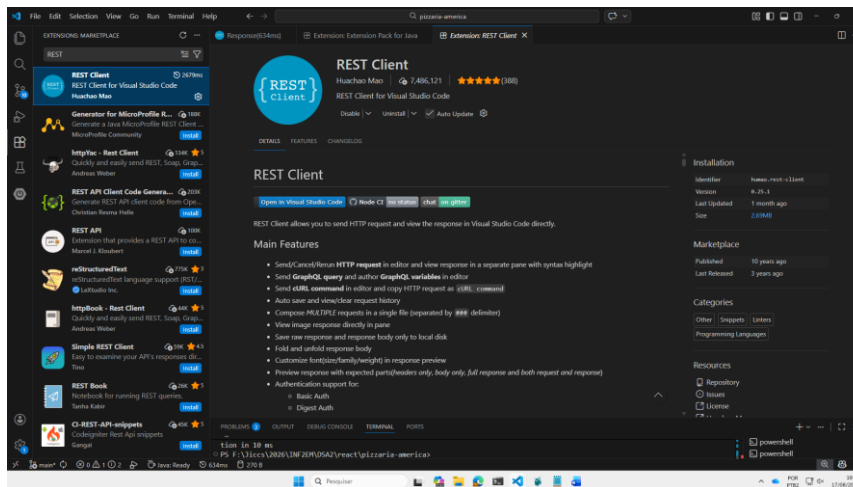
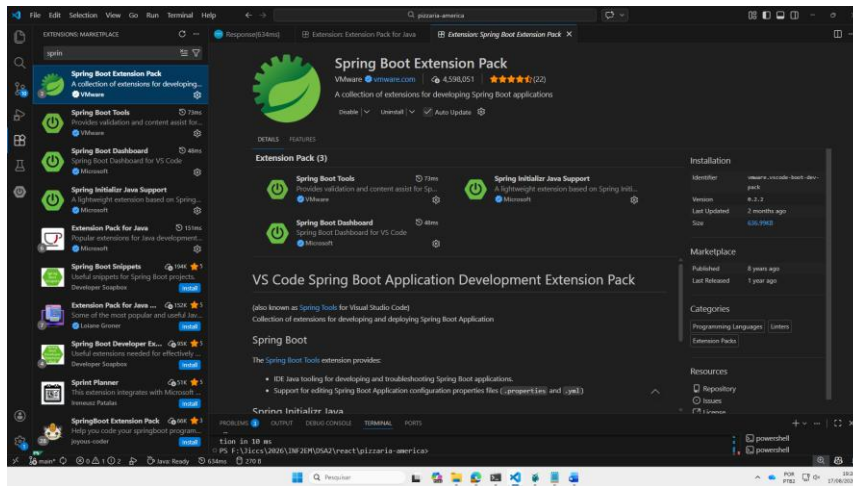


Abrir a pasta “pizzaria” com a IDE da sua preferência (Vs Code, IntelliJ etc)

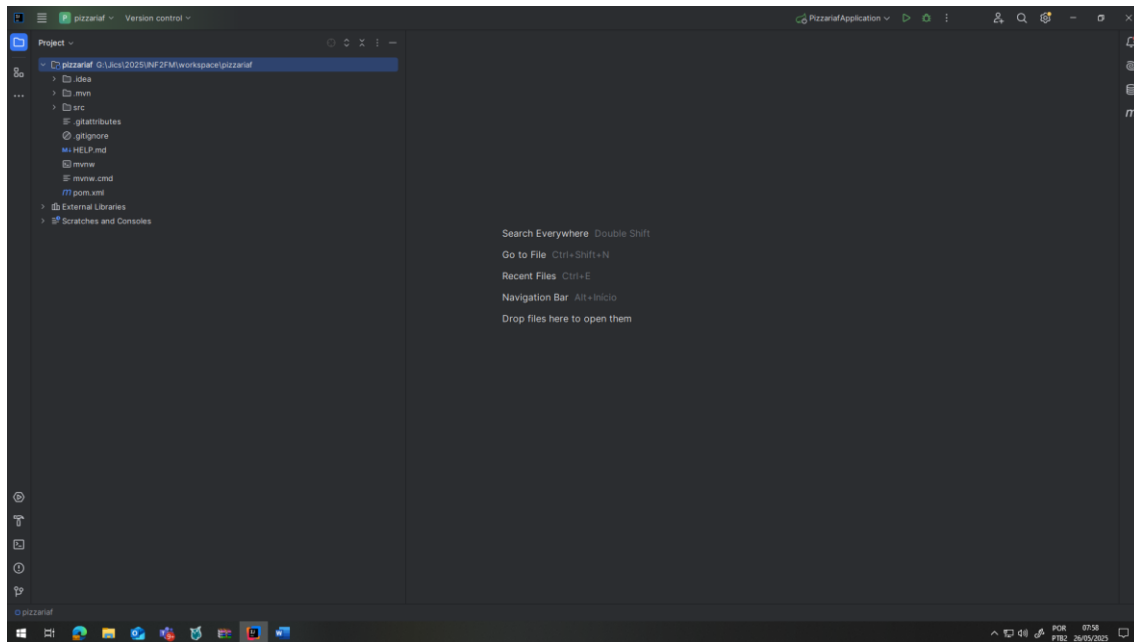
No **VS-CODE**, utilize as extensões:

- Extension Pack for Java
- Spring Boot Extension Pack
- REST Client

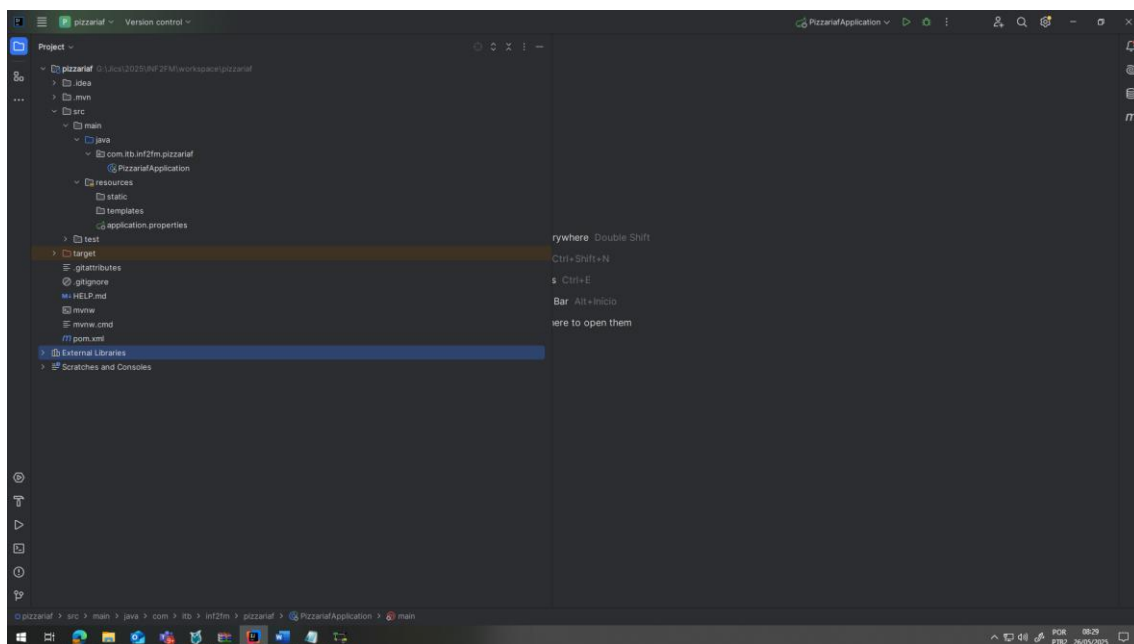




Nesta apostila, vamos utilizar o **IntelliJ**



Veja a estrutura de pastas:



src: Contém todo o código-fonte e testes da aplicação

src/main: Onde fica o código java da aplicação, estruturado por pacotes (ex: controller, service, repository, etc)

src/main/resources: Arquivos de configuração e recursos da aplicação

- application.properties ou application.yml : Configurações do Spring (porta, banco, etc)
- static : Arquivos estáticos (CSS, JS, imagens e arquivos)
- templates: Arquivos de templates (Html, Thymeleaf, freemarker, etc)

src/test : Contém os testes unitários e de integração. Estrutura igual a de main/java

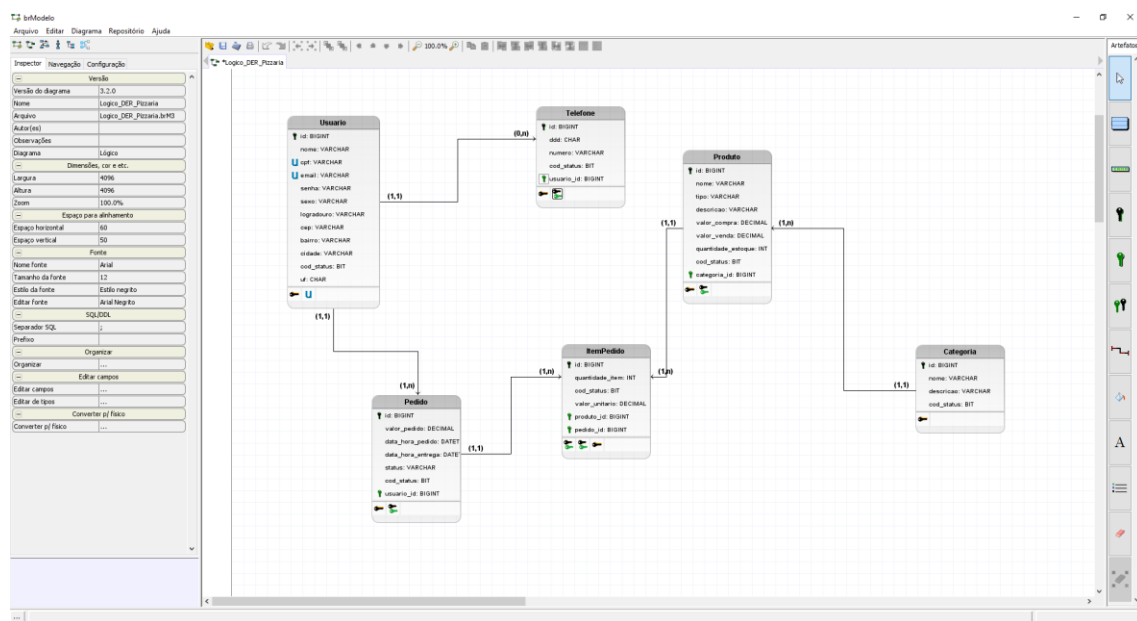
target: Gerado automaticamente após o mvn package ou mvn install. Contém:

- O .jar ou .war final da aplicação
- Classes compiladas
- Arquivos temporários de build

Nota: Você nunca edita essa pasta manualmente. Ela pode ser deletada a qualquer momento e será recriada.

Então vamos lá:

Para nossa pizzaria, vamos considerar o seguinte modelo de dados:



Antes... um pequeno Desafio

Desafio

1-) Acessar o site [Spring Initializr](https://spring.io/) e criar o projeto referente

ao seu trabalho TCC com as seguintes dependências:

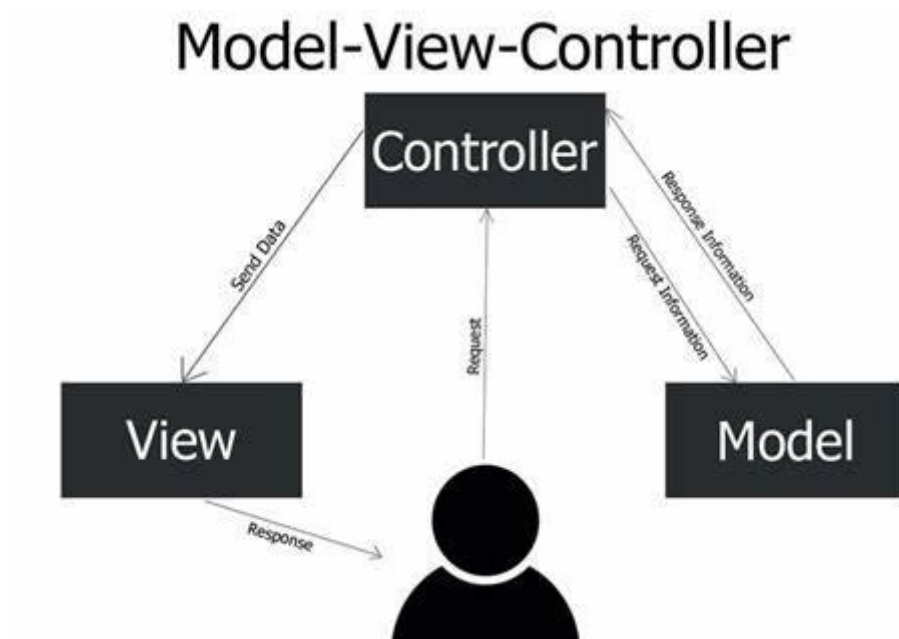
- Spring WEB
- Spring DevTools

2-) Abrir o projeto utilizando o IntelliJ ou o VsCode

Na construção do projeto, vamos adotar alguns padrões.

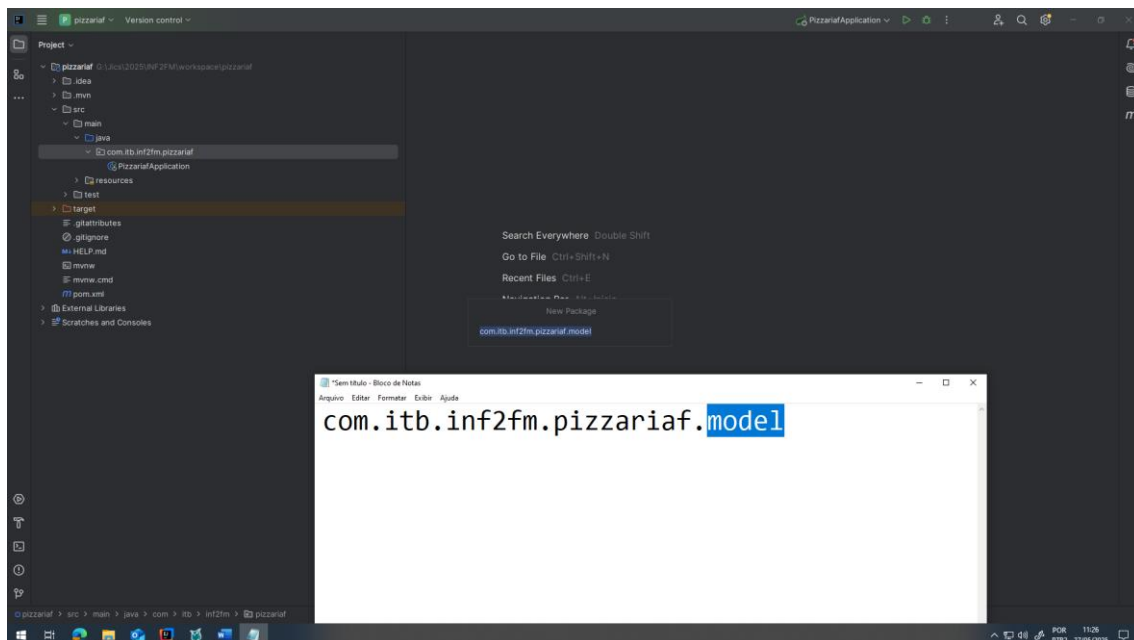
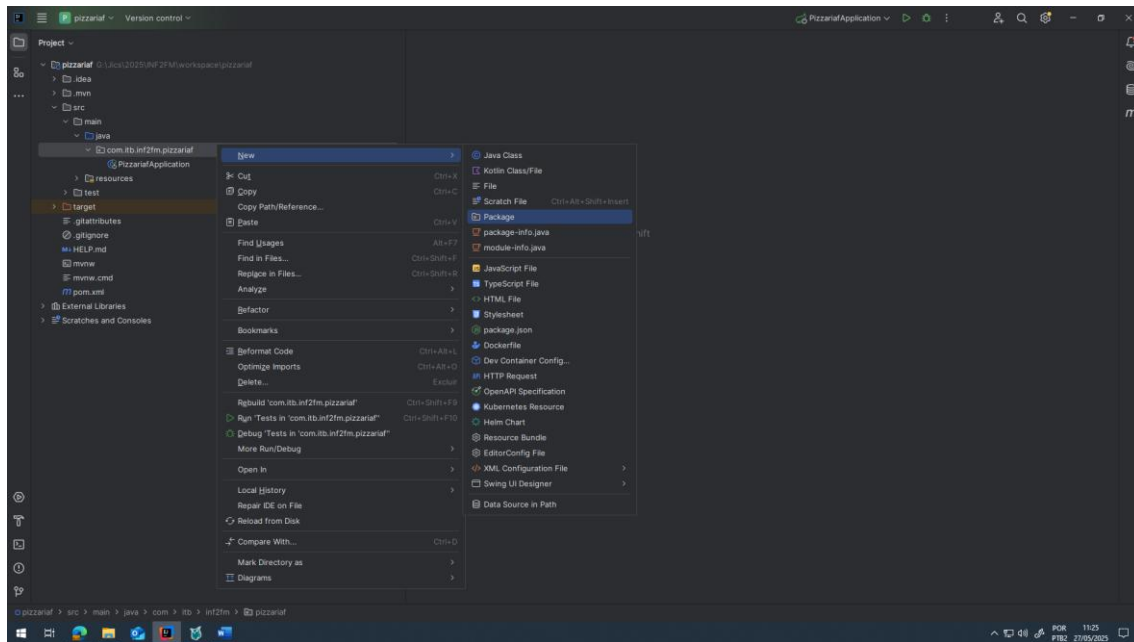
O primeiro padrão será um dos padrões de arquitetura mais utilizado em sistemas Web, o **MVC** (Model, View e Controller).

[O que é MVC? | Blog da TreinaWeb](#)



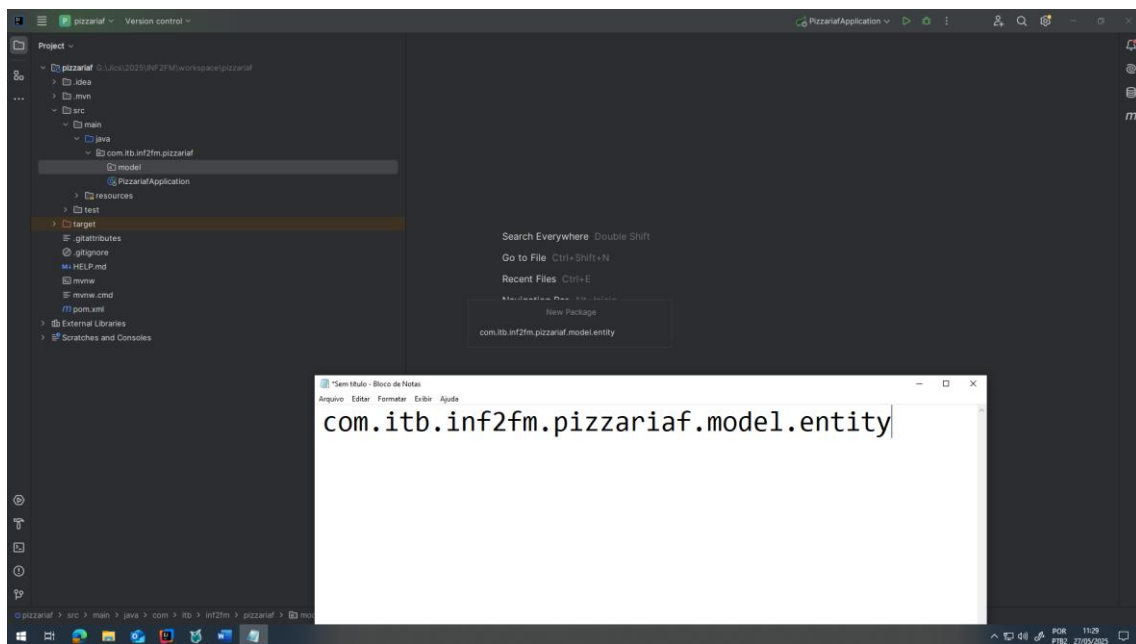
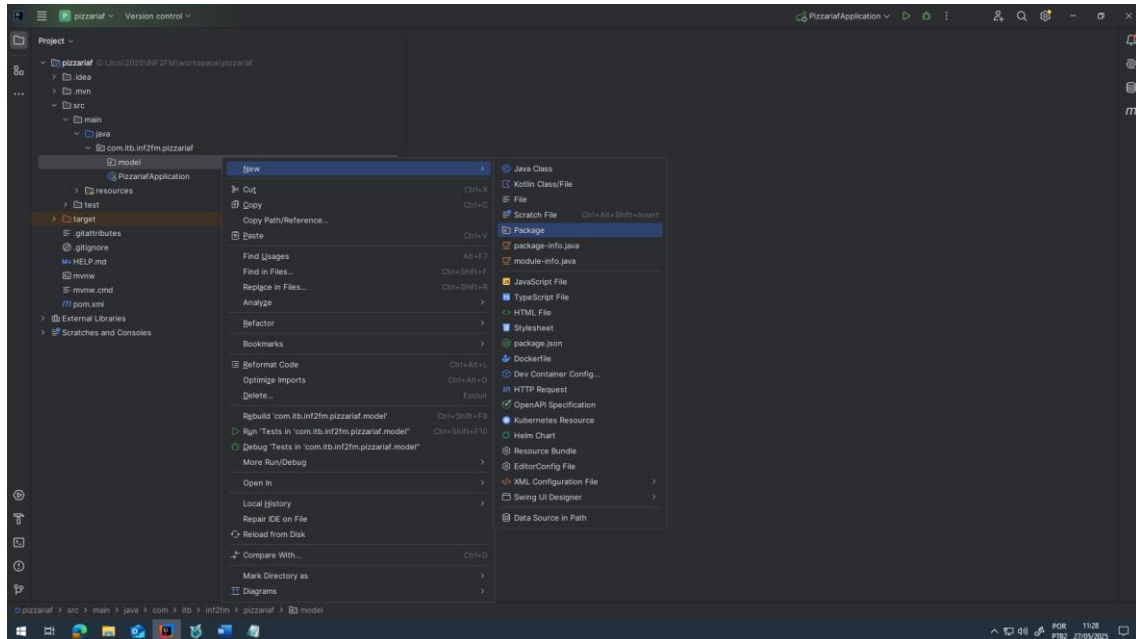
Sobre o pacote principal vamos criar um novo pacote referente a camada “model”

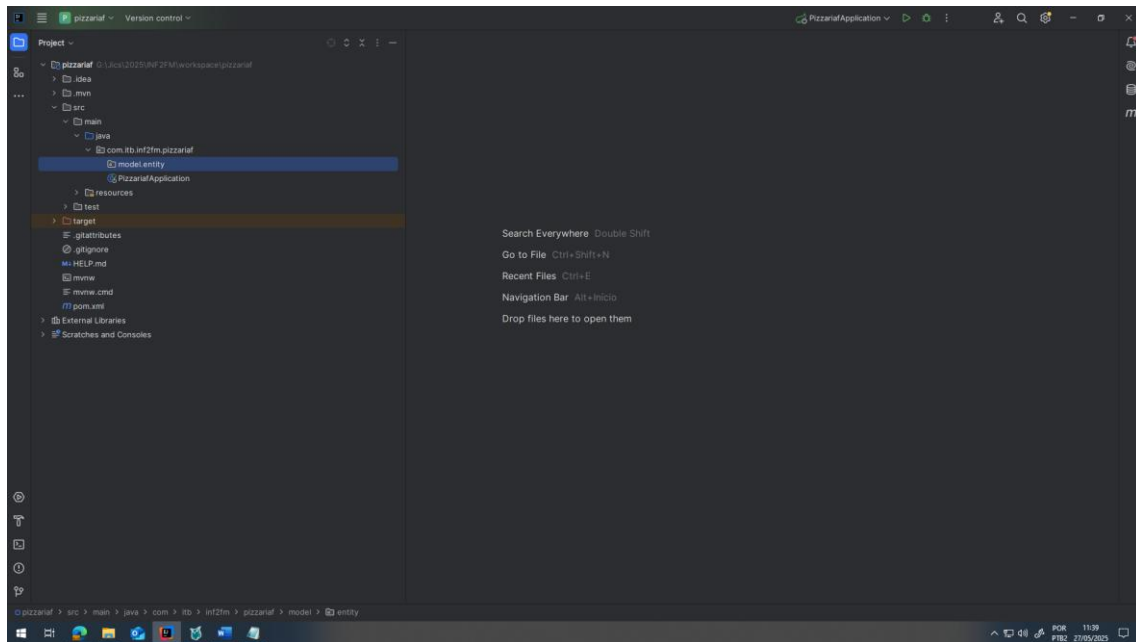
Veja:



O model pode ser dividido em outros pacotes, vamos criar um sub-pacote chamado “entity”

Veja:

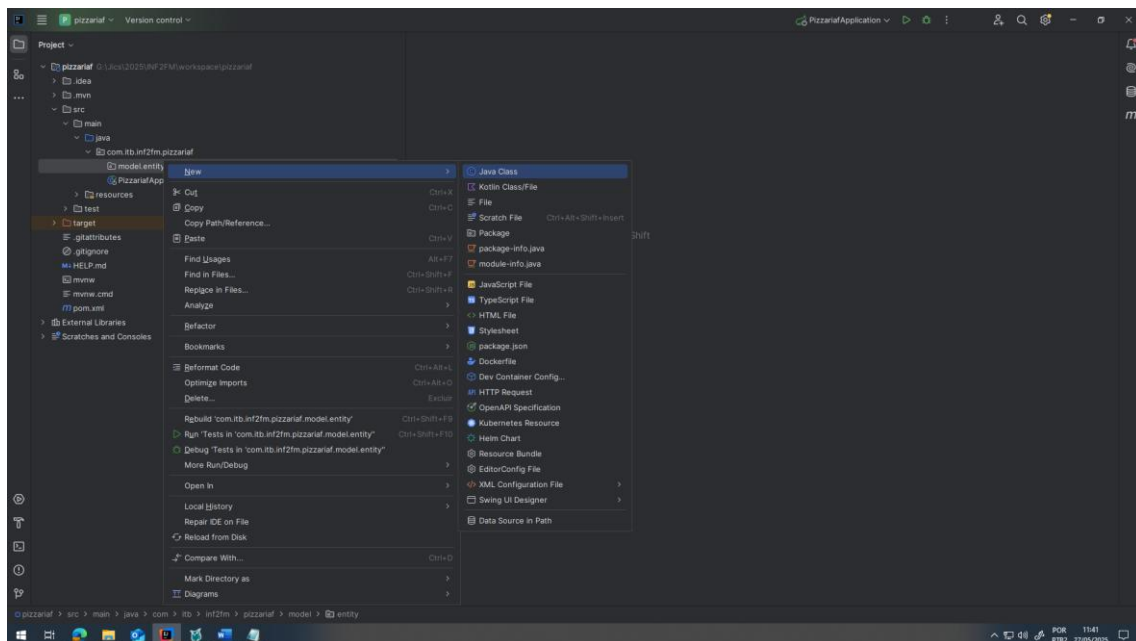


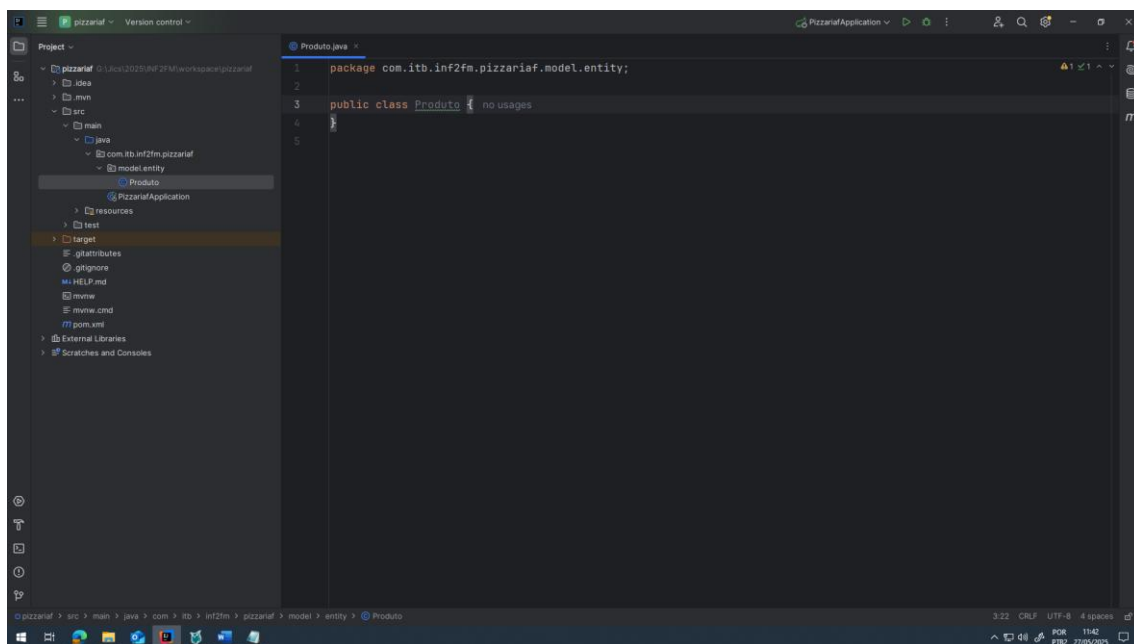
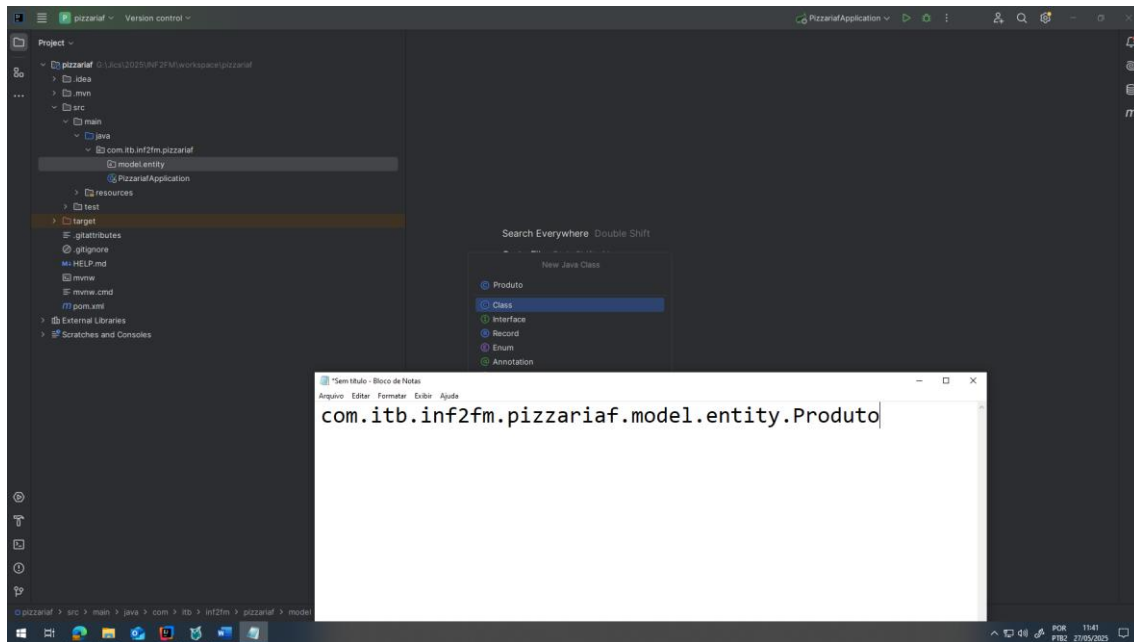


Vamos criar nossa primeira entidade!

Com o botão direito sobre o pacote “entity” criar a classe Produto

Veja:

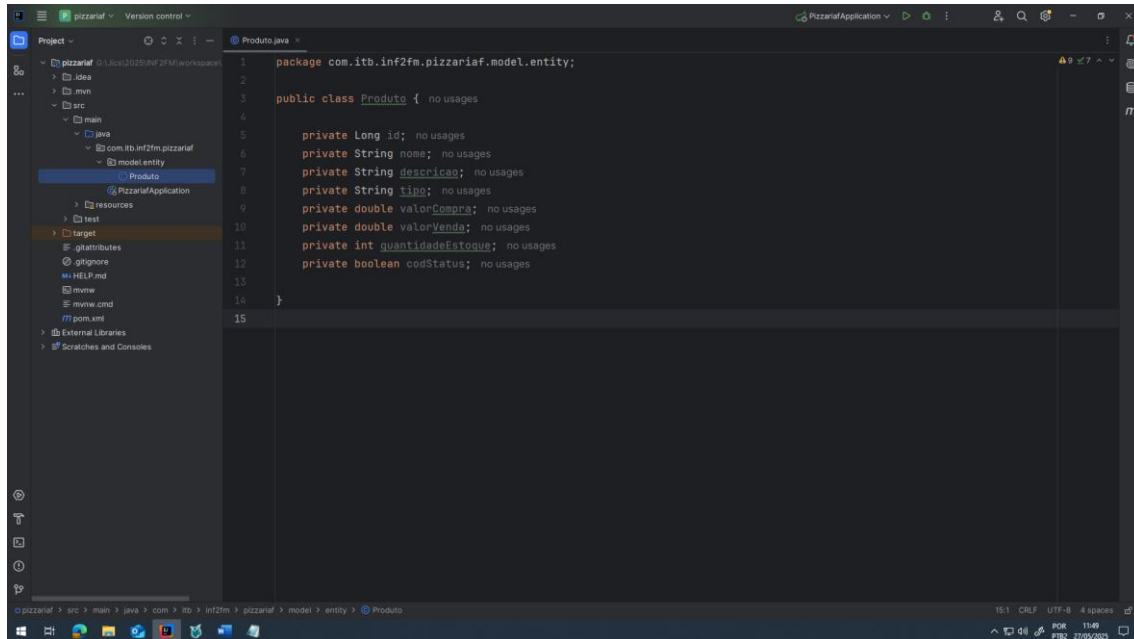




Criar os atributos de acordo com o seu model (DER) cada coluna é representada por um atributo.

Nota: Cada tabela do banco de dados corresponde a uma entity em nosso projeto.

Neste primeiro momento, vamos ignorar a chave estrangeira (FK)



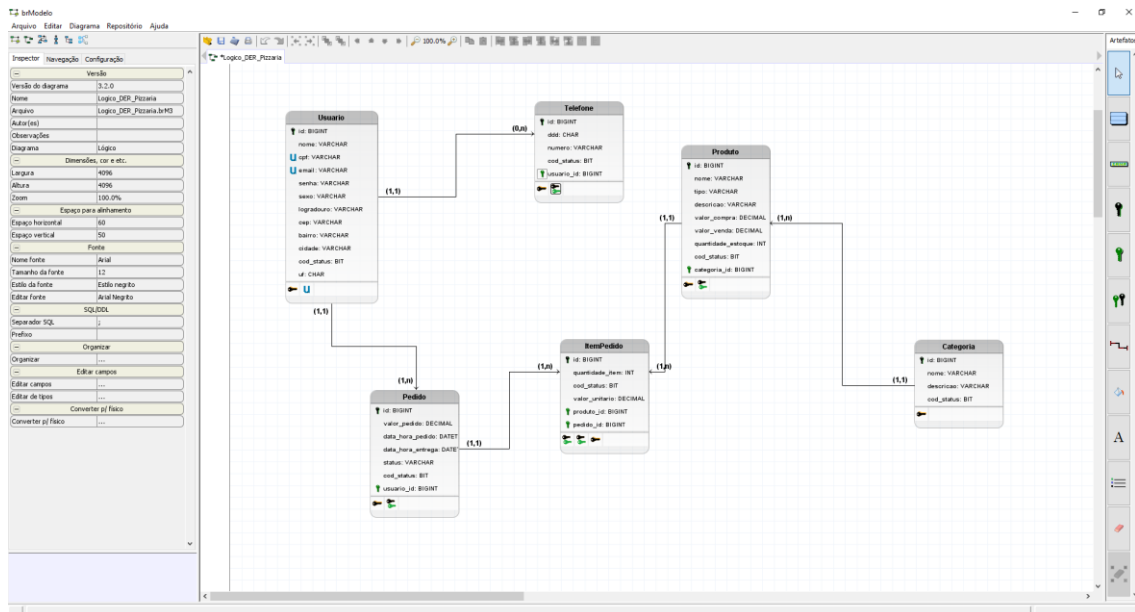
Desafio

Abrir seu projeto de TCC e criar o seu primeiro model “entity”, escolha qualquer tabela que não seja uma tabela associativa.

Obs: Aplicar o MVC na estruturação de pacotes, conforme foi feito no projeto pizzariaf.

Exercícios

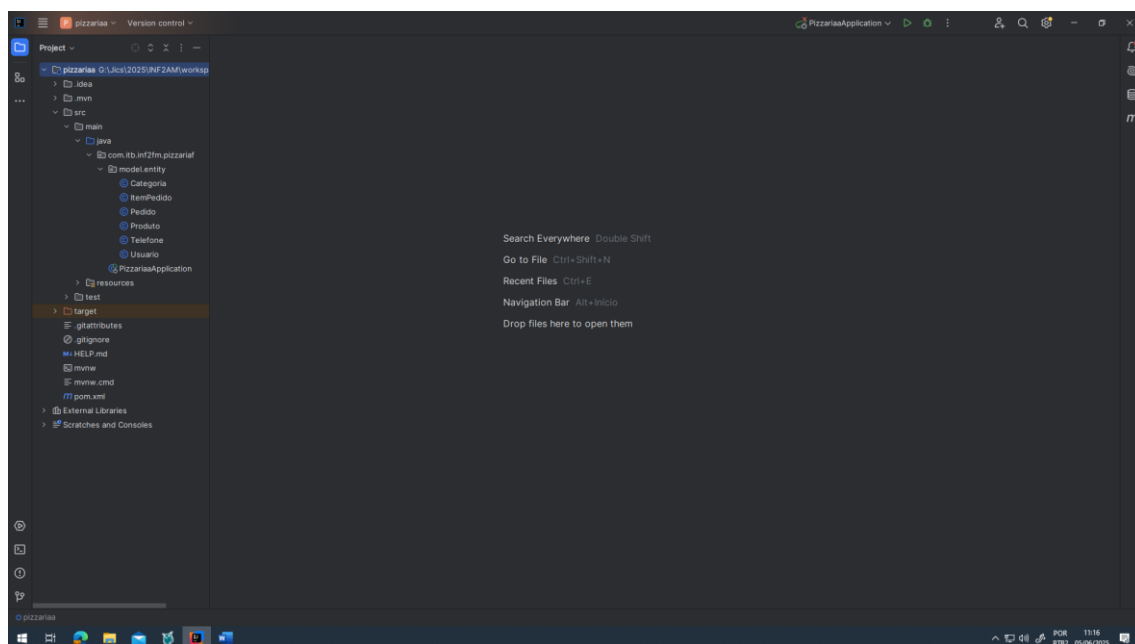
De acordo com o DER a seguir:



- 1-) Além do Produto, criar os demais modelos "entity" referente ao DER Pizzaria
- 2-) Criar os métodos setter's e getter's de cada atributo

Resolução

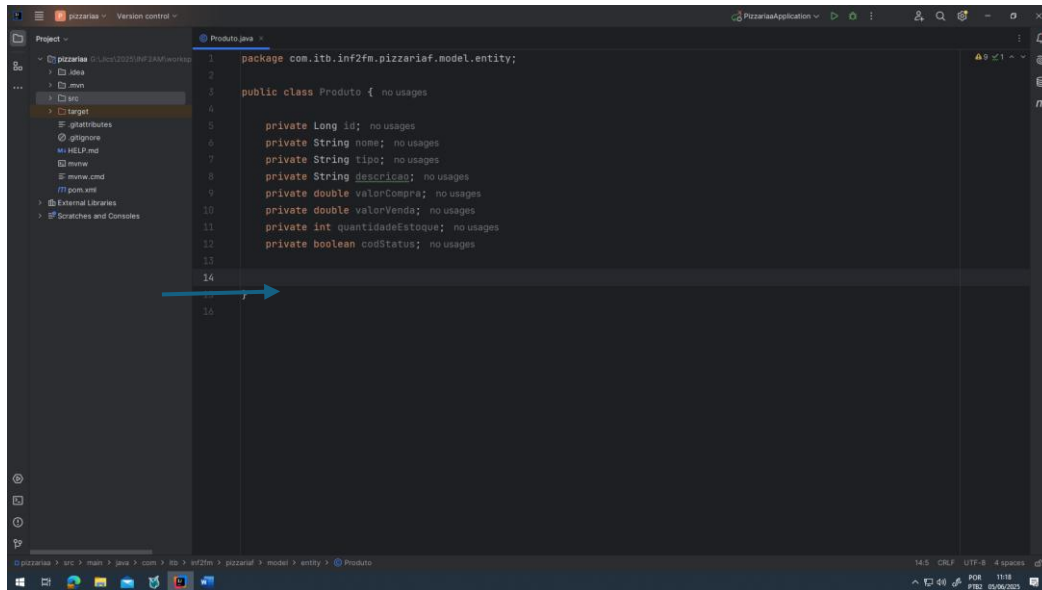
Ex. 1-) No pacote entity, todas as classes que representam o DER



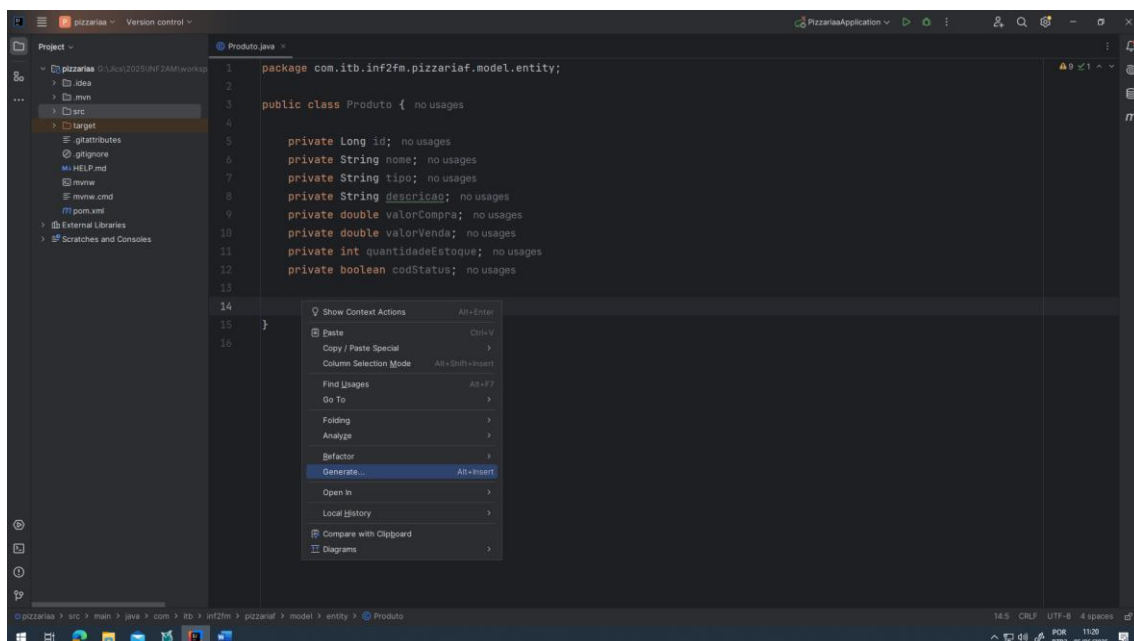
Ex. 2-) Criando Setter's e Getter's de cada classe:

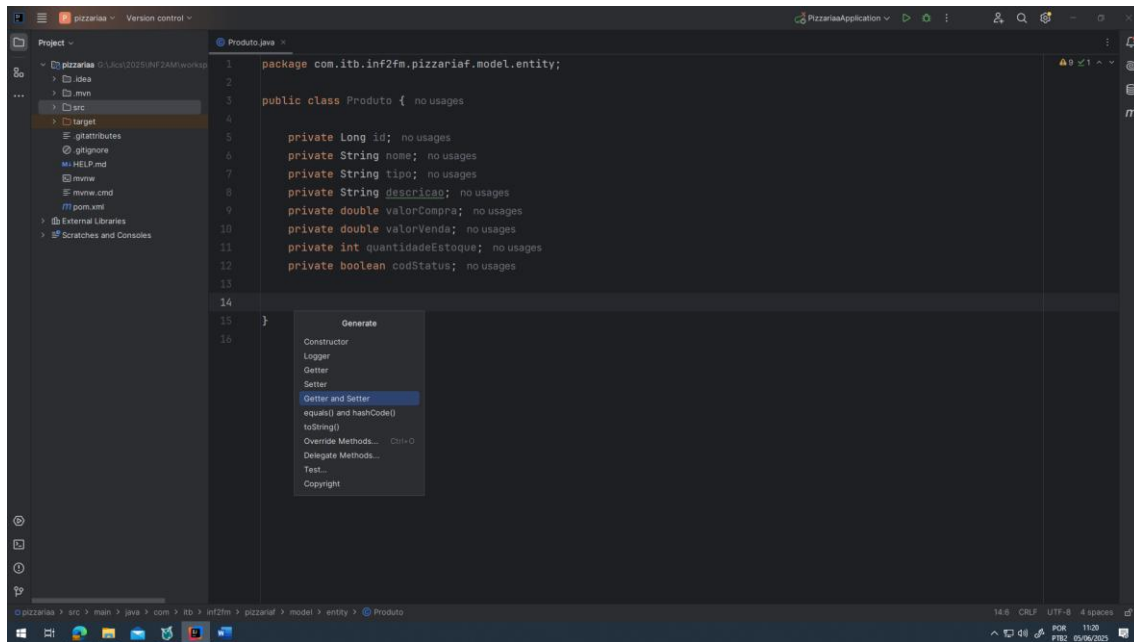
A seguir, as imagens da classe **Produto**

Posicionar o curso do mouse abaixo dos atributos.

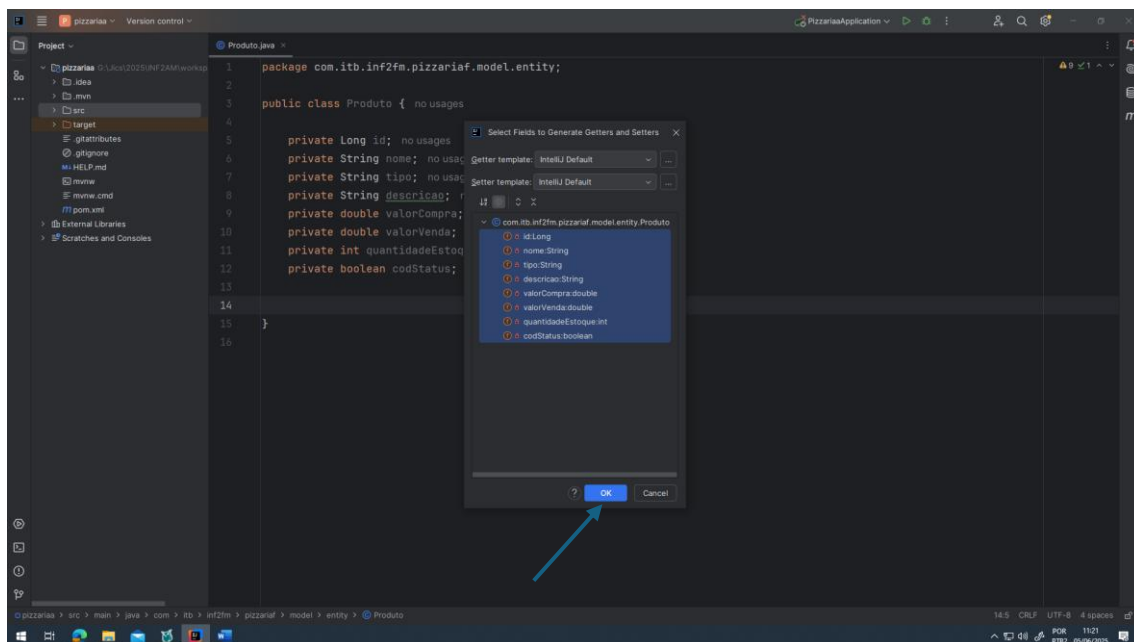


Atalho [Alt + Insert] -> generate -> getter and Setter





Selecionar todos os campos



Clique em “OK”

Veja o resultado a seguir:

```

1 package com.itb.inf2fa.pizzariaf.model.entity;
2
3 public class Produto { no usages
4
5     private Long id; 2 usages
6     private String nome; 2 usages
7     private String tipo; 2 usages
8     private String descricao; 2 usages
9     private double valorCompra; 2 usages
10    private double valorVenda; 2 usages
11    private int quantidadeEstoque; 2 usages
12    private boolean codStatus; 2 usages
13
14    public Long getId() { no usages
15        return id;
16    }
17
18    public void setId(Long id) { no usages
19        this.id = id;
20    }
21
22    public String getNome() { no usages
23        return nome;
24    }
25
26    public void setNome(String nome) { no usages
27        this.nome = nome;
28    }
29
30    public String getTipo() { no usages

```

```

31    public void setTipo(String tipo) { no usages
32        this.tipo = tipo;
33    }
34
35    public String getDescricao() { no usages
36        return descricao;
37    }
38
39    public void setDescricao(String descricao) { no usages
40        this.descricao = descricao;
41    }
42
43    public double getValorCompra() { no usages
44        return valorCompra;
45    }
46
47    public void setValorCompra(double valorCompra) { no usages
48        this.valorCompra = valorCompra;
49    }
50
51    public double getValorVenda() { no usages

```

```

52    public void setValorVenda(double valorVenda) { no usages
53        this.valorVenda = valorVenda;
54    }
55
56    public int getQuantidadeEstoque() { no usages
57        return quantidadeEstoque;
58    }
59
60    public void setQuantidadeEstoque(int quantidadeEstoque) { no usages
61        this.quantidadeEstoque = quantidadeEstoque;
62    }
63
64    public boolean isCodStatus() { no usages
65        return codStatus;
66    }
67
68    public void setCodStatus(boolean codStatus) { no usages
69        this.codStatus = codStatus;
70    }
71
72    }
73
74    }

```

Fazer com todas as demais classes (Pedido, Telefone, ItemPedido, Usuario e Categoria)

Annotations

No Spring é comum adicionarmos anotações (annotations) em classes, atributos e métodos.

Uma annotation é uma forma de adicionar informações (metadados) ao seu código que podem ser interpretadas pelo compilador ou em tempo de execução por ferramentas e frameworks como o Spring.

Uma annotation é uma palavra iniciada com '@' que você coloca acima de uma classe, método, atributo ou parâmetro, para dar instruções extras ao Java ou a algum framework sobre como aquele elemento deve ser tratado.

Vamos dar continuidade em nosso projeto agora focando na camada “**controller**”, as annotations referente ao MVC fazem parte do “Spring Web” uma das dependências que instalamos no início do projeto.

Confira no arquivo **pom.xml**

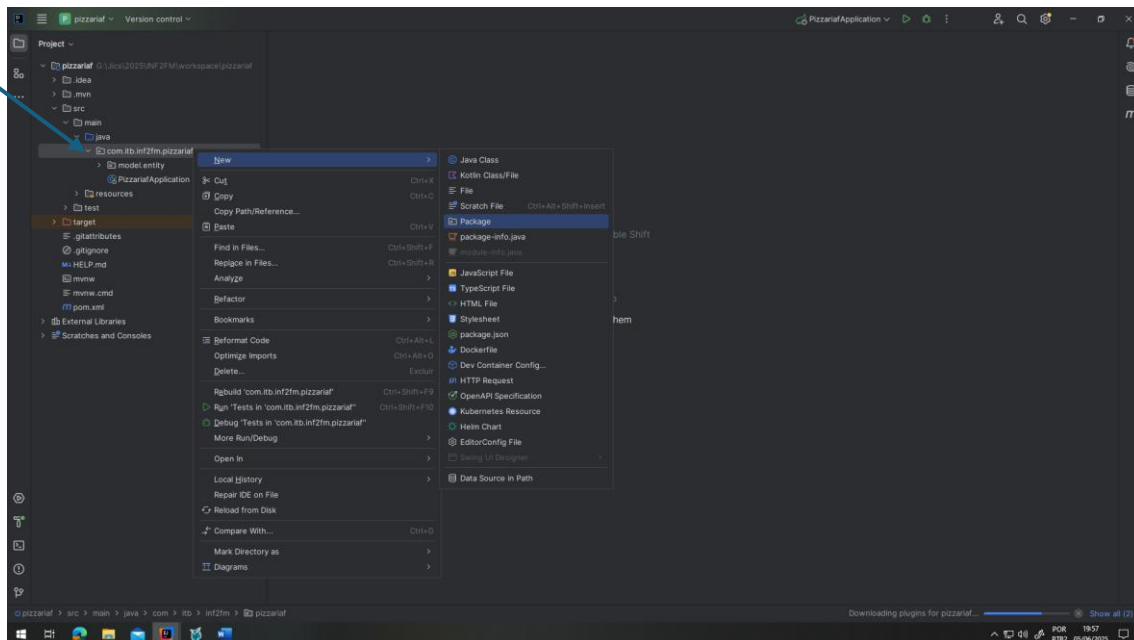
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Abaixo o quadro com as principais annotations do **Spring Web** que iremos utilizar.

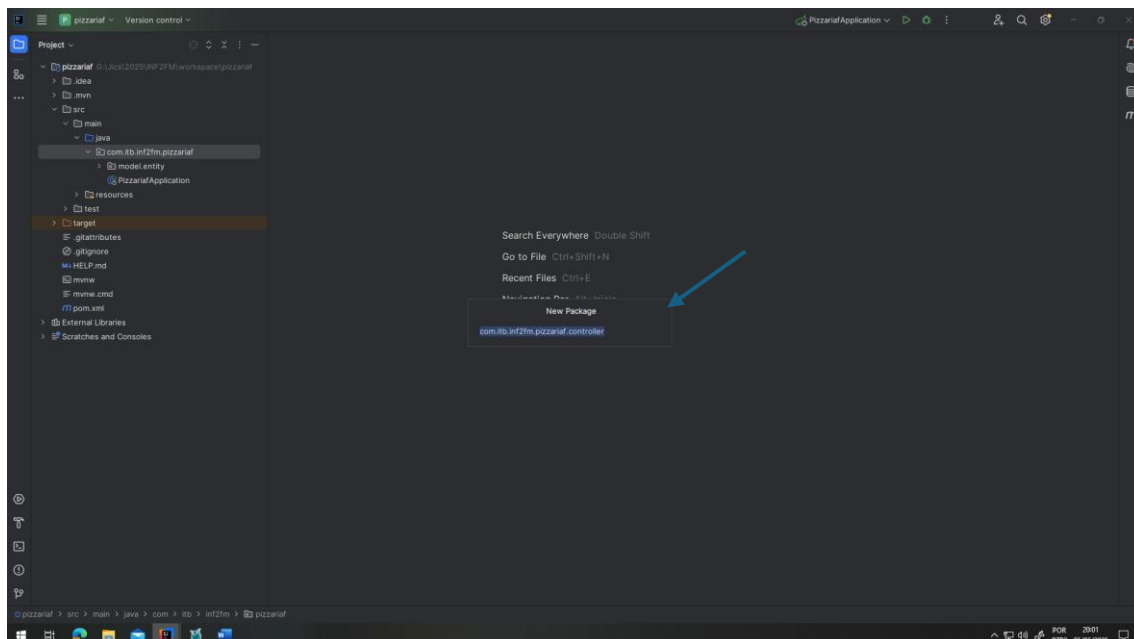
Anotação	Método HTTP	Finalidade	Aplicação
@Controller		Indica que a classe é um controlador Spring MVC (retorna páginas/view)	Classe
@RestController		Indica que a classe é um controlador REST (retorna JSON diretamente)	Classe
@GetMapping	GET	Mapeia requisições GET para métodos	Método
@PostMapping	POST	Mapeia requisições POST para criar novos recursos	Método
@PutMapping	PUT	Mapeia requisições PUT para atualizar completamente um recurso.	Método
@PatchMapping	PATCH	Mapeia requisições PATCH para atualizar parciais.	Método
@DeleteMapping	DELETE	Mapeia requisições DELETE para deletar recursos.	Método
@RequestMapping	Qualquer (configurável)	Anotação genérica que permite configurar o tipo de requisição manualmente.	Classe e Método

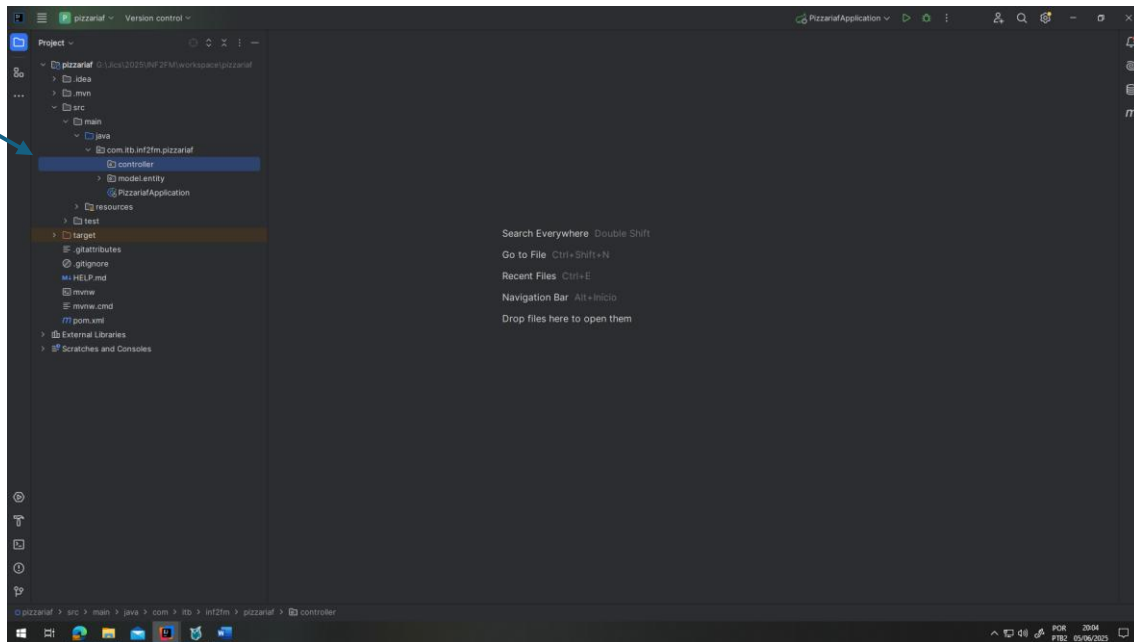
Em nosso projeto, seguindo o padrão MVC iremos criar a camada de “controller”

- Clique com o botão direito sobre o pacote principal **com.itb.inf2fm.pizzariaf**
- New Package



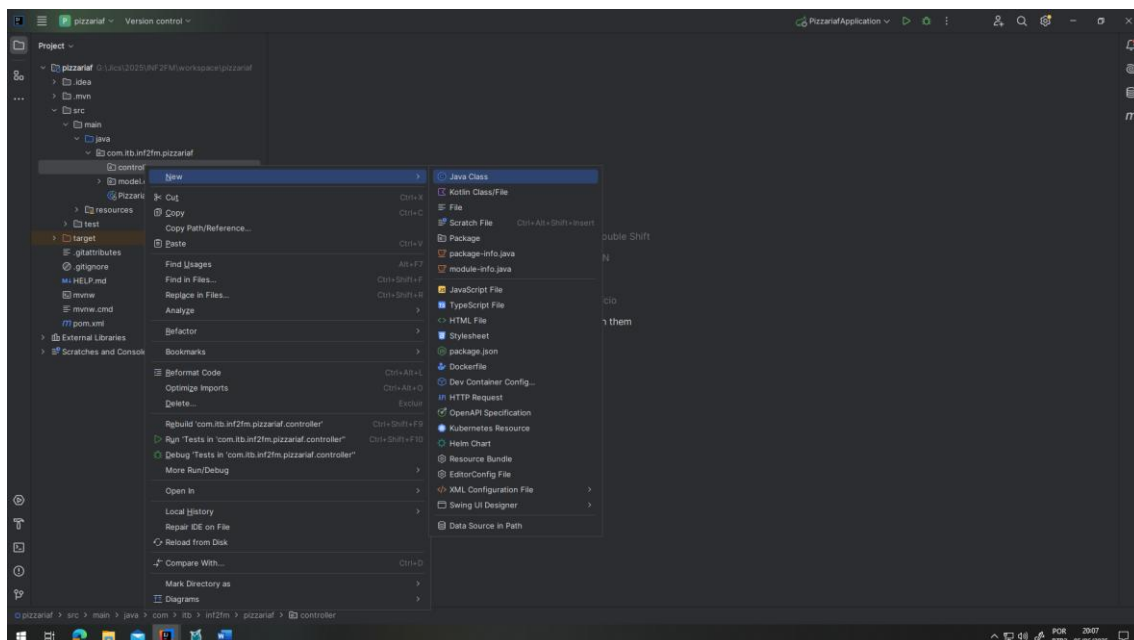
- Complemente com o nome “controller”
- **com.itb.inf2fm.pizzariaf.controller** <ENTER>



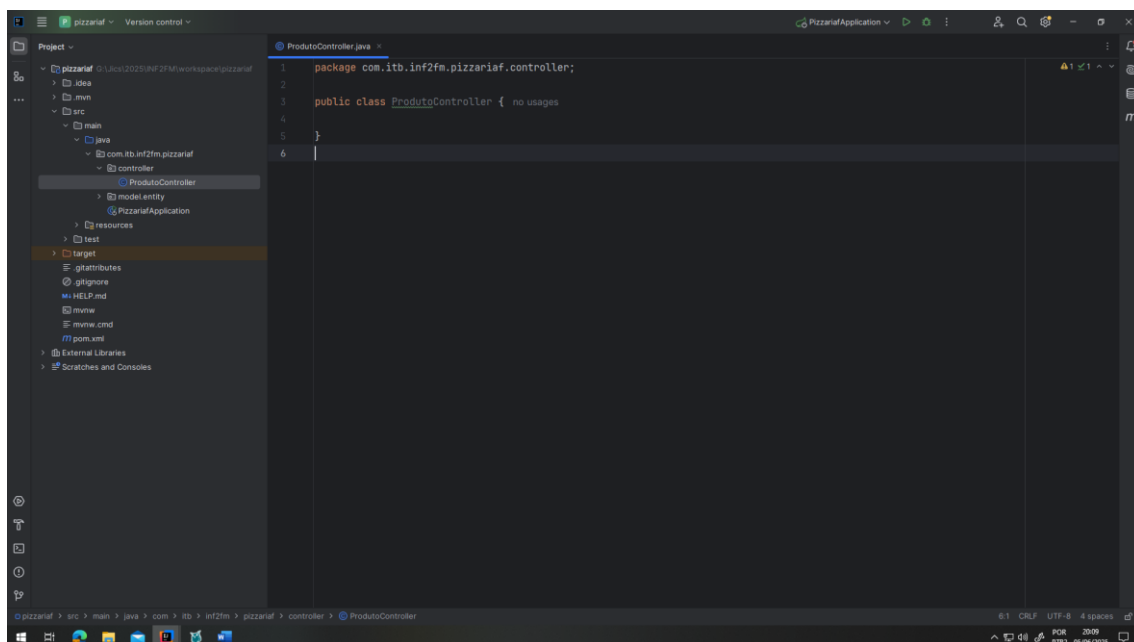
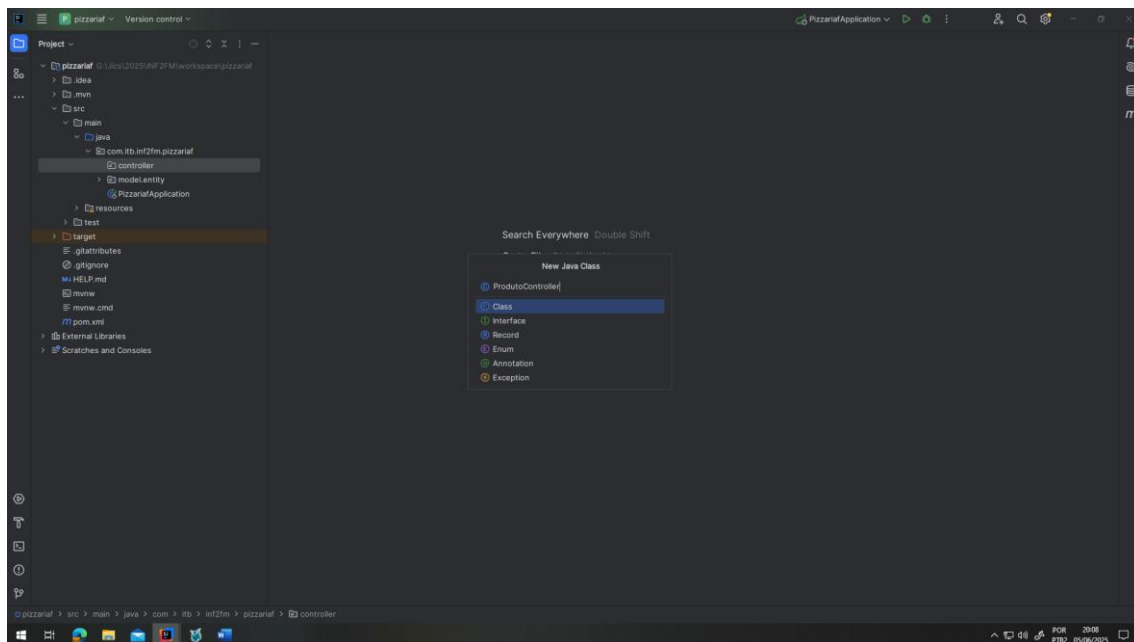


Sobre o pacote controller, crie a classe **ProdutoController**

- Clique com o botão direito sobre controller
- New Java Class

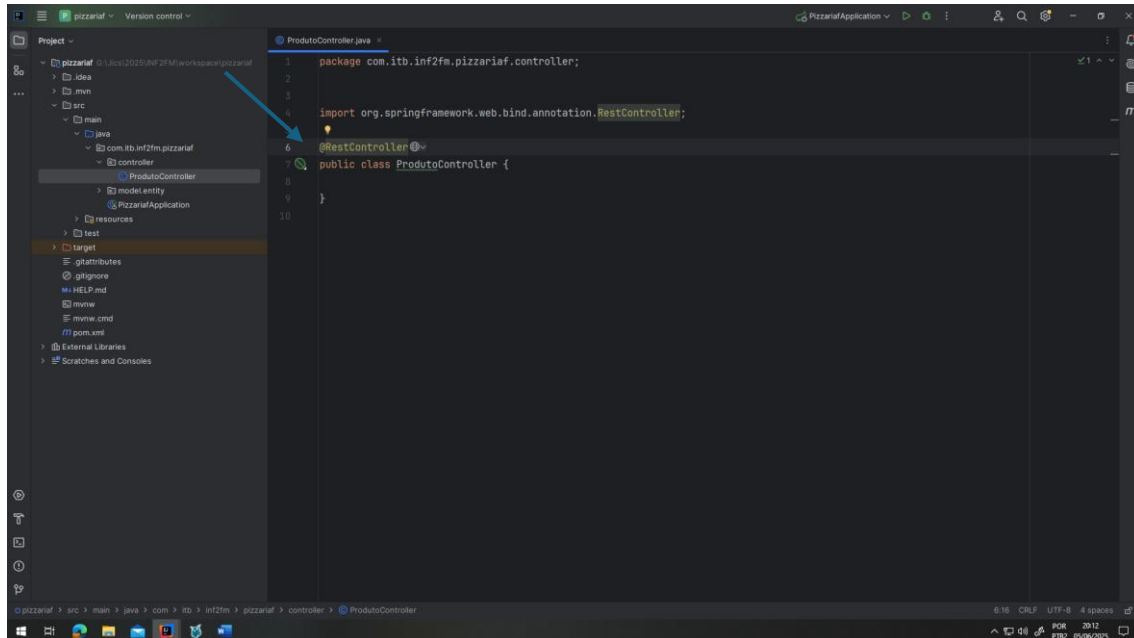


- Escreva “ProdutoController” <ENTER>

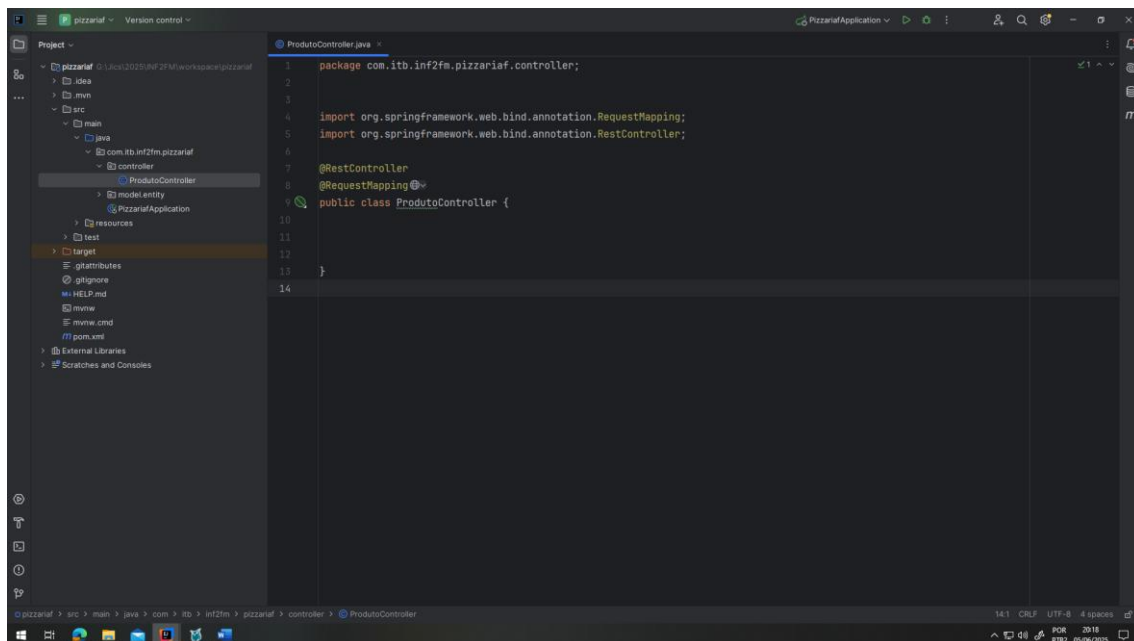


Vamos anotar nossa classe com a annotation **@RestController**

Veja:

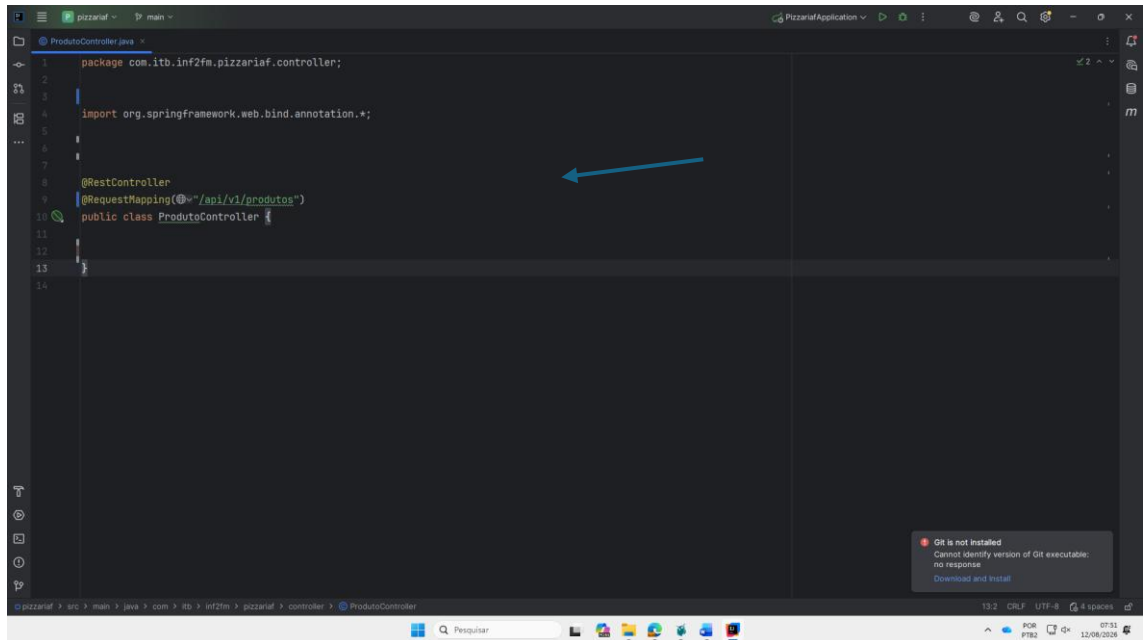


Adicione também a annotation **@RequestMapping**



Complemente a anotação **@RequestMapping** com o endereço principal do “endpoint”

- “/api/v1/produtos”

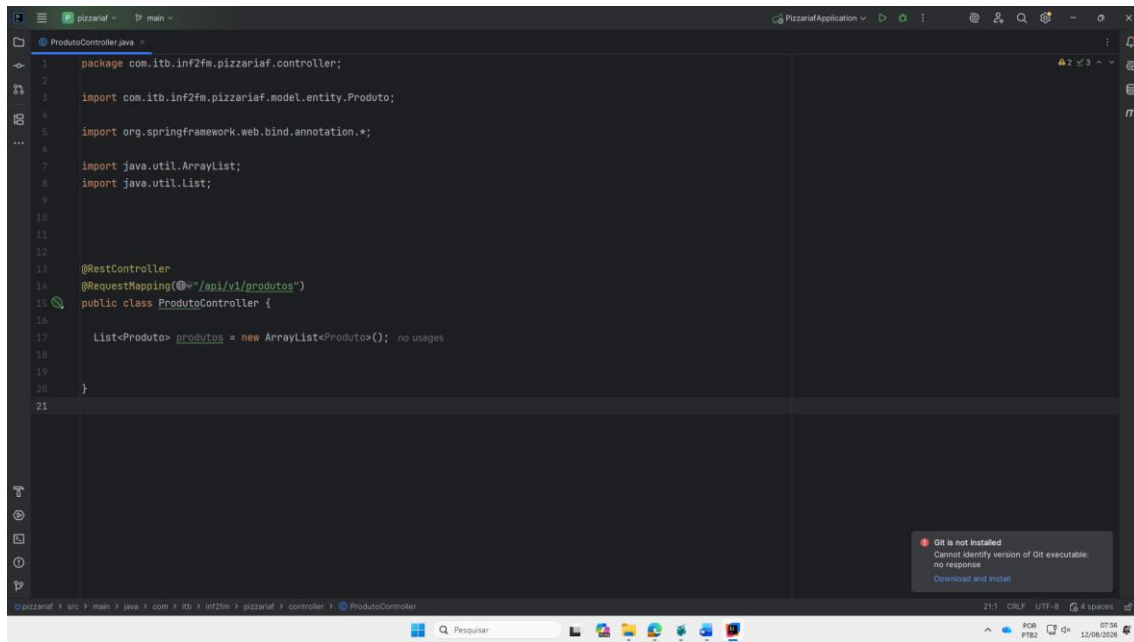


Agora, vamos criar um método para listar todos os produtos cadastrados

“findAll”. Veja:

Por enquanto vamos criar uma lista local para armazenar todos os produtos, ou seja, vamos simular um pequeno banco de dados.

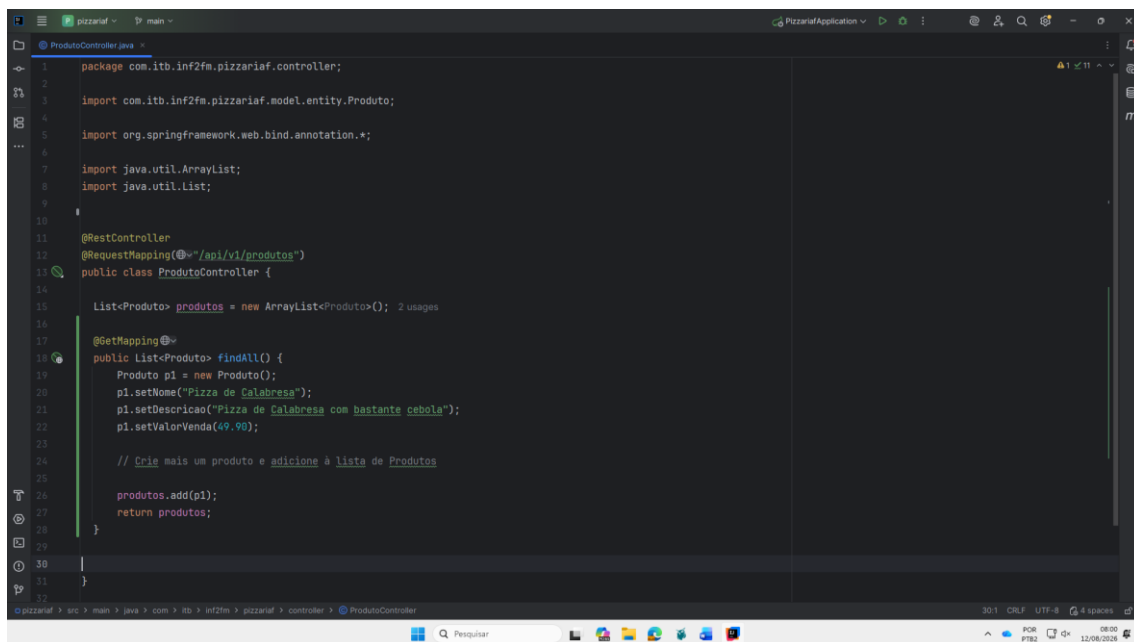
- `List<Produto> produtos = new ArrayList<Produto>();`



```
1 package com.itb.inf2fm.pizzariaf.controller;
2
3 import com.itb.inf2fm.pizzariaf.model.entity.Produto;
4
5 import org.springframework.web.bind.annotation.*;
6
7 import java.util.ArrayList;
8 import java.util.List;
9
10
11
12
13 @RestController
14 @RequestMapping("/api/v1/produtos")
15 public class ProdutoController {
16
17     List<Produto> produtos = new ArrayList<Produto>();
18
19 }
20
21
```

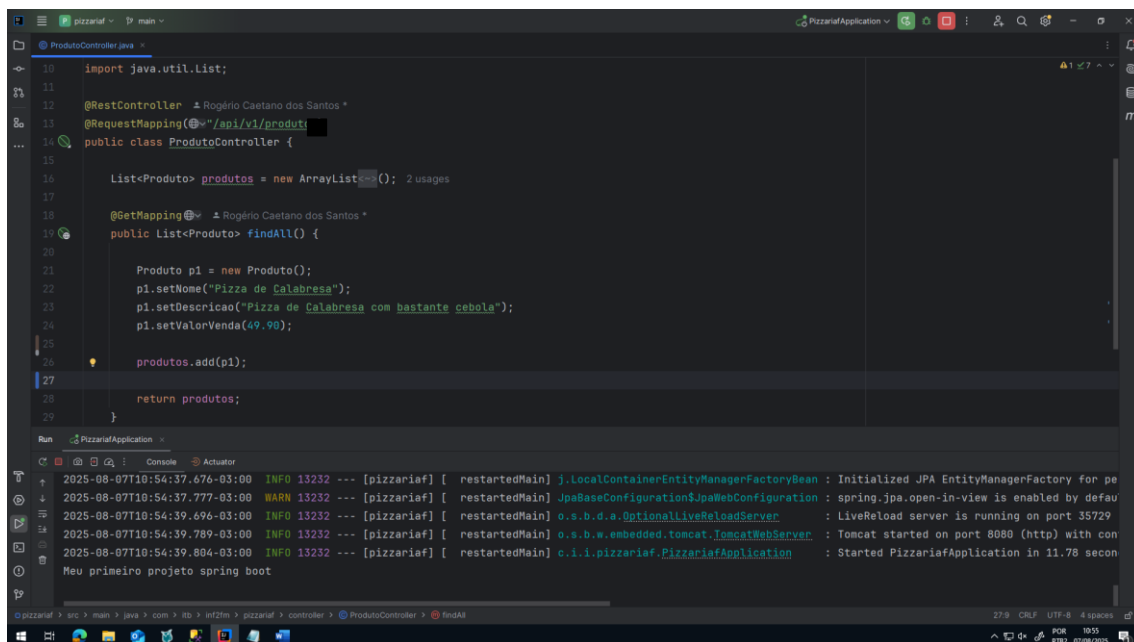
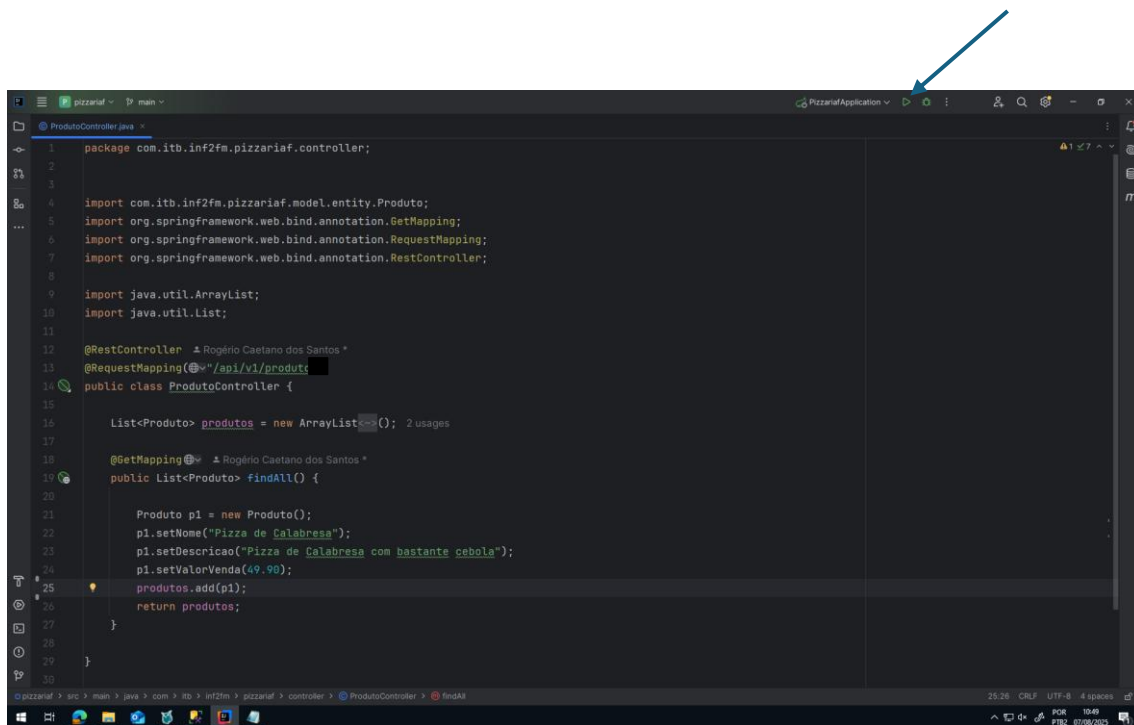
Git is not installed
Cannot identify version of Git executable:
no response
[Download and Install](#)

Simulando um cadastro “antes” da listagem, veja o método:



```
1 package com.itb.inf2fm.pizzariaf.controller;
2
3 import com.itb.inf2fm.pizzariaf.model.entity.Produto;
4
5 import org.springframework.web.bind.annotation.*;
6
7 import java.util.ArrayList;
8 import java.util.List;
9
10
11
12
13 @RestController
14 @RequestMapping("/api/v1/produtos")
15 public class ProdutoController {
16
17     List<Produto> produtos = new ArrayList<Produto>();
18
19     @GetMapping
20     public List<Produto> findAll() {
21         Produto p1 = new Produto();
22         p1.setNome("Pizza de Calabresa");
23         p1.setDescricao("Pizza de Calabresa com bastante cebola");
24         p1.setValorVenda(49.90);
25
26         // Crie mais um produto e adicione à lista de Produtos
27
28         produtos.add(p1);
29         return produtos;
30     }
31 }
32
```

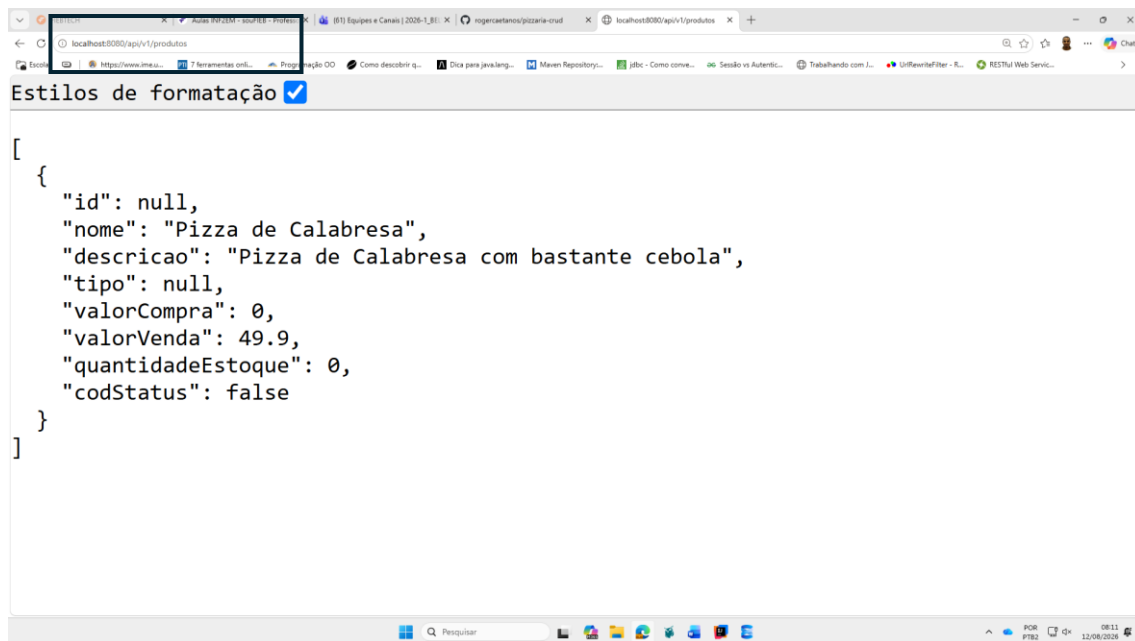
Execute o projeto



Para efetuar o teste, vamos utilizar o browser.

Endereço:

<http://localhost:8080/api/v1/produtos>



Exercício:

- 1-) Em ProdutoController, criar mais um produto e adicione à Lista de Produtos
- 2-) Criar a classe CategoriaController com o método “findAll” responsável em listar todas as categorias.

Obs: Levar em consideração o arquivo ProdutoController na resolução.

- 3-) Criar o repositório local e enviar o projeto para o GitHub.

Desafio

No projeto de TCC criar a mesma estrutura que foi trabalhado no projeto pizzaria

Pacotes:

- model.entity
 1. Criar as classes “Entity” de acordo com o seu DER (Diagrama Entidade Relacionamento) ignorando as “FK”
 2. Criar o métodos: Setter´s | Getter´s
- controller
 1. Escolha uma “Entity” qualquer e crie o “Controller” da referida.
Exemplo: OngController (Controlador da Entidade Ong)

Vamos conectar o nosso banco de dados?

O que é JPA e por que usar com Spring Boot?

O **JPA (Java Persistence API)** é uma especificação para mapeamento objeto-relacional (ORM) em Java. Ele permite manipular dados no banco como objetos Java, facilitando operações como salvar, atualizar e buscar registros.

Já o **Spring Boot** é um framework que simplifica a criação de aplicações Spring, **com jpa**, oferecem uma solução robusta, padronizada e com mínima configuração para acesso a dados

A seguir, algumas anotações referente ao Spring Boot JPA e Spring Data JPA

Annotation	Origem	Aplicação	Explicação
@Entity	JPA	Classe	Marca a classe como uma entidade JPA (será mapeada para uma tabela no banco).
@Table(name = "nome_tabela")	JPA	Classe	Define o nome da tabela no banco (opcional se for igual ao nome da classe).
@Id	JPA	Campo	Indica o campo que será a chave primária.
@GeneratedValue(strategy = ...)	JPA	Campo	Define como a chave primária será gerada (AUTO, IDENTITY, SEQUENCE, TABLE).
@Column(name = "...")	JPA	Campo	Mapeia o campo da classe para uma coluna específica da tabela.
@Transient	JPA	Campo	Ignora o campo (não será persistido no banco).
@Lob	JPA	Campo	Indica que o campo é um tipo grande, como TEXT ou BLOB.
@Enumerated(EnumType.STRING)	JPA	Campo Enum	Salva o enum como texto no banco (por padrão, salva como ordinal).
@Temporal(TemporalType.DATE)	JPA	Campo de data	Define o tipo de dado temporal (DATE, TIME ou TIMESTAMP).
@Embeddable	JPA	Classe	Indica que a classe pode ser embutida em outra entidade (@Embedded).
@Embedded	JPA	Campo	Insere os campos de uma classe @Embeddable nesta entidade.
@ElementCollection	JPA	Campo	Define uma coleção de elementos básicos ou @Embeddable.
@Access(AccessType.FIELD)	JPA	Classe/Campo	Define se o JPA acessa via campo ou via getter/setter.

Relacionamentos JPA			
@OneToOne	JPA	Relacionamento	Relacionamento 1-para-1 entre entidades. Pode ter mappedBy, cascade, fetch.
@OneToMany	JPA	Relacionamento	Um para muitos (ex: um cliente tem muitos pedidos).
@ManyToOne	JPA	Relacionamento	Muitos para um (ex: muitos pedidos pertencem a um cliente).
@ManyToMany	JPA	Relacionamento	Muitos para muitos (ex: alunos e cursos).
@JoinColumn(name = "...")	JPA	Relacionamento	Define a coluna de junção para @OneToOne ou @ManyToOne.
@JoinTable	JPA	Relacionamento	Tabela de junção usada em relacionamentos @ManyToMany.
@Fetch(FetchMode.JOIN)	Hibernate	Relacionamento	Define como o dado será carregado (JOIN, SELECT, SUBSELECT).
Comportamentos de Repositórios - Spring Data JPA			
@Repository	Spring	Interface/classe	Marca uma classe como repositório do Spring.
@Query("JPQL ou SQL")	Spring Data JPA	Método	Permite definir uma consulta JPQL personalizada.
@Modifying	Spring Data JPA	Método	Usado com @Query para INSERT, UPDATE, DELETE.
@Transactional	Spring	Classe ou método	Garante que o método execute em uma transação.
@EnableJpaRepositories	Spring	Configuração	Ativa o uso de repositórios JPA. Pode configurar basePackages.
@EntityScan(basePackages = "...")	Spring Boot	Configuração	Indica onde estão suas entidades JPA.
@EnableTransactionManagement	Spring	Configuração	Habilita o gerenciamento de transações no Spring.
Auditoria e Eventos			
@CreatedDate	Spring Data JPA	Campo	Define o campo como data de criação (precisa de auditoria habilitada).
@LastModifiedDate	Spring Data JPA	Campo	Define o campo como última data de modificação.
@CreatedBy	Spring Data JPA	Campo	Armazena quem criou (usado com auditoria).
@LastModifiedBy	Spring Data JPA	Campo	Armazena quem modificou por último.
@EntityListeners(AuditingEntityListener.class)	Spring Data JPA	Classe	Ativa as anotações de auditoria em uma entidade.

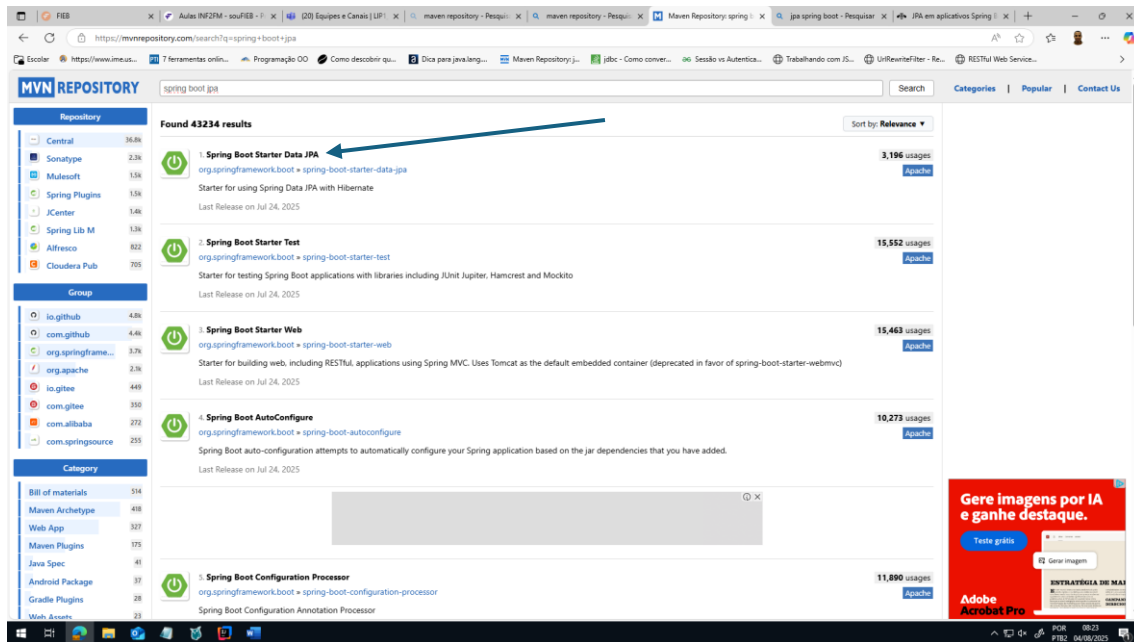
Para trabalhar com JPA no Spring, precisamos instalar algumas dependências a saber:

- [Spring Boot Starter Data JPA](#)
- [Microsoft JDBC Driver For SQL Server](#)

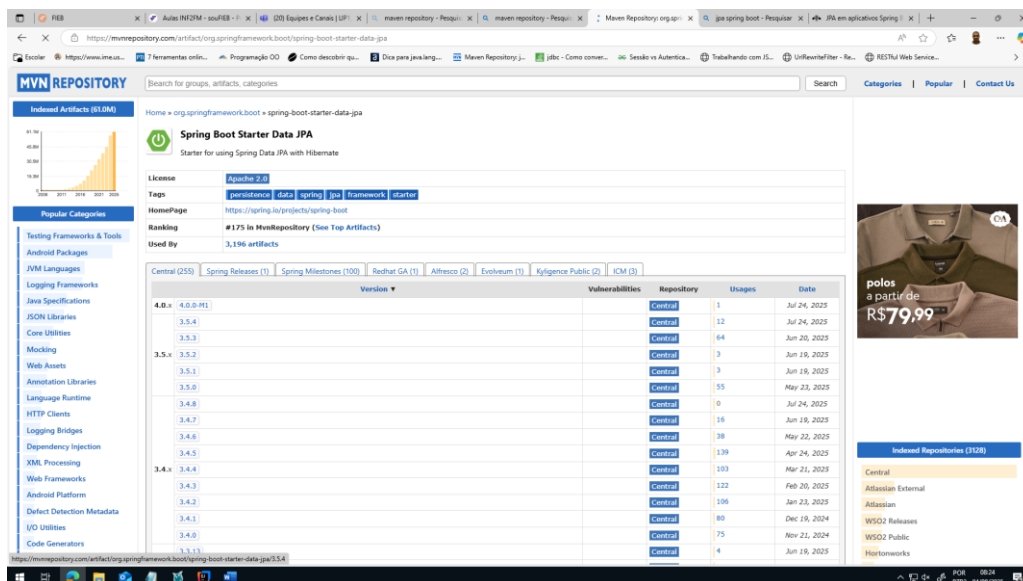
É possível baixar as dependências no site oficial do Maven:

<https://mvnrepository.com/>

Veja:



Após pesquisa clique no link indicado



Escolha qualquer versão

The screenshot shows the Maven Repository page for 'Spring Boot Starter Data JPA'. The 'Versions' column lists several versions, with '3.5.4' highlighted. A blue arrow points to this version. The 'Ranking' is #173 in the 'Spring Boot' category. The 'Used By' section shows 3,196 artifacts.

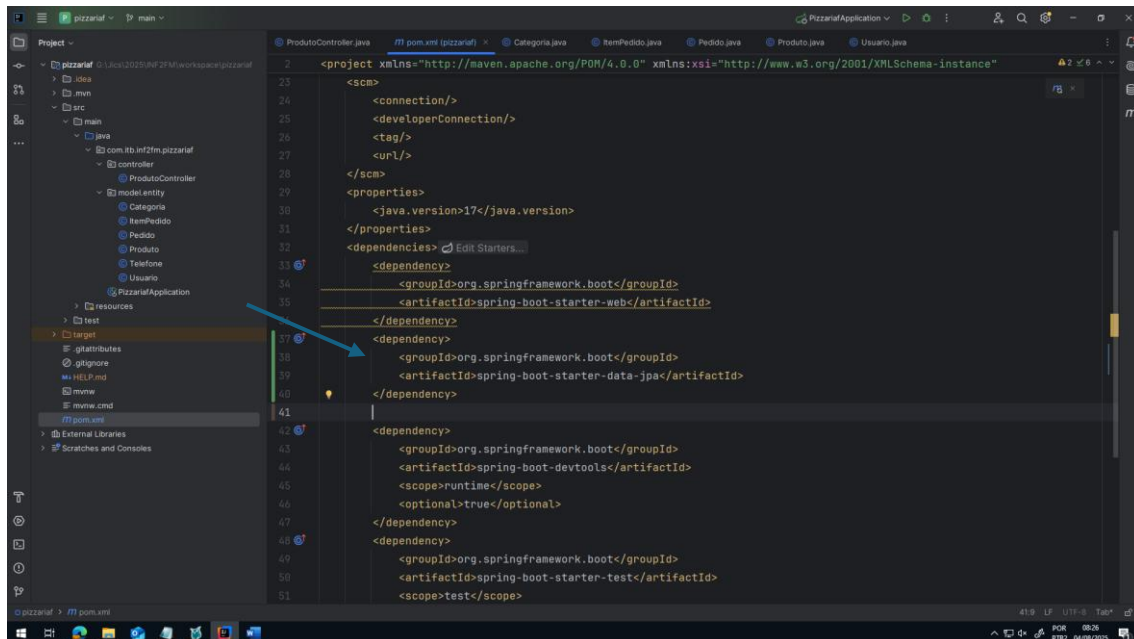
The screenshot shows the Maven Repository page for 'Spring Boot Starter Data JPA' with version '3.5.4' selected. The 'New Version' section shows '3.5.4-RC1'. The 'Compile Dependencies' section is visible at the bottom.

Copiar o código xml

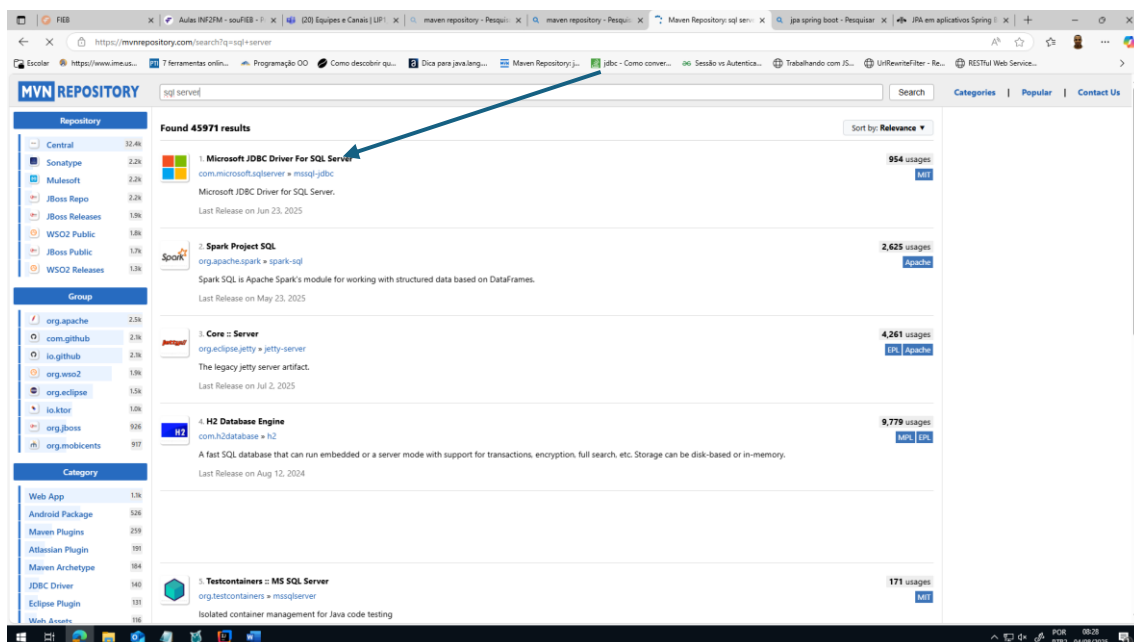
The screenshot shows the Maven Repository page for 'Spring Boot Starter Data JPA' with version '3.5.4' selected. The 'Compile Dependencies' section is visible at the bottom. A blue arrow points to the '3.5.4' version in the 'Versions' column.

Abrir o arquivo pom.xml do projeto pizzariaf e colar no setor de “dependencies” e

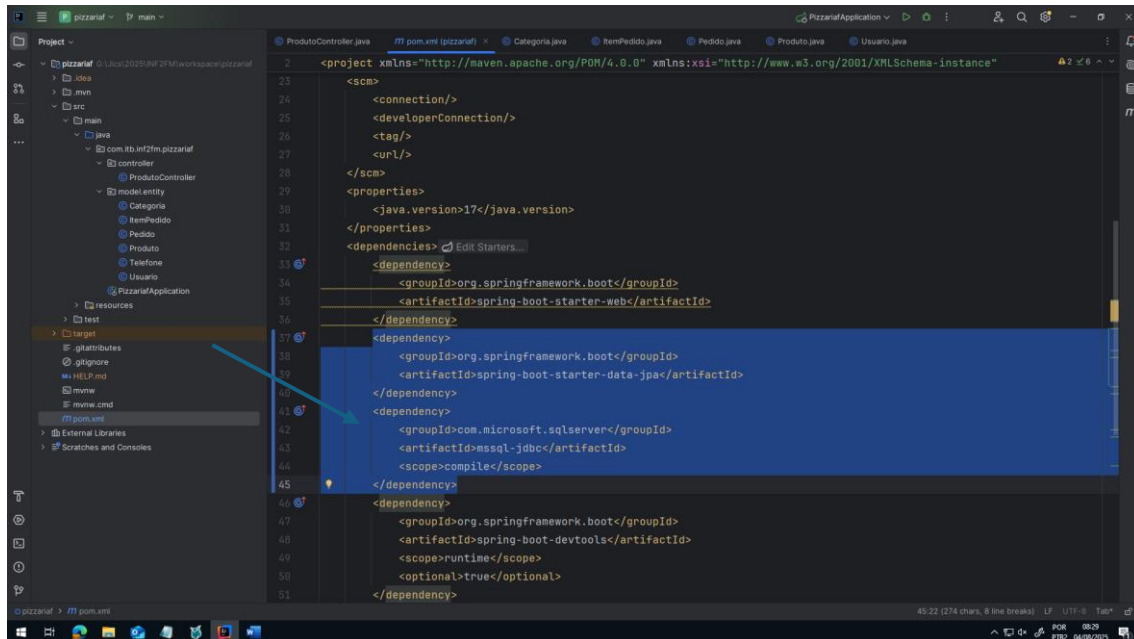
APAGUE A TAG <version> </version>



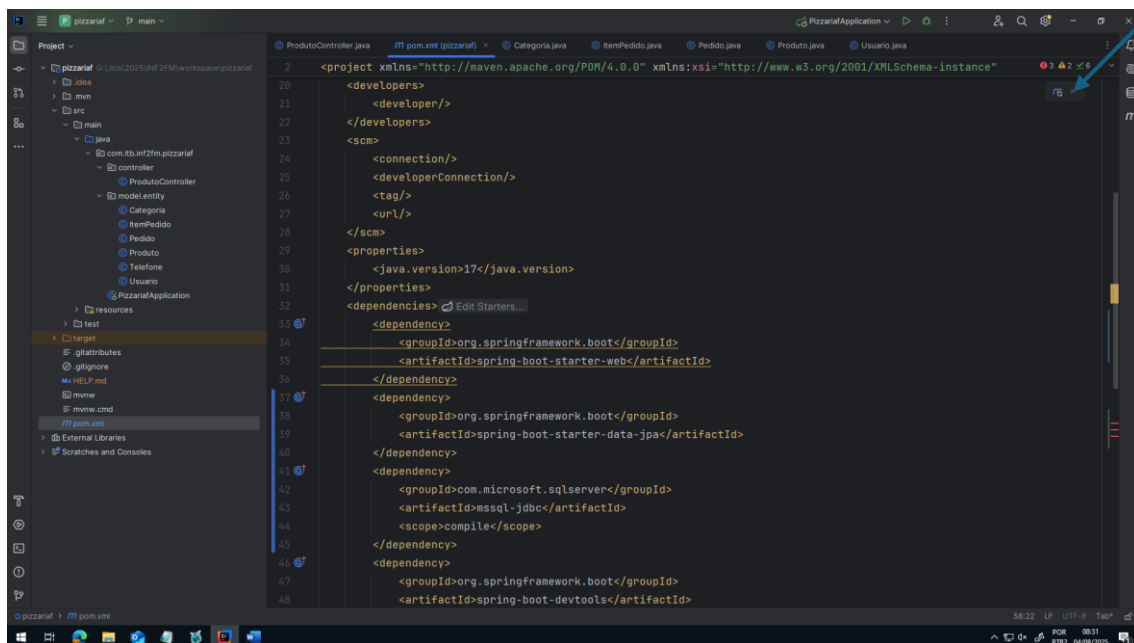
Voltar no site do maven e repetir o processo para a dependência do SQL Server



Veja o resultado final:

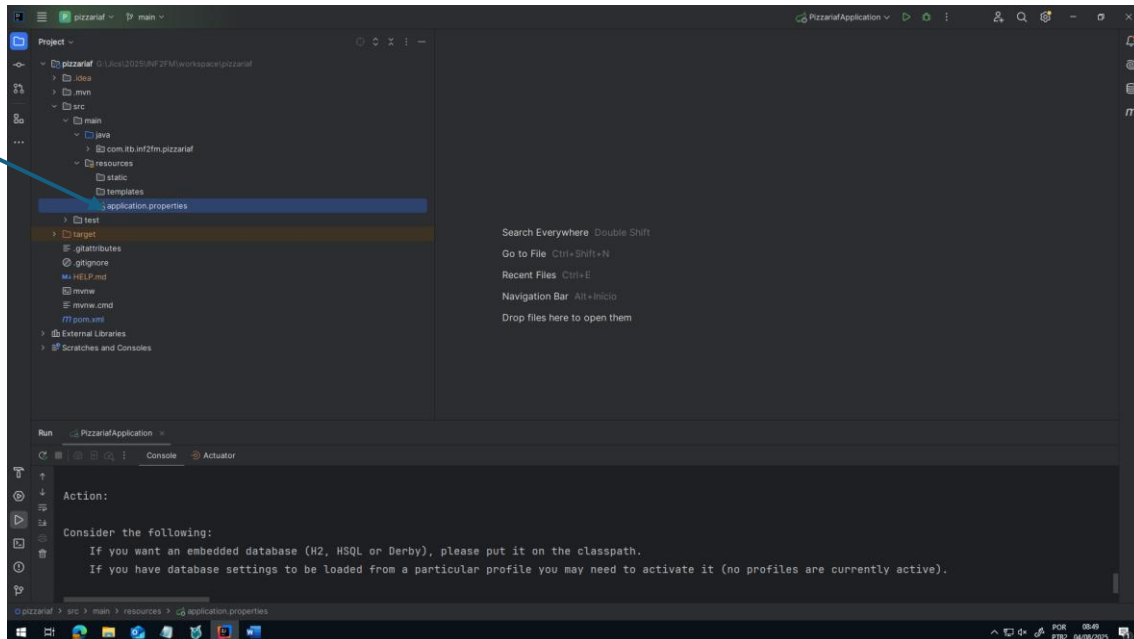


Clique no “m” na parte superior direito do arquivo para baixar as dependências

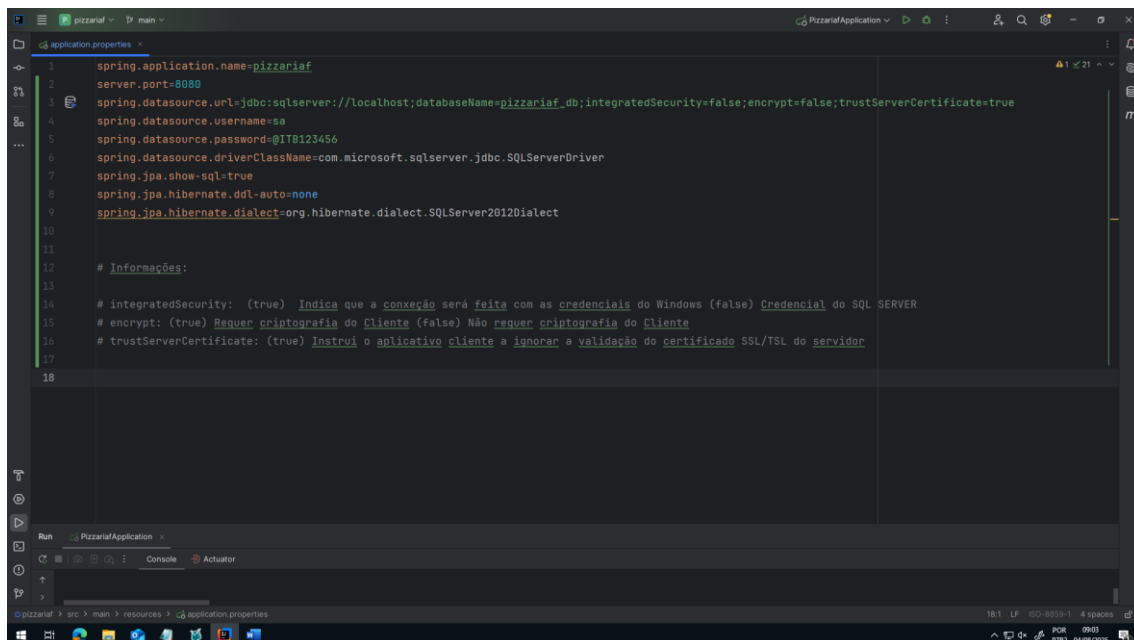


Ao executar o projeto, perceberemos que o projeto apresentará um erro na execução, normal, pois carece da configuração da conexão com o banco de dados:

Abrir o arquivo “**application.properties**” dentro da pasta resources.



Digite as seguintes informações:

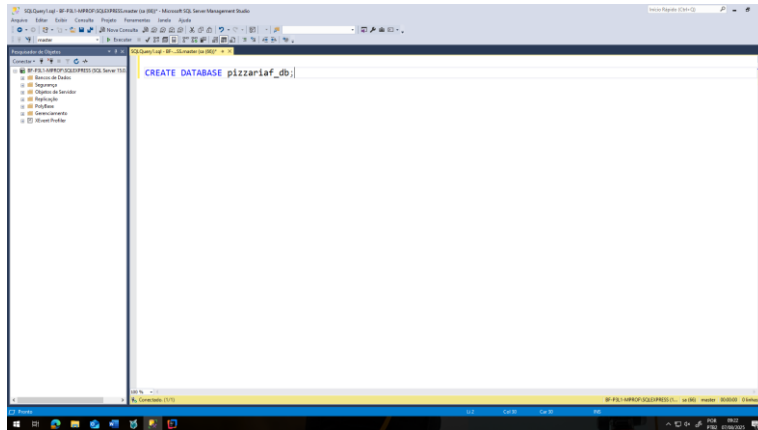


Agora, o objetivo é executar o projeto “sem erros.

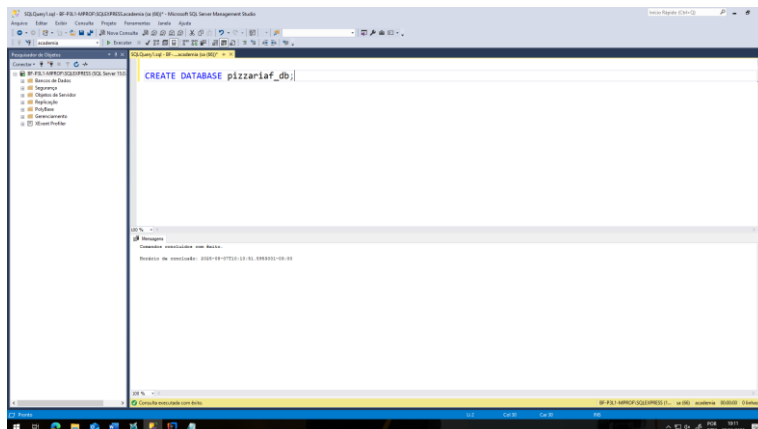
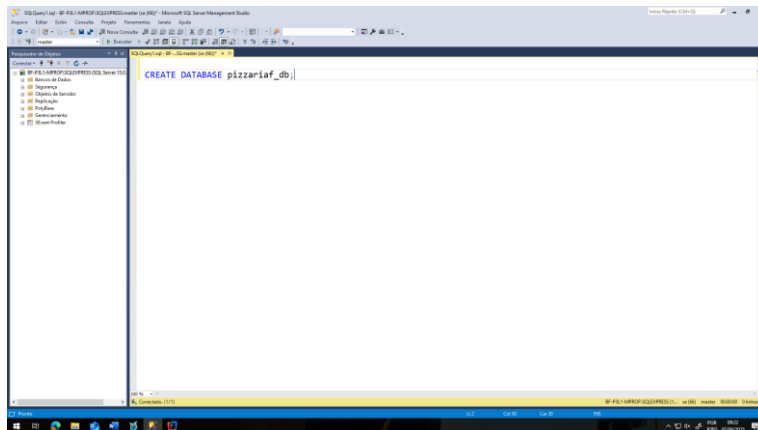
Para tal, vamos criar o nosso banco de dados. Lembrando que optamos pelo “none” em **spring-jpa-hibernate.ddl-auto**

Abrir seu gerenciador de Banco de dados “SQL Server Management Studio” e criar o banco de dados **pizzariaf_db**

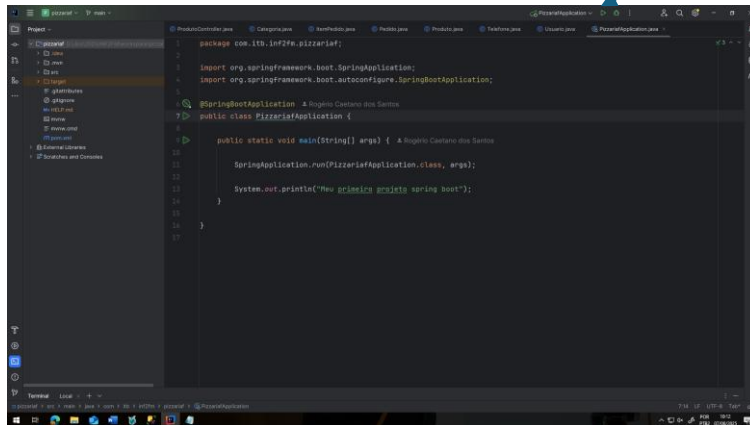
Veja:



F5 para executar o script

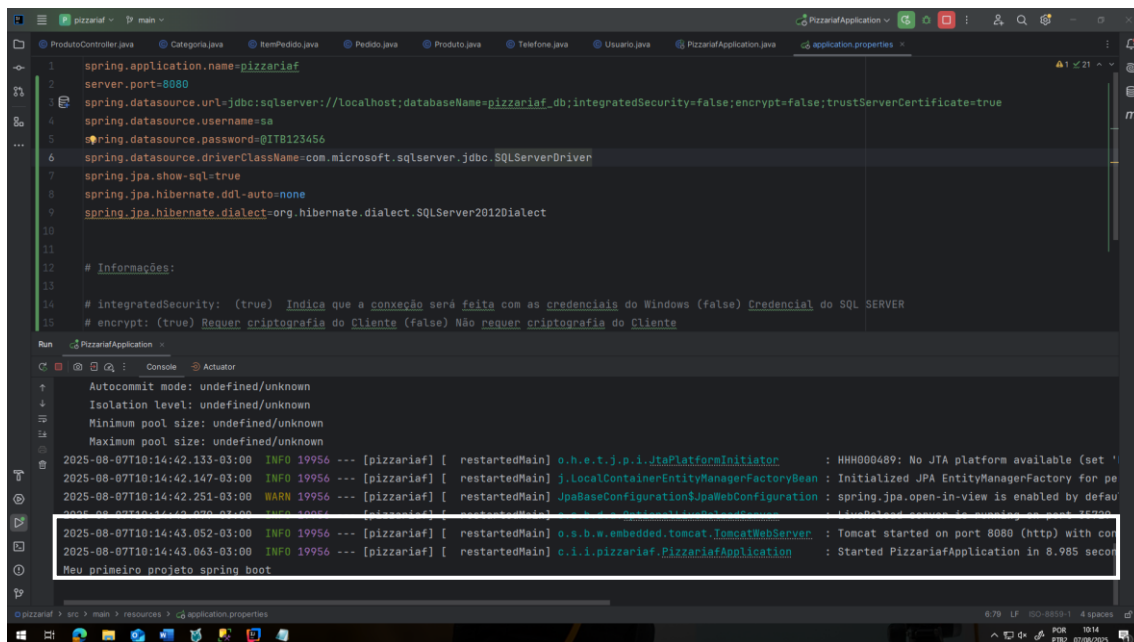


Volte para o seu projeto pizzeria e Execute



O servidor deve subir sem problemas!

Veja:

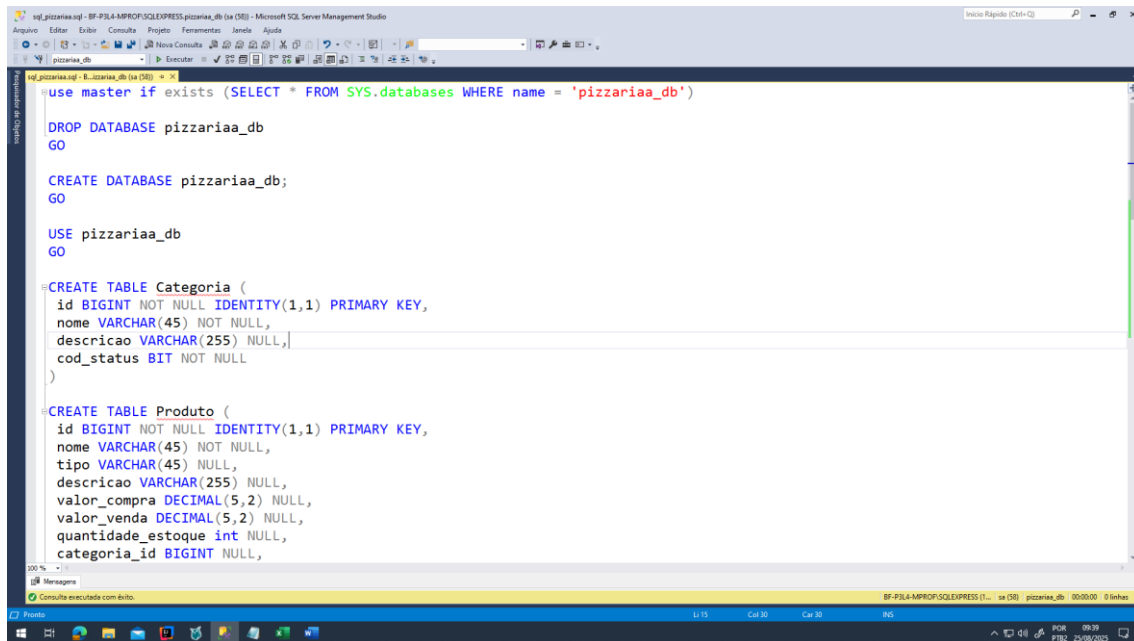


Exercícios:

1-) Replicar no TCC o que foi executado no projeto pizzaria, ou seja, instalação e configuração do JPA com JDBC do SQL SERVER

Agora vamos prosseguir na criação do nosso banco de dados

Veja o script:



```

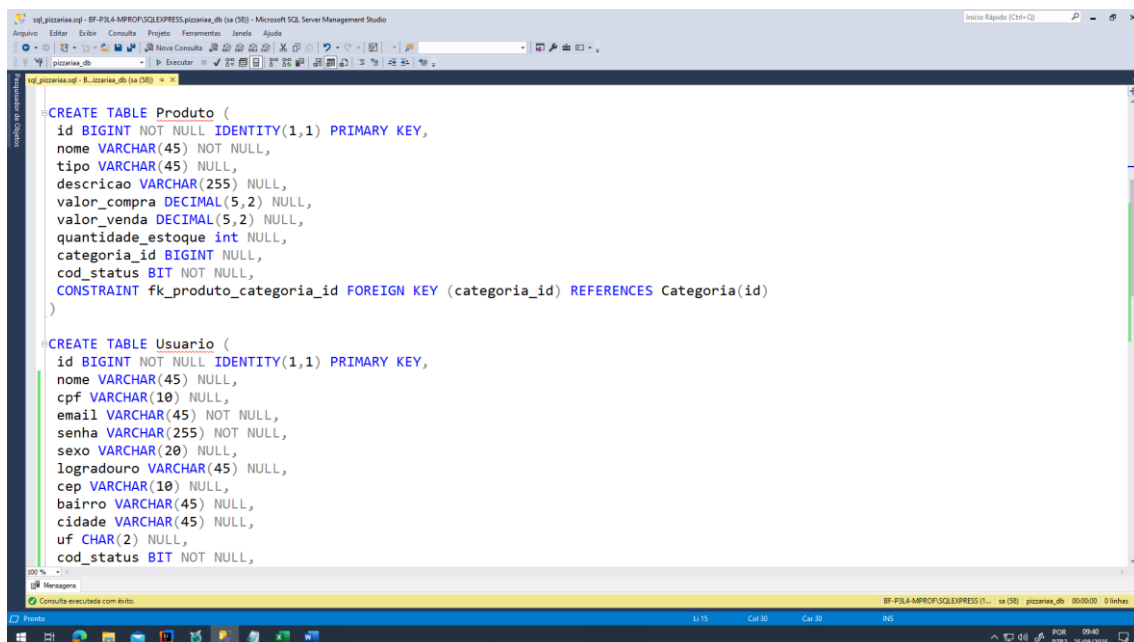
use master if exists (SELECT * FROM SYS.databases WHERE name = 'pizzariaa_db')
DROP DATABASE pizzariaa_db
GO

CREATE DATABASE pizzariaa_db;
GO

USE pizzariaa_db
GO

CREATE TABLE Categoria (
    id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(45) NOT NULL,
    descricao VARCHAR(255) NULL,
    cod_status BIT NOT NULL
)

CREATE TABLE Produto (
    id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(45) NOT NULL,
    tipo VARCHAR(45) NULL,
    descricao VARCHAR(255) NULL,
    valor_compra DECIMAL(5,2) NULL,
    valor_venda DECIMAL(5,2) NULL,
    quantidade_estoque int NULL,
    categoria_id BIGINT NULL,
    cod_status BIT NOT NULL,
    CONSTRAINT fk_produto_categoria_id FOREIGN KEY (categoria_id) REFERENCES Categoria(id)
)
  
```



```

CREATE TABLE Usuario (
    id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(45) NULL,
    cpf VARCHAR(10) NULL,
    email VARCHAR(45) NOT NULL,
    senha VARCHAR(255) NOT NULL,
    sexo VARCHAR(20) NULL,
    logradouro VARCHAR(45) NULL,
    cep VARCHAR(10) NULL,
    bairro VARCHAR(45) NULL,
    cidade VARCHAR(45) NULL,
    uf CHAR(2) NULL,
    cod_status BIT NOT NULL,
)
  
```

```

CREATE TABLE Usuario (
  id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
  nome VARCHAR(45) NULL,
  cpf VARCHAR(10) NULL,
  email VARCHAR(45) NOT NULL,
  senha VARCHAR(255) NOT NULL,
  sexo VARCHAR(20) NULL,
  logradouro VARCHAR(45) NULL,
  cep VARCHAR(10) NULL,
  bairro VARCHAR(45) NULL,
  cidade VARCHAR(45) NULL,
  uf CHAR(2) NULL,
  cod_status BIT NOT NULL,
)

CREATE TABLE Telefone (
  id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
  ddd VARCHAR(3) NOT NULL,
  numero VARCHAR(15) NOT NULL,
  cod_status BIT NOT NULL,
  usuario_id BIGINT NOT NULL,
  CONSTRAINT fk_telefone_usuario_id FOREIGN KEY (usuario_id) REFERENCES Usuario(id)
)

CREATE TABLE Pedido (

```

```

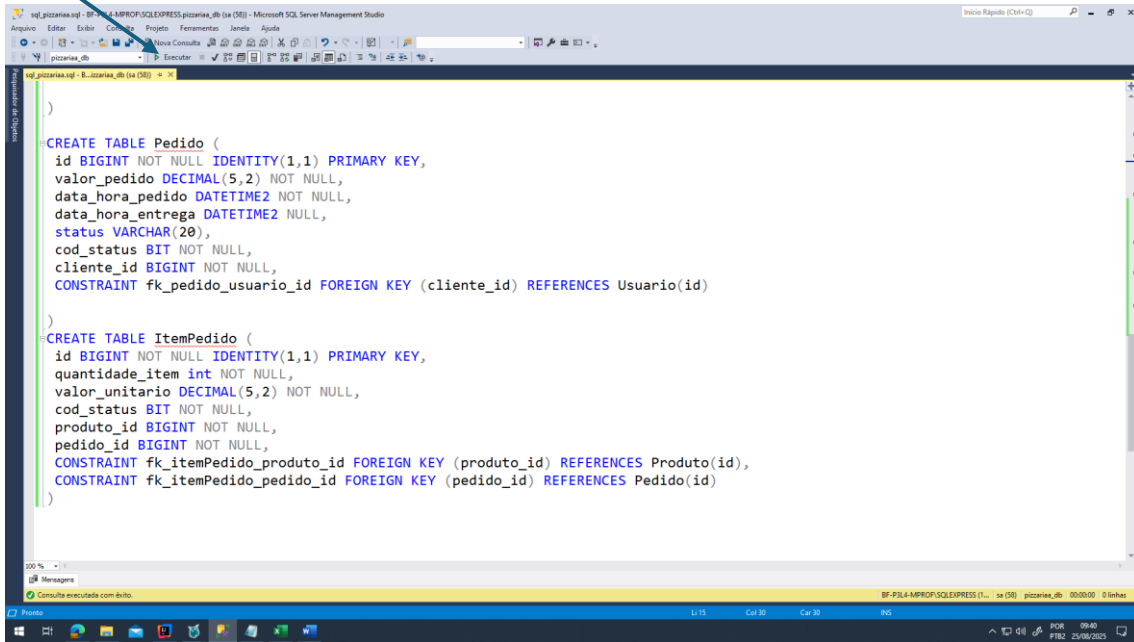
)

CREATE TABLE Pedido (
  id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
  valor_pedido DECIMAL(5,2) NOT NULL,
  data_hora_pedido DATETIME2 NOT NULL,
  data_hora_entrega DATETIME2 NULL,
  status VARCHAR(20),
  cod_status BIT NOT NULL,
  cliente_id BIGINT NOT NULL,
  CONSTRAINT fk_pedido_usuario_id FOREIGN KEY (cliente_id) REFERENCES Usuario(id)
)

CREATE TABLE ItemPedido (
  id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
  quantidade_item int NOT NULL,
  valor_unitario DECIMAL(5,2) NOT NULL,
  cod_status BIT NOT NULL,
  produto_id BIGINT NOT NULL,
  pedido_id BIGINT NOT NULL,
  CONSTRAINT fk_itemPedido_produto_id FOREIGN KEY (produto_id) REFERENCES Produto(id),
  CONSTRAINT fk_itemPedido_pedido_id FOREIGN KEY (pedido_id) REFERENCES Pedido(id)
)

```

Podemos executar o script para o tratamento de eventuais erros



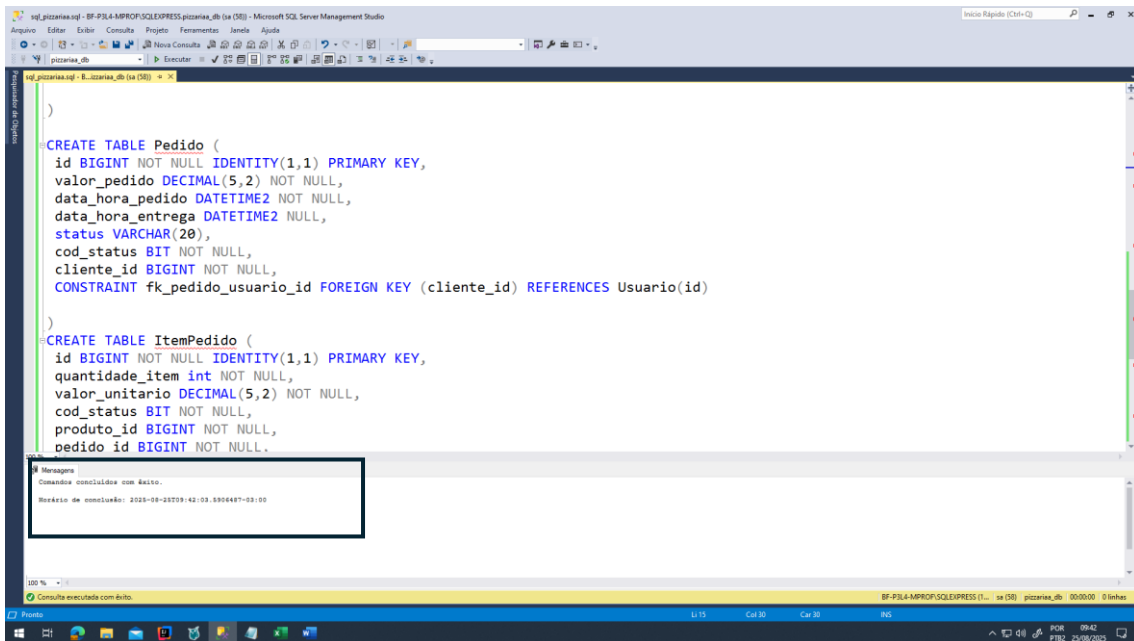
```

)

CREATE TABLE Pedido (
    id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    valor_pedido DECIMAL(5,2) NOT NULL,
    data_hora_pedido DATETIME2 NOT NULL,
    data_hora_entrega DATETIME2 NULL,
    status VARCHAR(20),
    cod_status BIT NOT NULL,
    cliente_id BIGINT NOT NULL,
    CONSTRAINT fk_pedido_usuario_id FOREIGN KEY (cliente_id) REFERENCES Usuario(id)
)

CREATE TABLE ItemPedido (
    id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    quantidade_item int NOT NULL,
    valor_unitario DECIMAL(5,2) NOT NULL,
    cod_status BIT NOT NULL,
    produto_id BIGINT NOT NULL,
    pedido_id BIGINT NOT NULL,
    CONSTRAINT fk_itemPedido_produto_id FOREIGN KEY (produto_id) REFERENCES Produto(id),
    CONSTRAINT fk_itemPedido_pedido_id FOREIGN KEY (pedido_id) REFERENCES Pedido(id)
)

```



```

)

CREATE TABLE Pedido (
    id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    valor_pedido DECIMAL(5,2) NOT NULL,
    data_hora_pedido DATETIME2 NOT NULL,
    data_hora_entrega DATETIME2 NULL,
    status VARCHAR(20),
    cod_status BIT NOT NULL,
    cliente_id BIGINT NOT NULL,
    CONSTRAINT fk_pedido_usuario_id FOREIGN KEY (cliente_id) REFERENCES Usuario(id)
)

CREATE TABLE ItemPedido (
    id BIGINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    quantidade_item int NOT NULL,
    valor_unitario DECIMAL(5,2) NOT NULL,
    cod_status BIT NOT NULL,
    produto_id BIGINT NOT NULL,
    pedido_id BIGINT NOT NULL,
    CONSTRAINT fk_itemPedido_produto_id FOREIGN KEY (produto_id) REFERENCES Produto(id),
    CONSTRAINT fk_itemPedido_pedido_id FOREIGN KEY (pedido_id) REFERENCES Pedido(id)
)

```

Messages

Comando concluído com êxito.

Horário de execução: 2025-08-25T09:43:03.590467-03:00

Com o banco de dados criado, vamos prosseguir na Anotações das nossas Entidades, para tal vamos usar o quadro das páginas 25 e 26.

Vamos começar pela Entity **Produto**

Veja:

Obs: Confira a propriedade “length” de @Column de acordo com seu banco de dados, pode ter divergências com o código do professor abaixo.

```

package com.itb.inf2fm.pizzariaf.model.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "Produto") // Quando o nome da classe for igual ao nome da tabela, o @Table é facultativo
public class Produto {

    @Id // PK
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO-INCREMENTO SEQUENCIAL 1 e 1
    private Long id;

    @Column(length = 100, nullable = false) // length : tamanho MAX, nullable : false (NOT NULL) true: (NULL) 2 usages
    private String nome;

    @Column(length = 250, nullable = true) // valores padrão, aparece na cor 'CINZA', logo é opcional 2 usages
    private String descricao;

    @Column(length = 45, nullable = true) 2 usages
    private String tipo;

    @Column(columnDefinition = "DECIMAL(5,2)", nullable = true) 2 usages
    private double valorCompra;

    @Column(columnDefinition = "DECIMAL(5,2)", nullable = true) 2 usages
    private double valorVenda;

    @Column(nullable = true) 2 usages
    private int quantidadeEstoque;

    private boolean codStatus; 2 usages

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }
}

```

Exercícios

1-) Vamos prosseguir nas demais anotações do Model Entity

- Categoria
- Usuario
- Telefone
- Pedido
- ItemPedido

Correção

Categoria

```

1 package com.itb.inf2fm.pizzariaf.model.entity;
2
3 import jakarta.persistence.*;
4
5 @Entity no usages 1 Rogério Caetano dos Santos *
6 @Table(name = "Categoria")
7 public class Categoria {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12    @Column(length = 45, nullable = false) 2 usages
13    private String nome;
14    @Column(length = 250, nullable = true) 2 usages
15    private String descricao;
16    private boolean codStatus; 2 usages
17
18    public Long getId() { no usages 1 Rogério Caetano dos Santos
19        return id;
20    }
21
22    public void setId(Long id) { this.id = id; }
23
24    public String getNome() { return nome; }
25
26    public void setNome(String nome) { this.nome = nome; }
27
28    public String getDescricao() { return descricao; }
29
30 }

```

Telefone

```

1 package com.itb.inf2fm.pizzariaf.model.entity;
2
3 import jakarta.persistence.*;
4
5 @Entity no usages 1 Rogério Caetano dos Santos *
6 @Table(name = "Telefone")
7 public class Telefone {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12    @Column(length = 5, nullable = false) 2 usages
13    private String ddd;
14    @Column(length = 15, nullable = false) 2 usages
15    private String numero;
16    private boolean codStatus; 2 usages
17
18    public Long getId() { return id; }
19
20    public void setId(Long id) { this.id = id; }
21
22    public String getDdd() { return ddd; }
23
24    public void setDdd(String ddd) { this.ddd = ddd; }
25
26    public String getNumero() { return numero; }
27
28    public void setNumero(String numero) { this.numero = numero; }
29
30 }

```

Pedido

```

1 package com.itb.inf2fm.pizzariaf.model.entity;
2
3
4 import jakarta.persistence.*;
5
6 import java.time.LocalDateTime;
7
8 @Entity no usages 1 Rogério Caetano dos Santos
9 @Table(name = "Pedido")
10 public class Pedido {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15     @Column(columnDefinition = "DECIMAL(5,2)", nullable = false) 2 usages
16     private double valorPedido;
17     @Column(nullable = false) 2 usages
18     private LocalDateTime dataHoraPedido;
19     @Column(nullable = true) 2 usages
20     private LocalDateTime dataHoraEntrega;
21     @Column(length = 20, nullable = false) 2 usages
22     private String status;
23     private boolean codStatus; 2 usages
24
25     public Long getId() { return id; }
26
27     public void setId(Long id) { this.id = id; }
28
29     public double getValorPedido() { return valorPedido; }
30
31
32
33
34
35

```

Run PizzariaApplication
 pizzariaf > src > main > java > com > itb > inf2fm > pizzariaf > model > entity > Pedido > @dataHoraPedido

ItemPedido

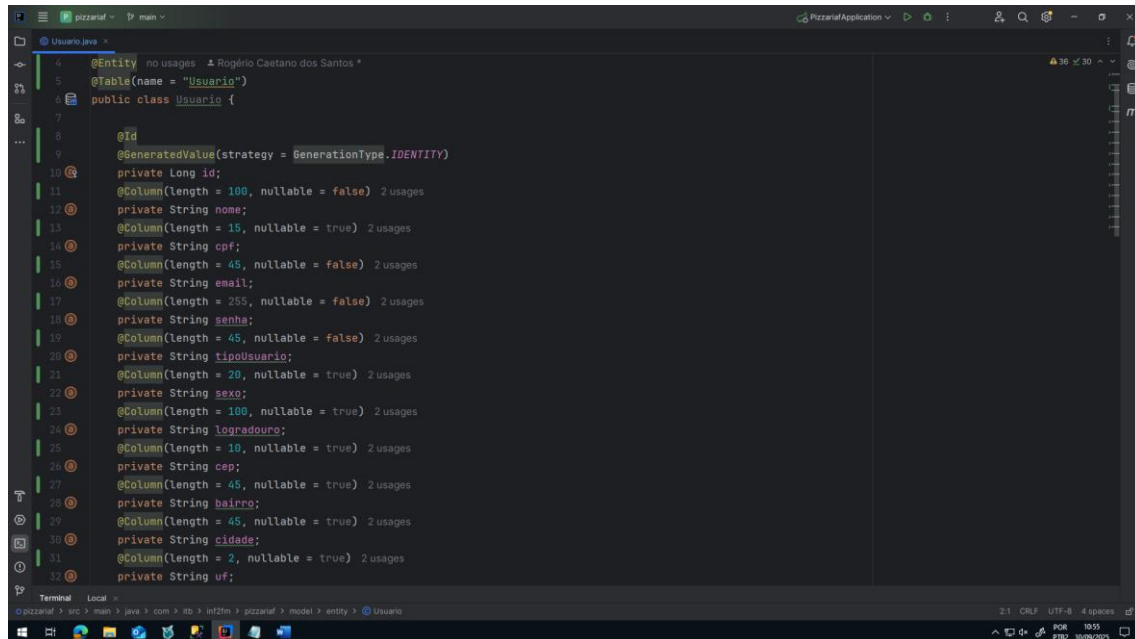
```

1 package com.itb.inf2fm.pizzariaf.model.entity;
2
3
4 import jakarta.persistence.*;
5
6 @Entity no usages 1 Rogério Caetano dos Santos
7 @Table(name = "ItemPedido")
8 public class ItemPedido {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13     @Column(nullable = false) 2 usages
14     private int quantidadeItem;
15     @Column(columnDefinition = "DECIMAL(5,2)", nullable = false) 2 usages
16     private double valorUnitario;
17     private boolean codStatus; 2 usages
18
19     public Long getId() { no usages 1 Rogério Caetano dos Santos
20         return id;
21     }
22
23     public void setId(Long id) { no usages 1 Rogério Caetano dos Santos
24         this.id = id;
25     }
26
27     public int getQuantidadeItem() { return quantidadeItem; }
28
29     public void setQuantidadeItem(int quantidadeItem) { this.quantidadeItem = quantidadeItem; }
30
31
32
33
34
35

```

Run PizzariaApplication
 pizzariaf > src > main > java > com > itb > inf2fm > pizzariaf > model > entity > ItemPedido > @getId

Usuario



Após término, atualizar o git / github

Nota: Caso não tenha o repositório, fazer o exercício 3 da página 24

Abrir o Terminal ...

git add . <<ENTER>>

git commit -m "Anotações principais de Model Entity concluída, configuração da conexão com o banco SQL Server" <<ENTER>>

git push <<ENTER>>

Model Repository

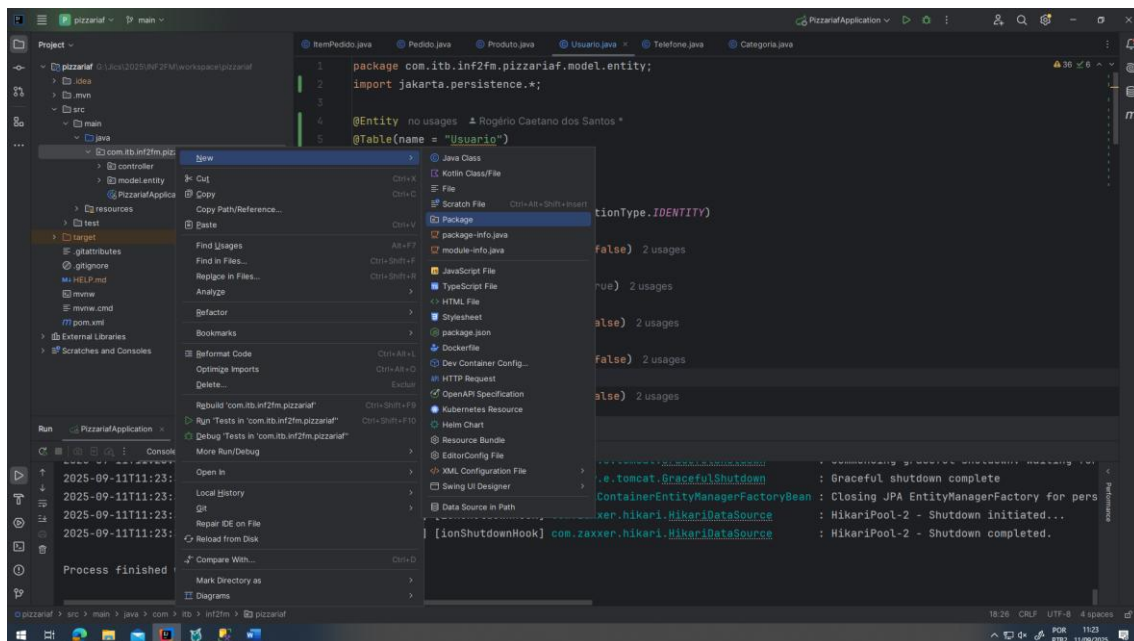
Agora, vamos prosseguir na camada Model Repository, esta camada será responsável em executar as ações (INSERT, SELECT, UPDATE e DELETE) com o banco de dados.

Para tal, precisaremos utilizar as anotações contidas na seção:

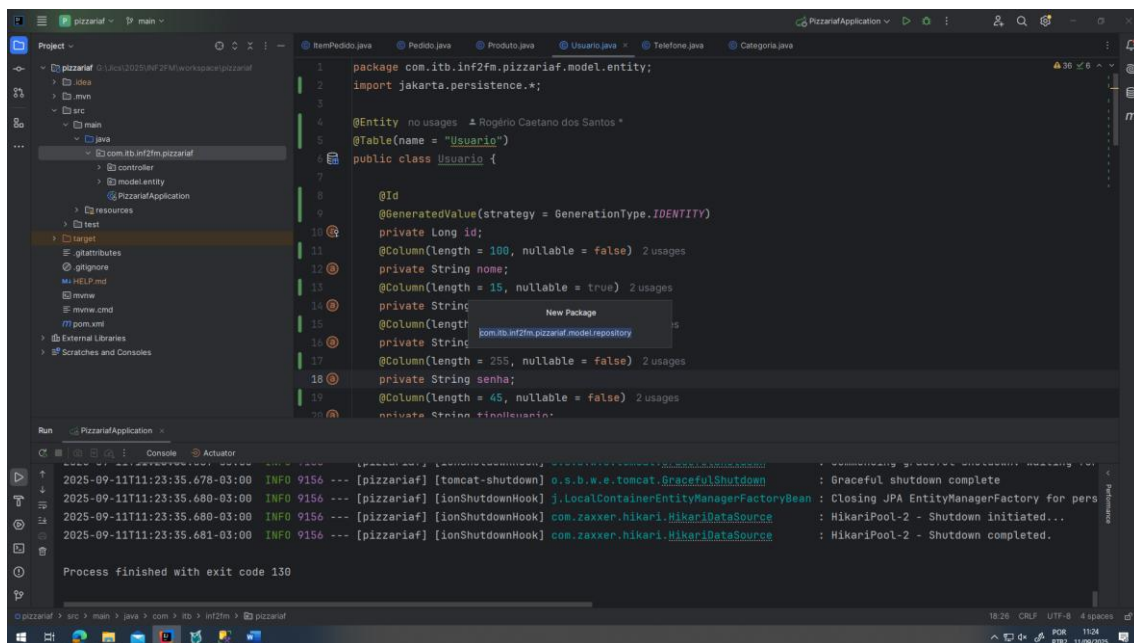
Comportamentos de Repositórios – Spring Data JPA (página 26)

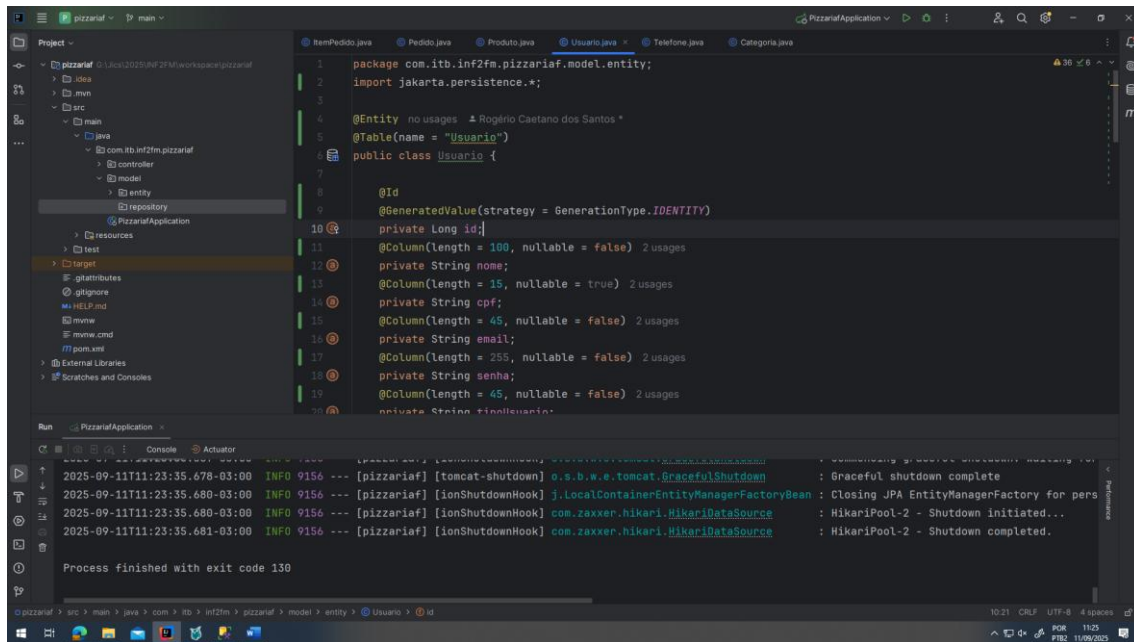
Sobre o pacote principal, criar o pacote “model.repository”

Veja:

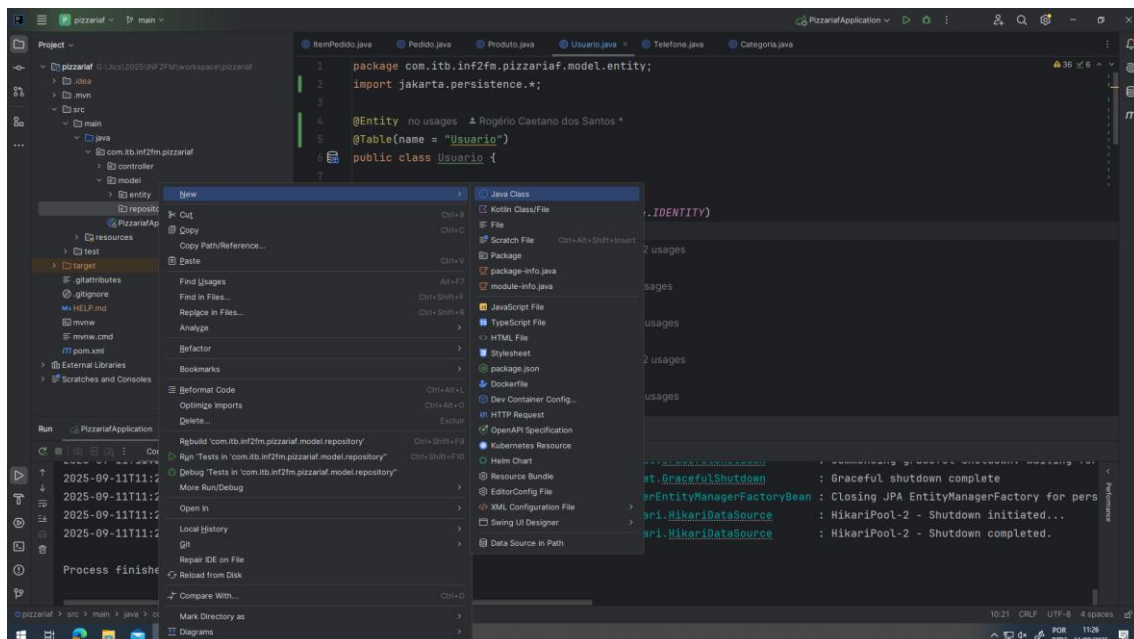


complemente com: model.repository

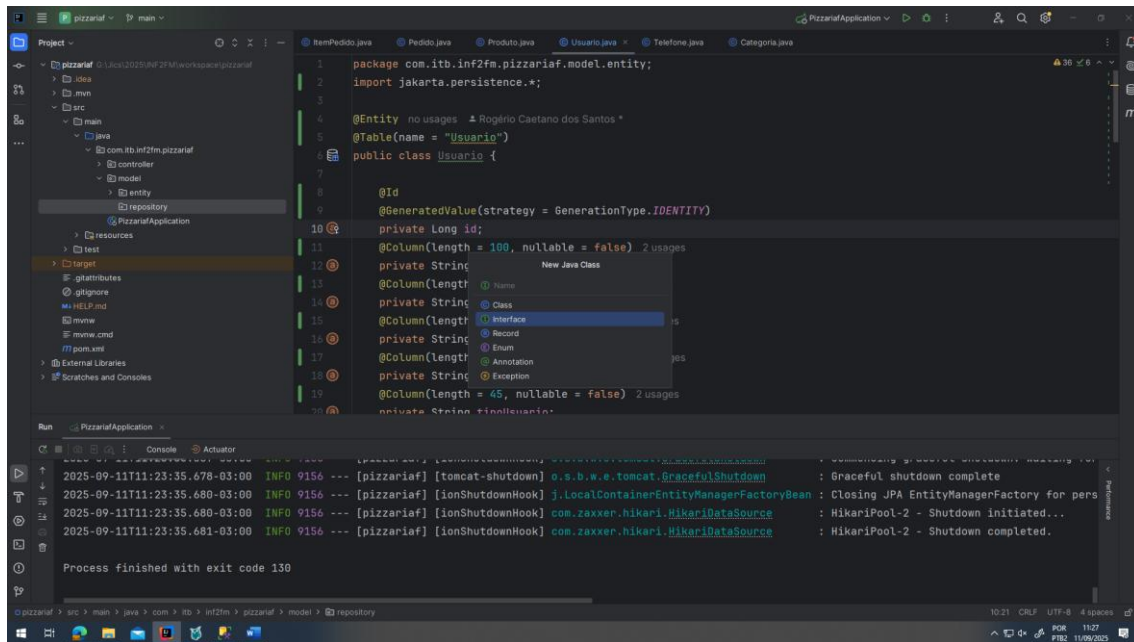




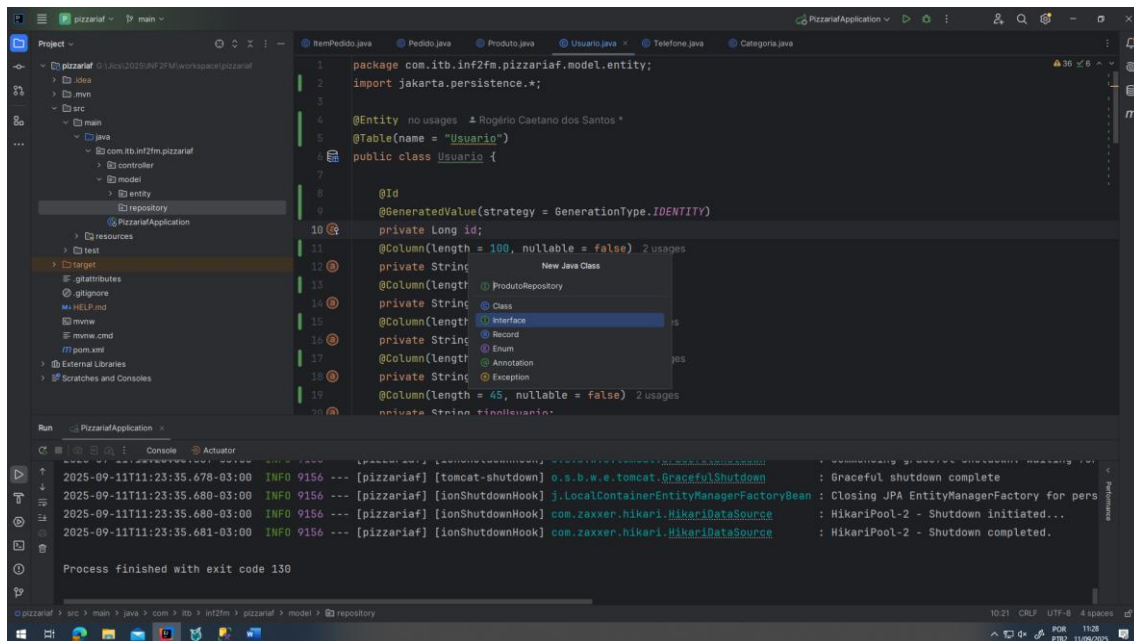
Botão direito sobre o “repositor” -> new Java Class

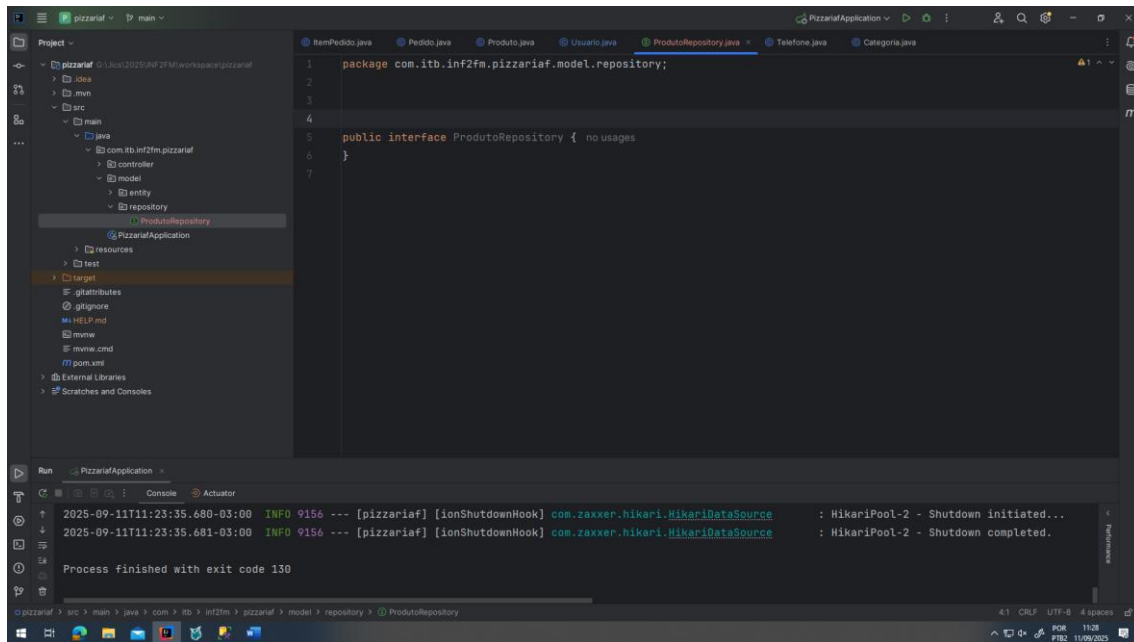


Escolha “Interface”



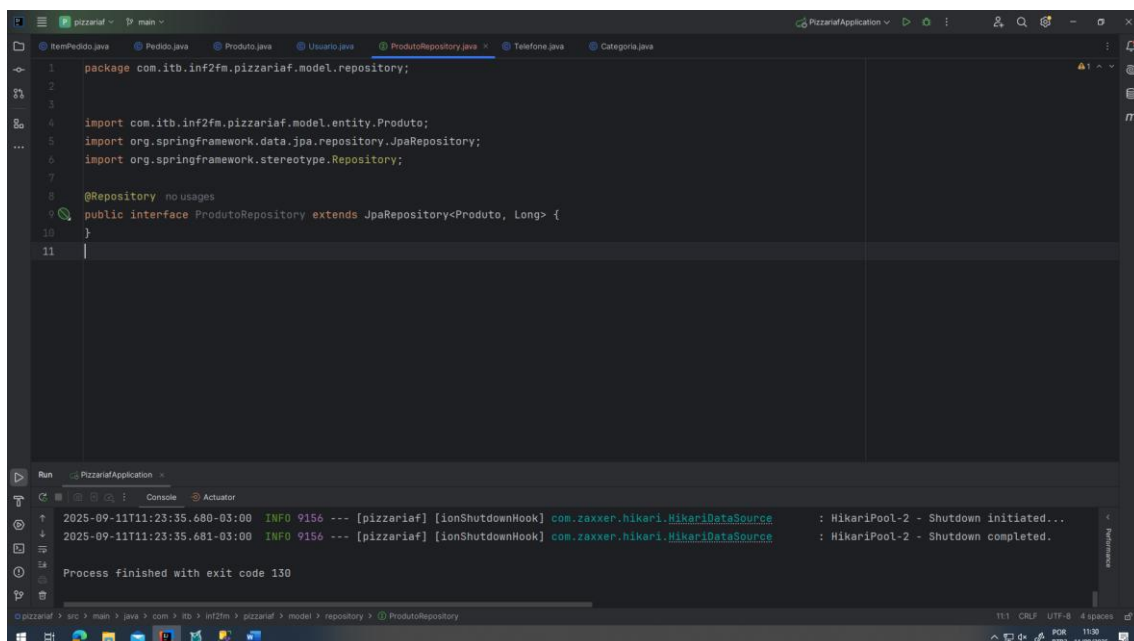
Preencha o nome com “ProdutoRepository”





Prossiga nas anotações

Veja:



public interface ProdutoRepository extends JpaRepository < Produto , Long > {
}

Entidade Manipulada

Tipo da chave primária

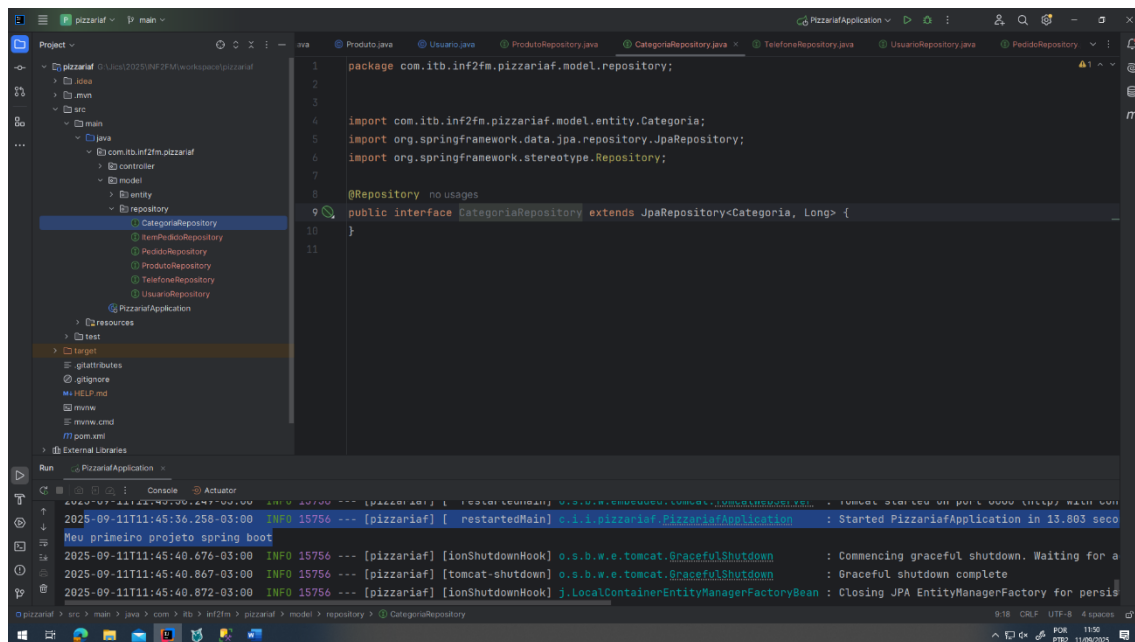
Exercícios

1-) Prossiga nos demais Repositórios:

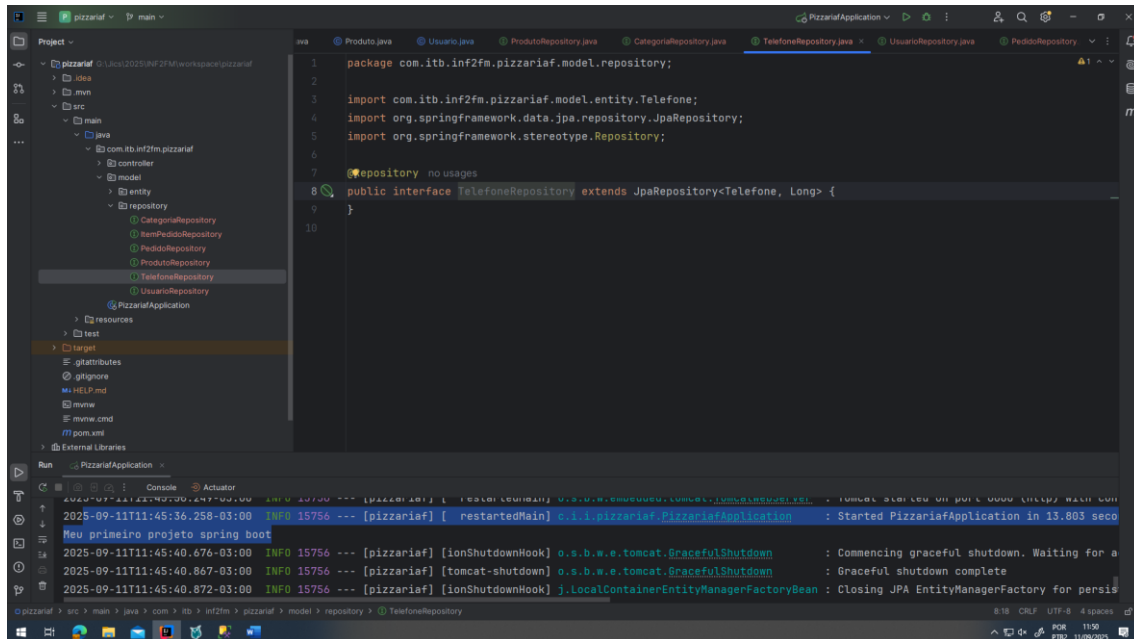
- CategoriaRepository
- TelefoneRepository
- UsuarioRepository
- PedidoRepository
- ItemPedidoRepository

Correção:

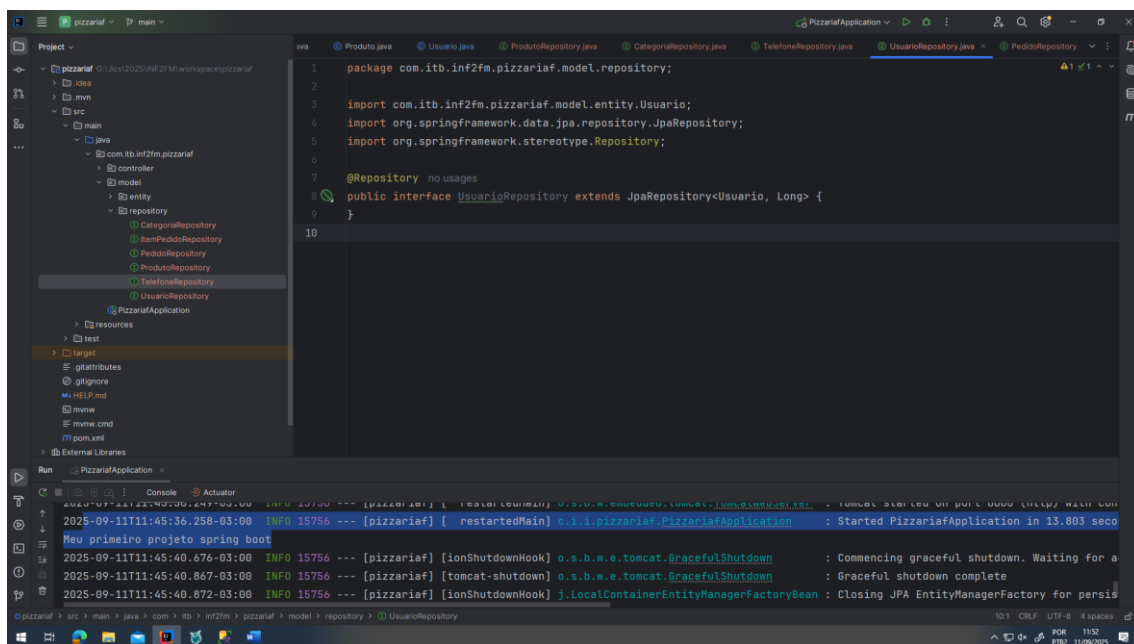
CategoriaRepository



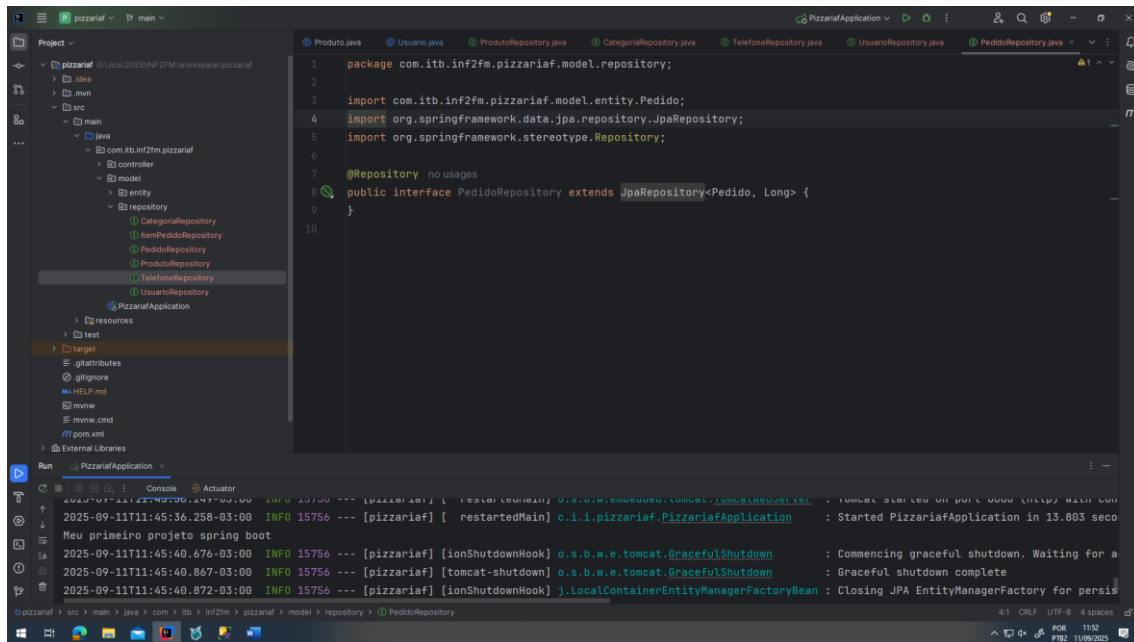
TelefoneRepository



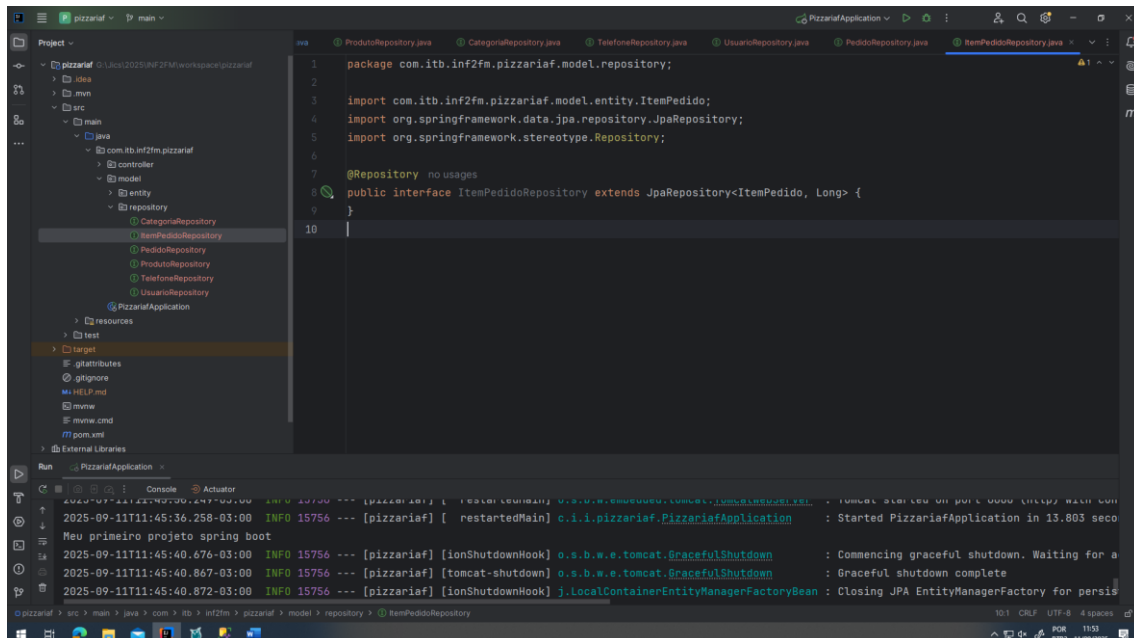
UsuarioRepository



PedidoRepository



ItemPedidoRepository



Após término, atualizar o git / github

Nota: Caso não tenha o repositório, fazer o exercício 3 da página 24

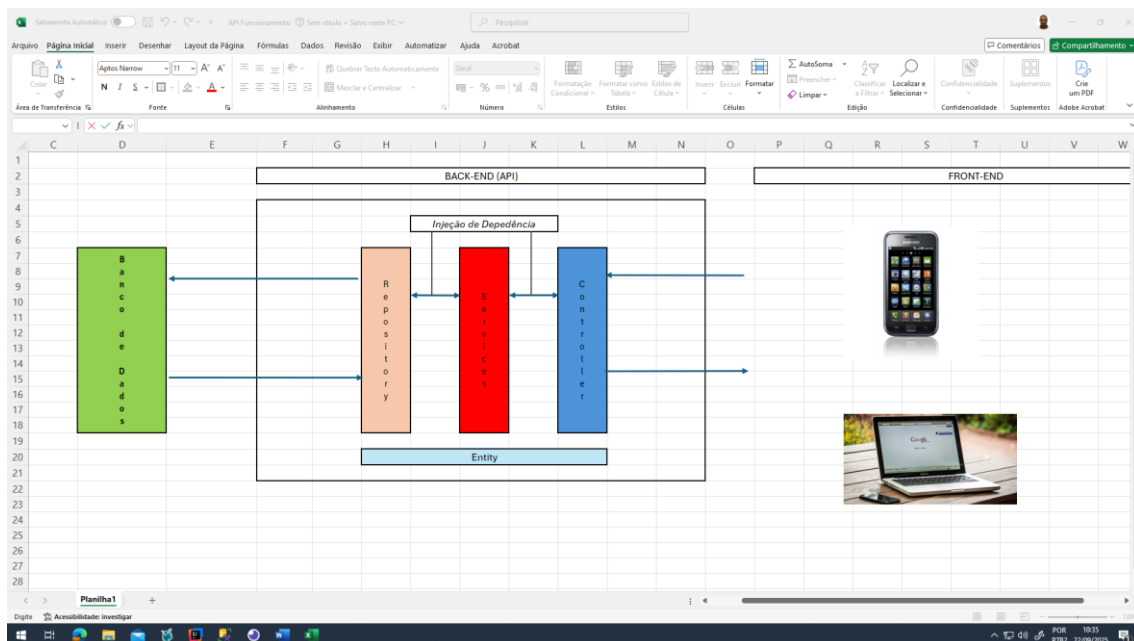
Abrir o Terminal ...

git add . <<ENTER>>

git commit -m "Anotações principais de Model Repository concluída" <<ENTER>>

git push <<ENTER>>

Antes de continuarmos, vamos relembrar a visão geral da nossa API!



Model Services

O que é Service Layer?

O Service Layer, ou Camada de Serviço, é um padrão arquitetural que atua como intermediário entre a interface do usuário e a lógica de negócios de uma aplicação. Ele encapsula a lógica de negócios e fornece uma interface para que diferentes partes do sistema possam interagir de forma organizada e eficiente. Essa abordagem é especialmente útil em [aplicações web](#), onde a separação de responsabilidades é crucial para a manutenção e escalabilidade do código.

Funções principais do Service Layer

Uma das principais funções do Service Layer é a centralização da lógica de negócios. Ao invés de espalhar essa lógica por diferentes partes da aplicação, o Service Layer permite que ela seja gerida em um único local. Isso não apenas facilita a manutenção, mas também melhora a testabilidade do código, uma vez que as regras de negócio podem ser testadas independentemente da interface do usuário. Além disso, o Service Layer pode atuar como um ponto de controle para a validação de dados, garantindo que apenas informações válidas sejam processadas.

Vantagens do uso do Service Layer

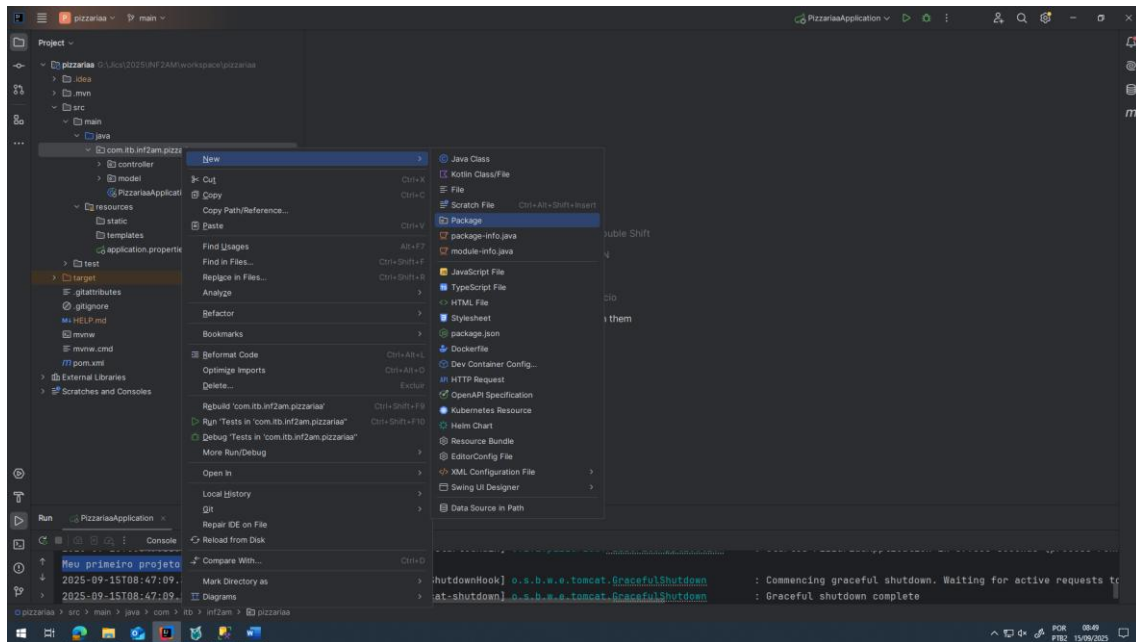
O uso do Service Layer traz diversas vantagens, como a promoção de um design mais limpo e organizado. Isso é especialmente importante em projetos de grande escala, onde a complexidade pode aumentar rapidamente. Outro benefício é a possibilidade de reutilização de código. Com a lógica de negócios centralizada, diferentes partes da aplicação podem reutilizar os mesmos serviços, evitando duplicação e facilitando a manutenção. Além disso, o Service Layer pode facilitar a integração com outros sistemas, já que ele pode expor APIs que podem ser consumidas por diferentes clientes.

Logo, vamos prosseguir na criação da nossa camada de serviços.

No projeto, vamos criar um novo “package”

Botão direito do mouse sobre o package principal:

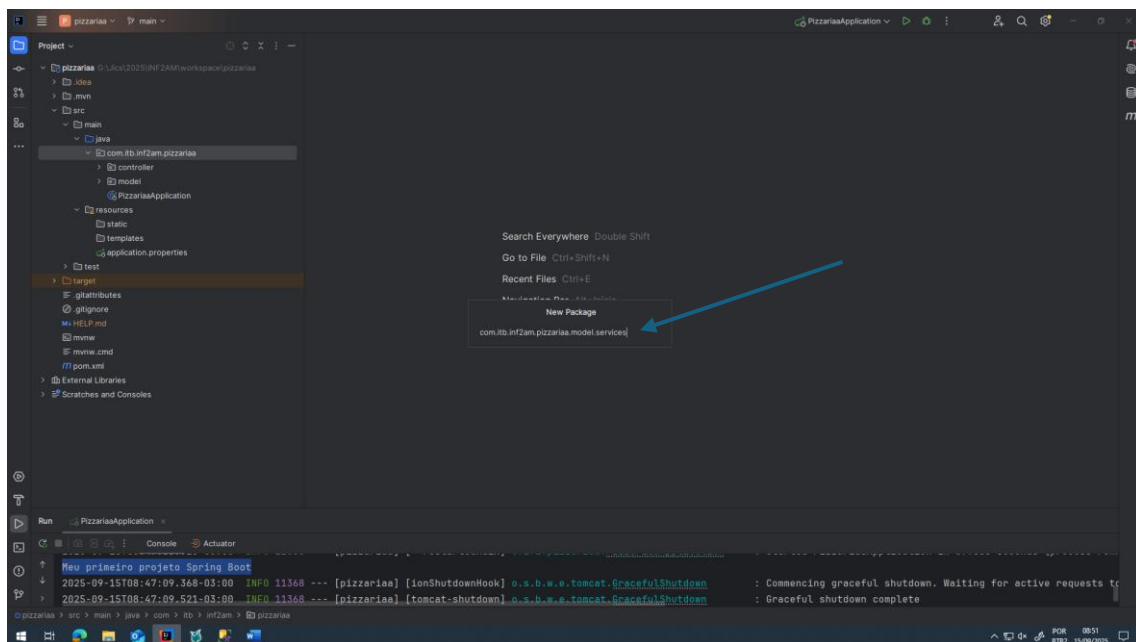
Veja

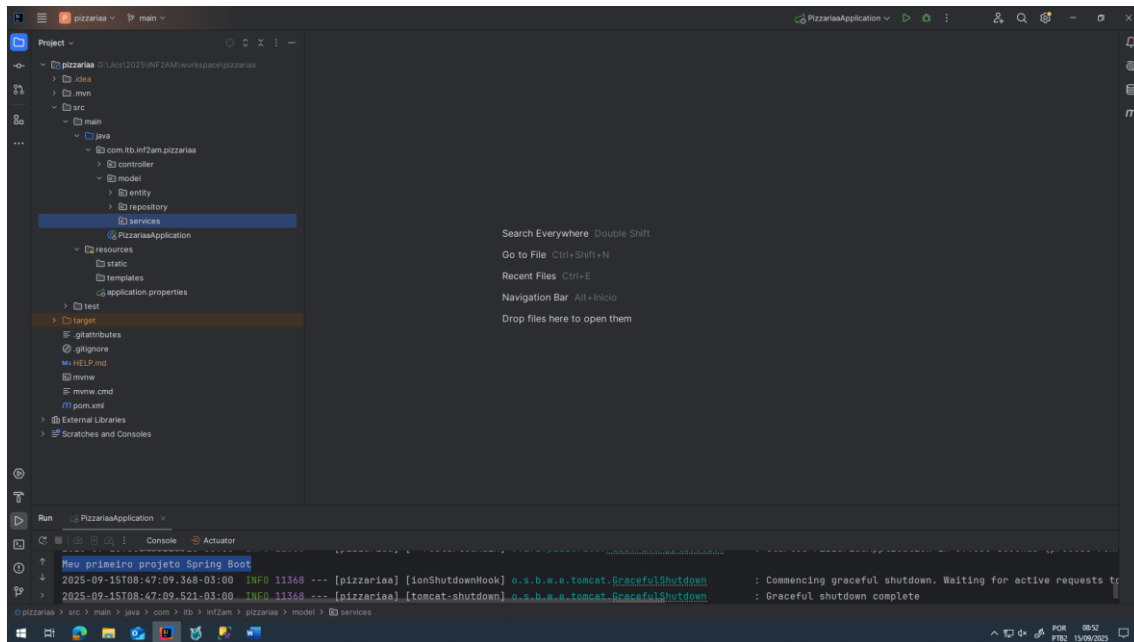


Complemente o nome:

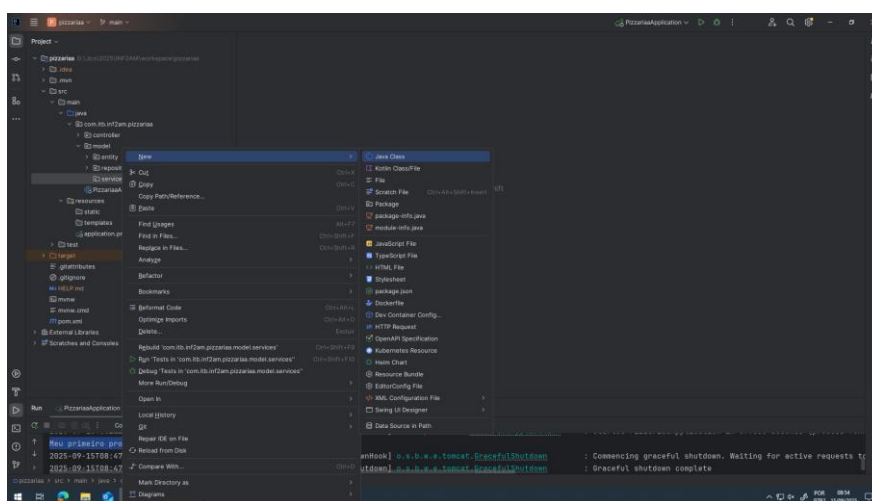
`com.itb.inf2am.pizzariaa.model.services`

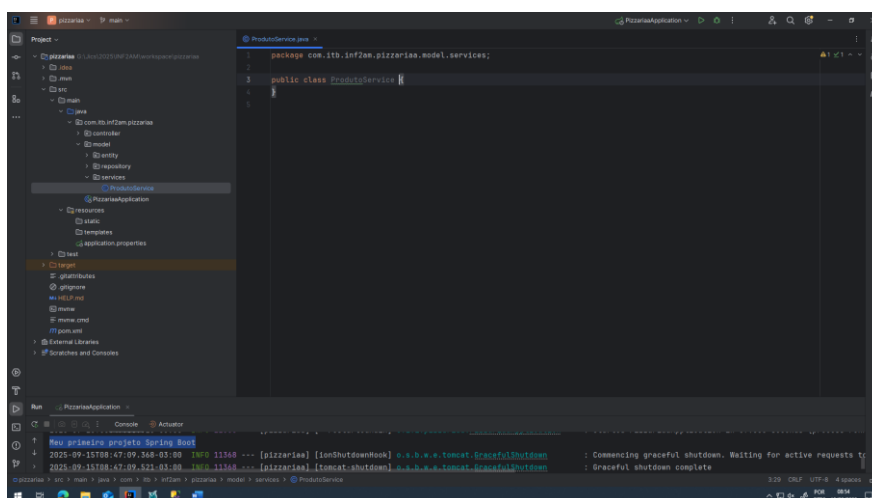
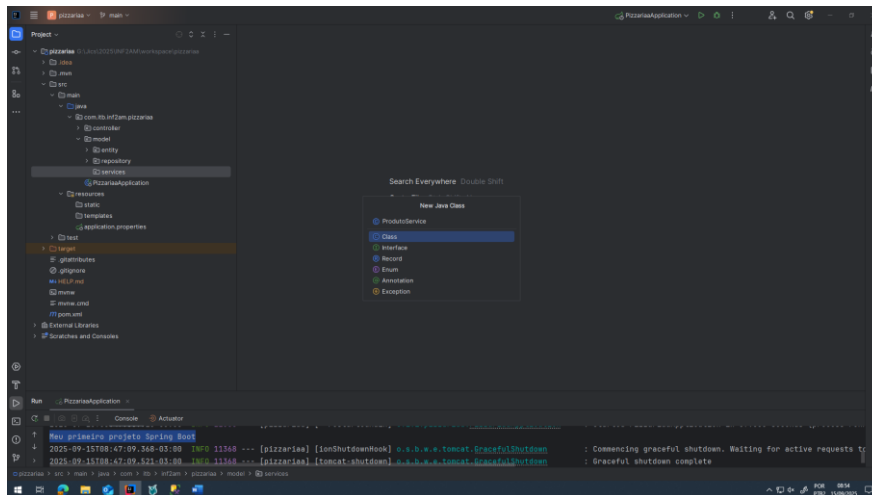
Veja:





Botão direito em “sevcies” -> new JavaClass -> **ProdutoService**





Na continuação, precisamos entender o conceito de **“Injeção de Dependência”**

A injeção de dependência é um padrão de projeto utilizado para diminuir o acoplamento entre as classes de um sistema. Em outras palavras, é uma forma de estruturar o código para que as classes dependam de abstrações, em vez de dependerem diretamente de outras classes concretas.

Mas o que isso significa na prática? Vamos supor que você esteja desenvolvendo um aplicativo que precisa se conectar a um banco de dados. Em vez de criar uma classe que faça a conexão diretamente, você cria uma interface que define os métodos que essa classe deve ter, como "conectar", "desconectar" e "executar consulta".

Em seguida, você implementa uma classe concreta que implementa essa interface e realiza a conexão com um banco de dados específico, como o MySQL. Agora, ao invés de outras classes do

seu sistema dependerem diretamente da classe de conexão com o MySQL, elas dependem da interface genérica que você definiu.

E é aí que a injeção de dependência entra em ação. Em vez de instanciar diretamente a classe de conexão dentro das classes dependentes, você assegura que essa dependência seja "injetada" por meio de construtores, métodos ou até mesmo configurações externas.

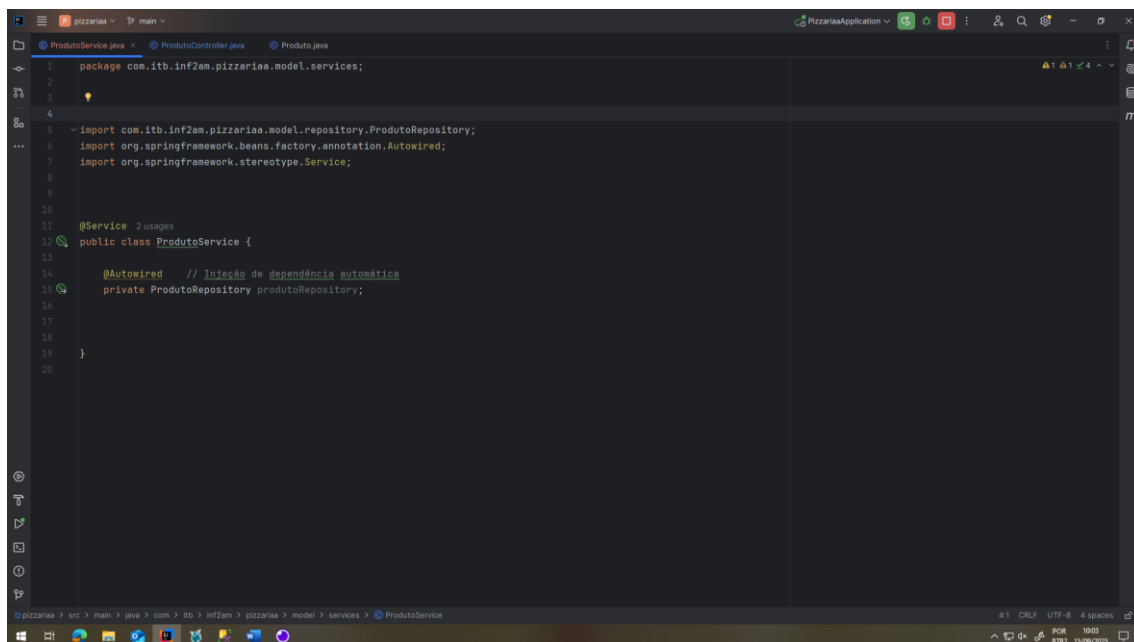
Nesse primeiro momento, vamos utilizar a Anotação Spring **@Autowired** para injeção de dependência.

Resumindo: Nossa camada **Model Services** "depende" da camada **Model Repository**:

Logo declare a seguinte variável em ProdutoServices:

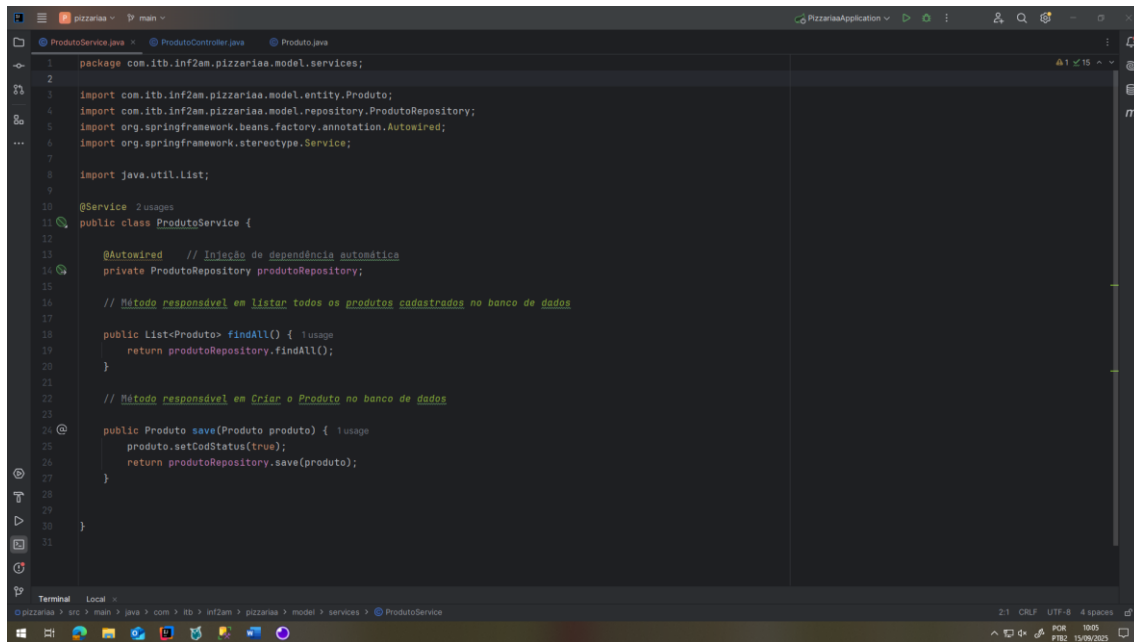
@Autowired // Injeção de dependência automática
private ProdutoRepository produtoRepository;

Veja:



Pronto, agora estamos aptos a criar os serviços de CRUD

Veja:



```
1 package com.itb.inf2am.pizzariaa.model.services;
2
3 import com.itb.inf2am.pizzariaa.model.entity.Produto;
4 import com.itb.inf2am.pizzariaa.model.repository.ProdutoRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7
8 import java.util.List;
9
10 @Service
11 public class ProdutoService {
12
13     @Autowired // Injeção de dependência automática
14     private ProdutoRepository produtoRepository;
15
16     // Método responsável em listar todos os produtos cadastrados no banco de dados
17
18     public List<Produto> findAll() {
19         return produtoRepository.findAll();
20     }
21
22     // Método responsável em criar o Produto no banco de dados
23
24     public Produto save(Produto produto) {
25         produto.setCodStatus(true);
26         return produtoRepository.save(produto);
27     }
28
29
30
31 }
```

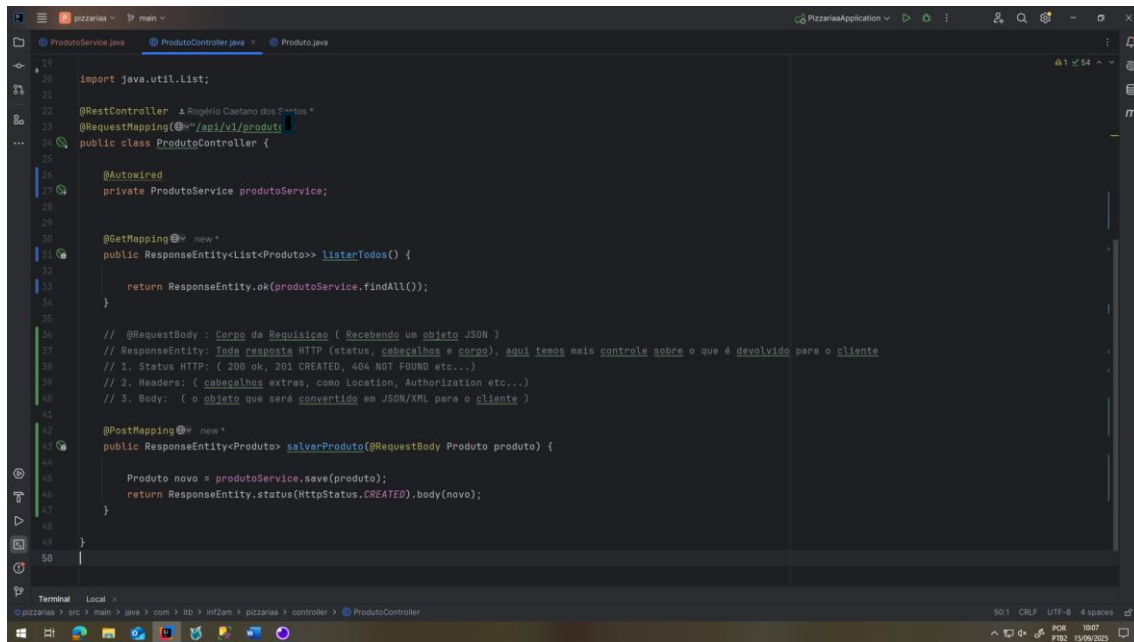
Terminal Local

```
pizzariaa > src > main > java > com > itb > inf2am > pizzariaa > model > services > ProdutoService
```

Para testar os serviços, precisamos “alterar” nosso ProdutoController. Agora já temos o nosso banco de dados:

Veja:

ProdutoController.java



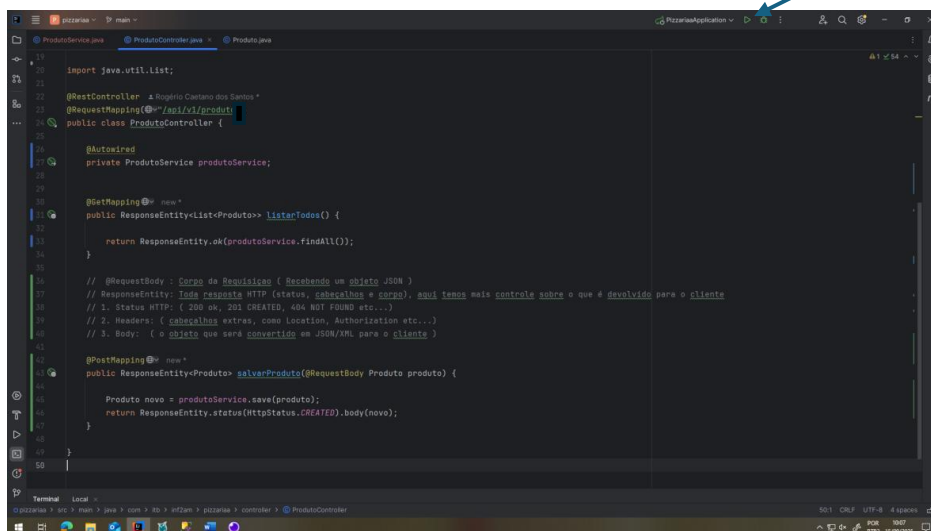
```
19
20
21 import java.util.List;
22
23 @RestController @Rogério Caetano dos Santos
24 @RequestMapping("/api/v1/produto")
25 public class ProdutoController {
26
27     @Autowired
28     private ProdutoService produtoService;
29
30     @GetMapping("/new")
31     public ResponseEntity<List<Produto>> listarTodos() {
32
33         return ResponseEntity.ok(produtoService.findAll());
34     }
35
36     // @RequestBody : Corpo da Requisição ( Recebendo um objeto JSON )
37     // ResponseEntity: Toda resposta HTTP (status, cabeçalhos e corpo), aqui temos mais controle sobre o que é devolvido para o cliente
38     // 1. Status HTTP: ( 200 OK, 201 CREATED, 404 NOT FOUND etc...)
39     // 2. Headers: ( cabeçalhos extras, como Location, Authorization etc...)
40     // 3. Body: ( o objeto que será convertido em JSON/XML para o cliente )
41
42     @PostMapping("/new")
43     public ResponseEntity<Produto> salvarProduto(@RequestBody Produto produto) {
44
45         Produto novo = produtoService.save(produto);
46         return ResponseEntity.status(HttpStatus.CREATED).body(novo);
47     }
48 }
49
50
```

Terminal Local

pizzaria > src > main > java > com > itb > infzen > pizzaria > controller > ProdutoController

50:1 CRLF UTF-8 4 spaces

Execute a aplicação...



```
19
20
21 import java.util.List;
22
23 @RestController @Rogério Caetano dos Santos
24 @RequestMapping("/api/v1/produto")
25 public class ProdutoController {
26
27     @Autowired
28     private ProdutoService produtoService;
29
30     @GetMapping("/new")
31     public ResponseEntity<List<Produto>> listarTodos() {
32
33         return ResponseEntity.ok(produtoService.findAll());
34     }
35
36     // @RequestBody : Corpo da Requisição ( Recebendo um objeto JSON )
37     // ResponseEntity: Toda resposta HTTP (status, cabeçalhos e corpo), aqui temos mais controle sobre o que é devolvido para o cliente
38     // 1. Status HTTP: ( 200 OK, 201 CREATED, 404 NOT FOUND etc...)
39     // 2. Headers: ( cabeçalhos extras, como Location, Authorization etc...)
40     // 3. Body: ( o objeto que será convertido em JSON/XML para o cliente )
41
42     @PostMapping("/new")
43     public ResponseEntity<Produto> salvarProduto(@RequestBody Produto produto) {
44
45         Produto novo = produtoService.save(produto);
46         return ResponseEntity.status(HttpStatus.CREATED).body(novo);
47     }
48 }
49
50
```

Terminal Local

pizzaria > src > main > java > com > itb > infzen > pizzaria > controller > ProdutoController

50:1 CRLF UTF-8 4 spaces

Caso tenha executado sem erros...

```

24 public class ProdutoController {
25
26     // @RequestBody : Corpo da Requisição ( Recebendo um objeto JSON )
27     // ResponseEntity: Toda resposta HTTP (status, cabeçalhos e corpo), aqui temos mais controle sobre o que é devolvido para o cliente
28     // 1. Status HTTP: ( 200 OK, 201 CREATED, 404 NOT FOUND etc...)
29     // 2. Headers: ( cabeçalhos extras, como Location, Authorization etc...)
30     // 3. Body: ( o objeto que será convertido em JSON/XML para o cliente )
31
32     @PostMapping("/produtos")
33     public ResponseEntity<Produto> salvarProduto(@RequestBody Produto produto) {
34
35         Produto novo = produtoService.save(produto);
36         return ResponseEntity.status(HttpStatus.CREATED).body(novo);
37     }
38 }

```

```

[pizzaria] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform int
[pizzaria] [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
[pizzaria] [ restartedMain] JpaBaseConfigurationJpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view
[pizzaria] [ restartedMain] o.s.b.d.s.OptionalLiveReloadServer : LiveReload server is running on port 35729
[pizzaria] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
[pizzaria] [ restartedMain] c.i.i.pizzaria.PizzeriaApplication : Started PizzeriaApplication in 12.375 seconds (process running for 13.31)

```

Poderemos prosseguir no teste da nossa API.

Para tal, precisamos de um “testador de api´s” (Client Rest), existem vários:

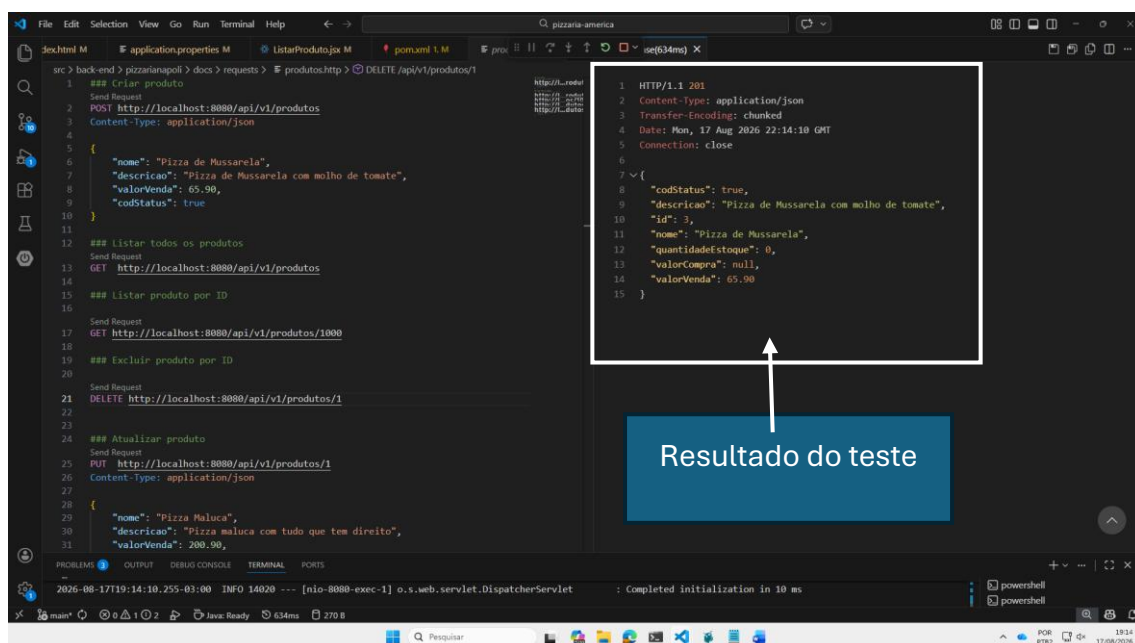
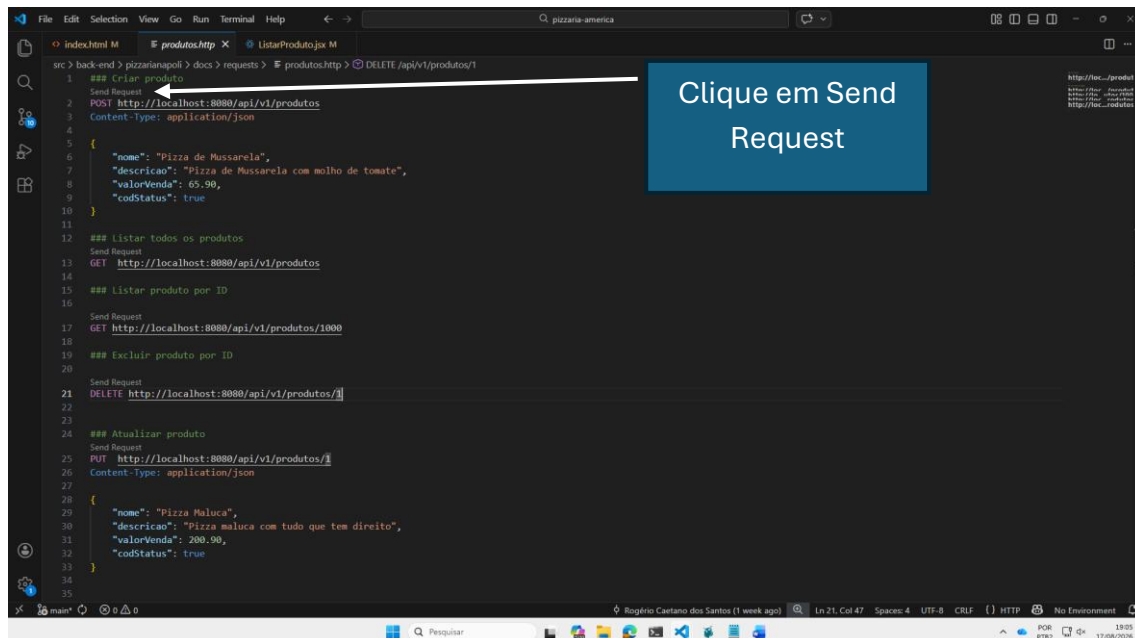
A seguir, alguns:

- Insomnia
- Postman
- SoapUI

OBS: CASO esteja utilizando **O VS-CODE**, **UTILIZAR O REST Client** para prosseguir nos testes, a seguir um exemplo de como montar o arquivo de teste com a extensão .http

Você pode criar uma pasta dentro do projeto para salvar o arquivo.

Exemplo: **pizzariaf/docs/requests/produtos.http**

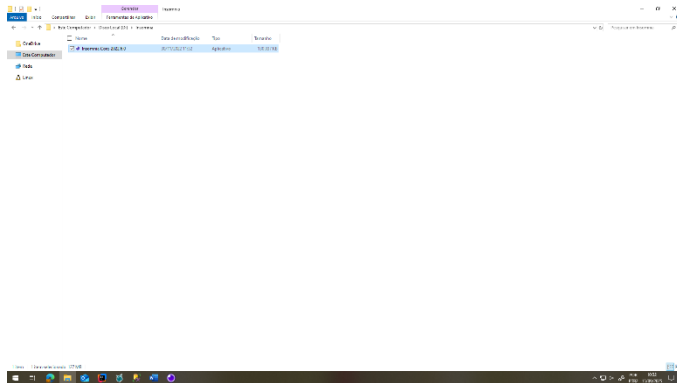


Se você está no VS-CODE, Prossiga para a página 71...

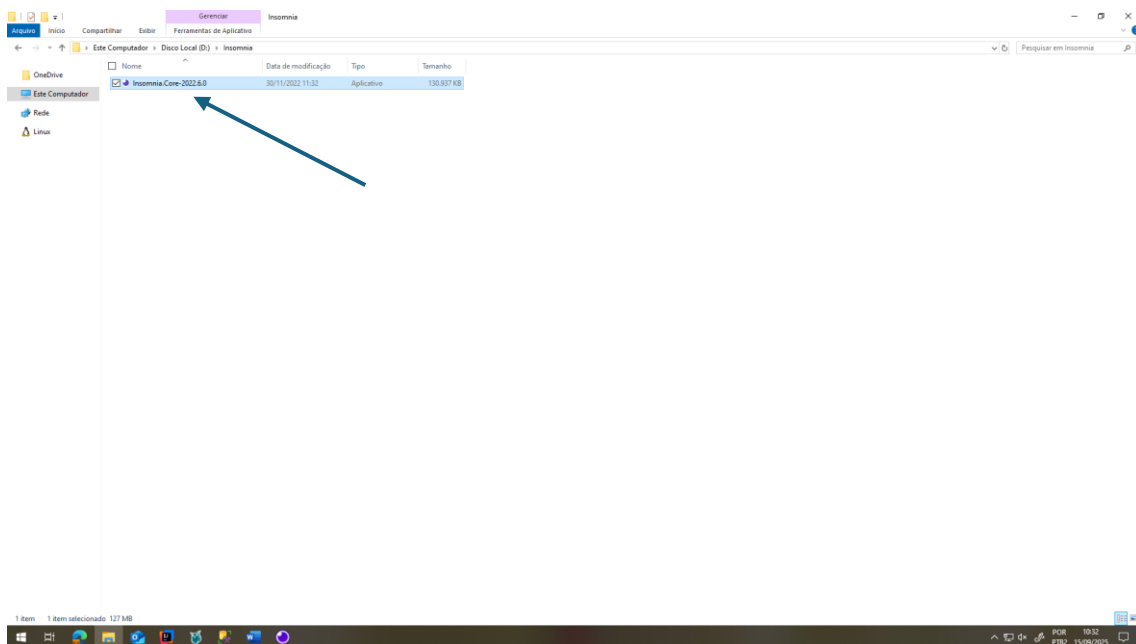
Em nossos testes no IntelliJ, vamos utilizar o **Insomnia**. Em nossos laboratórios você irá encontrá-lo no seguinte endereço:

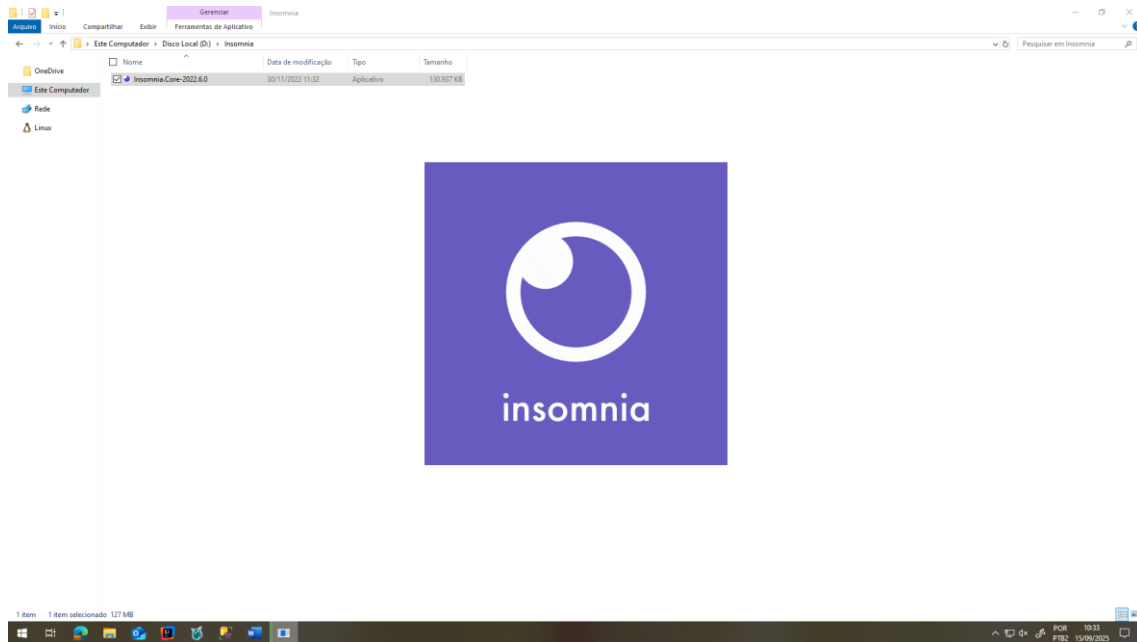
D:\Insomnia

Veja:



Duplo clique para abrir o programa:





Na sua casa você pode fazer o download, ele é gratuito para utilizar de forma limitada.

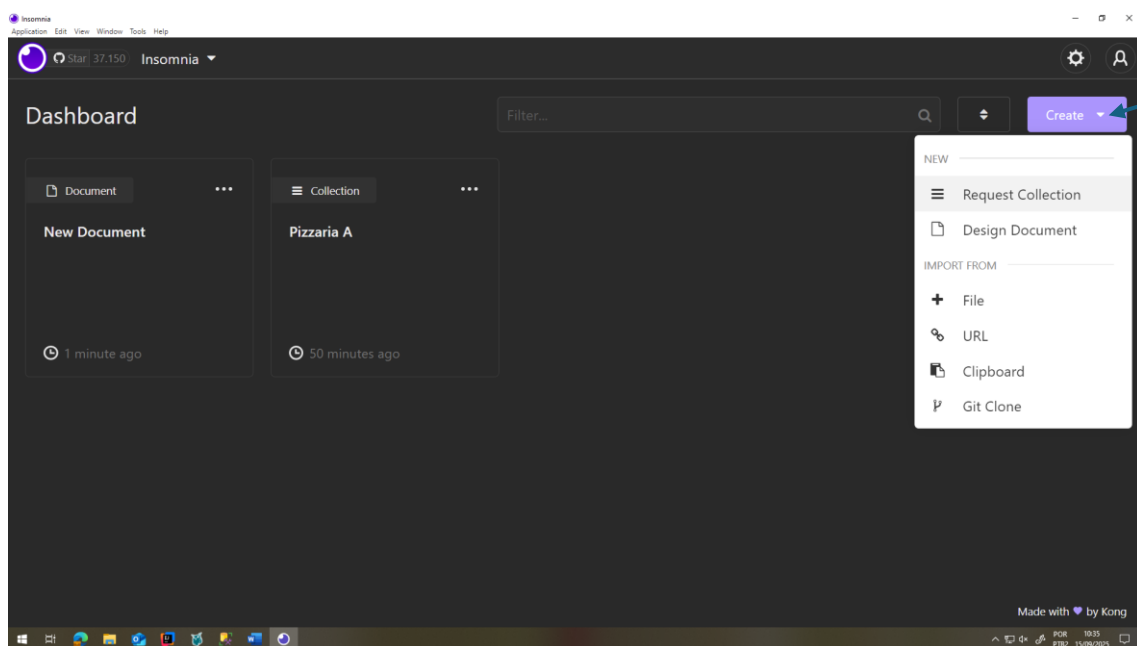
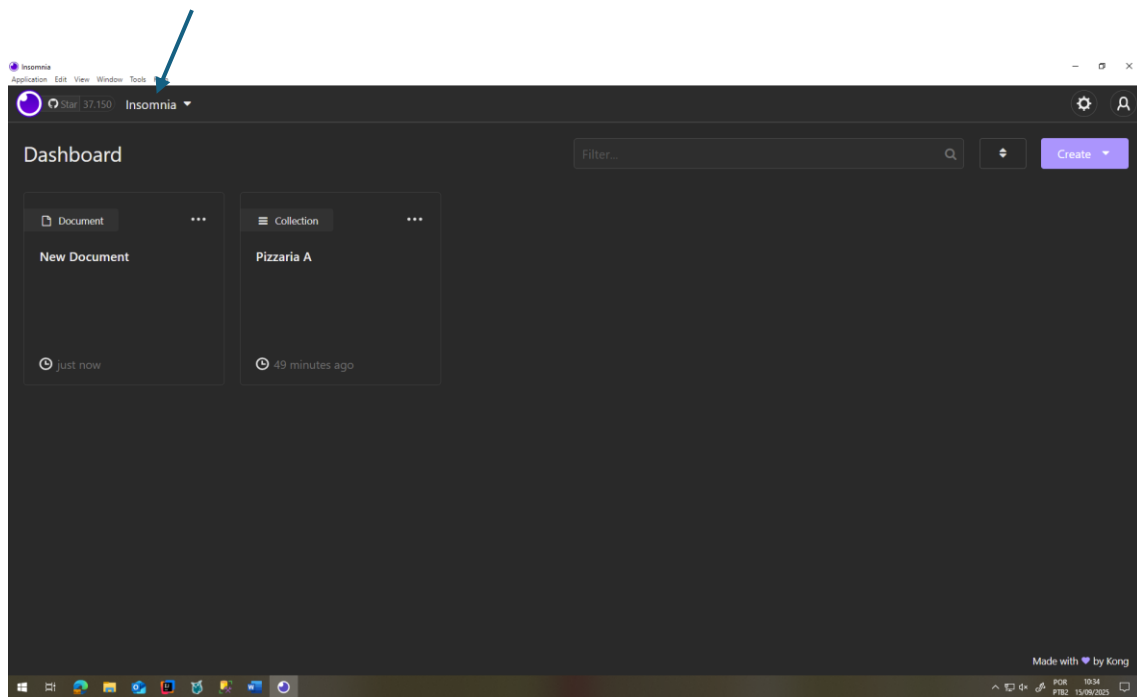
[Download - Insomnia](#)

Se preferir pode utilizar o Postman

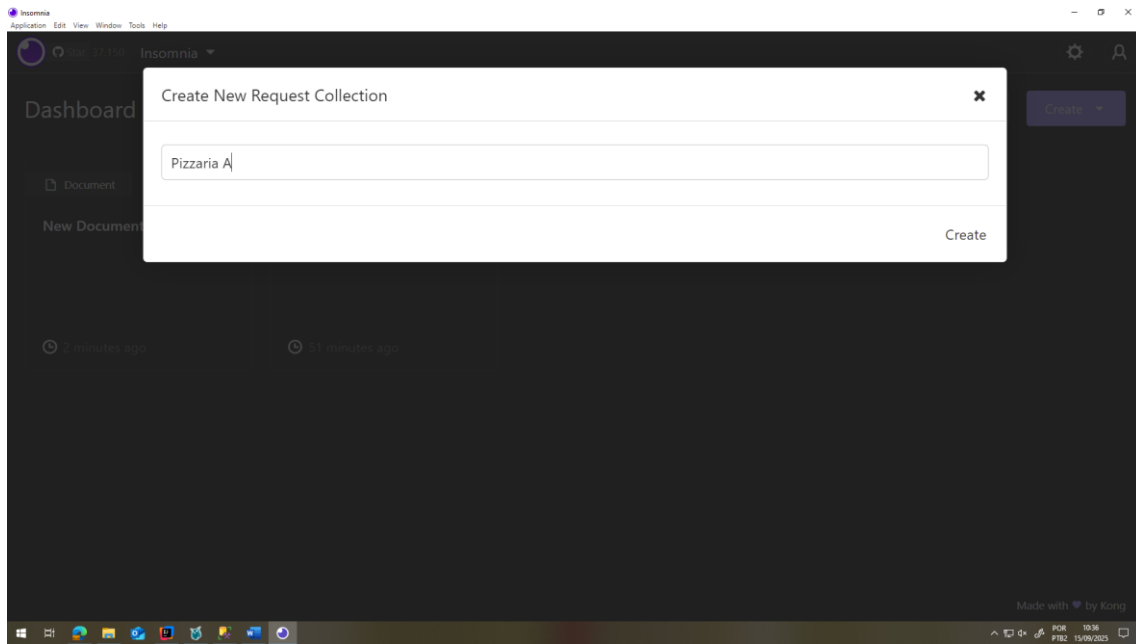
[Download Postman | Get Started for Free](#)

Em nosso projeto vamos utilizar o Insomnia

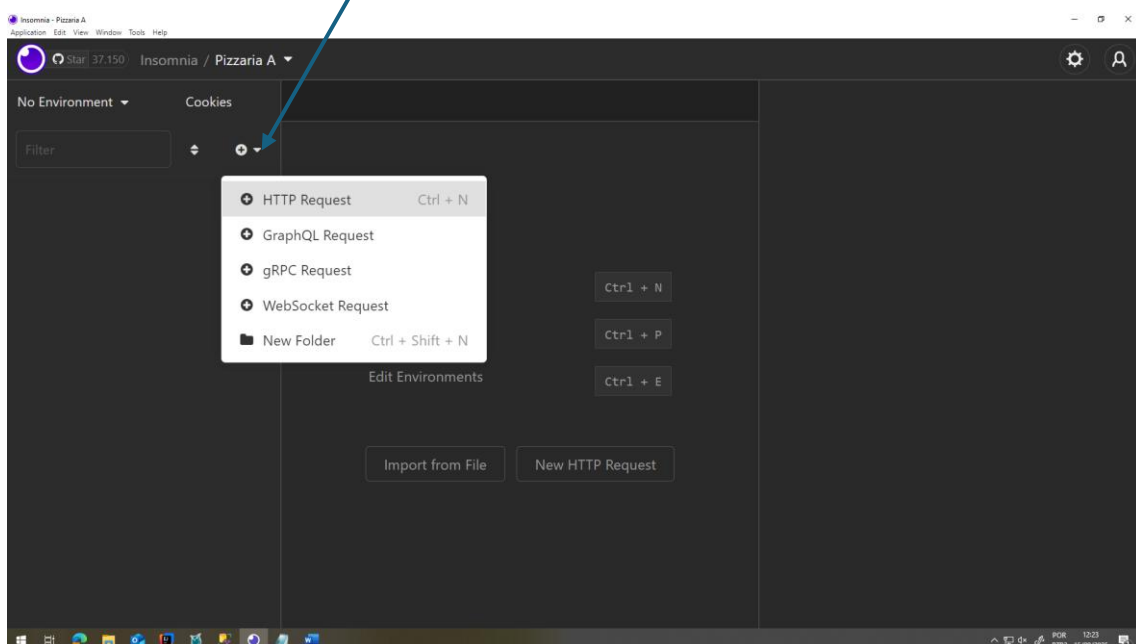
Clique em Insomnia para criar um novo projeto:



Escolha Request Collection

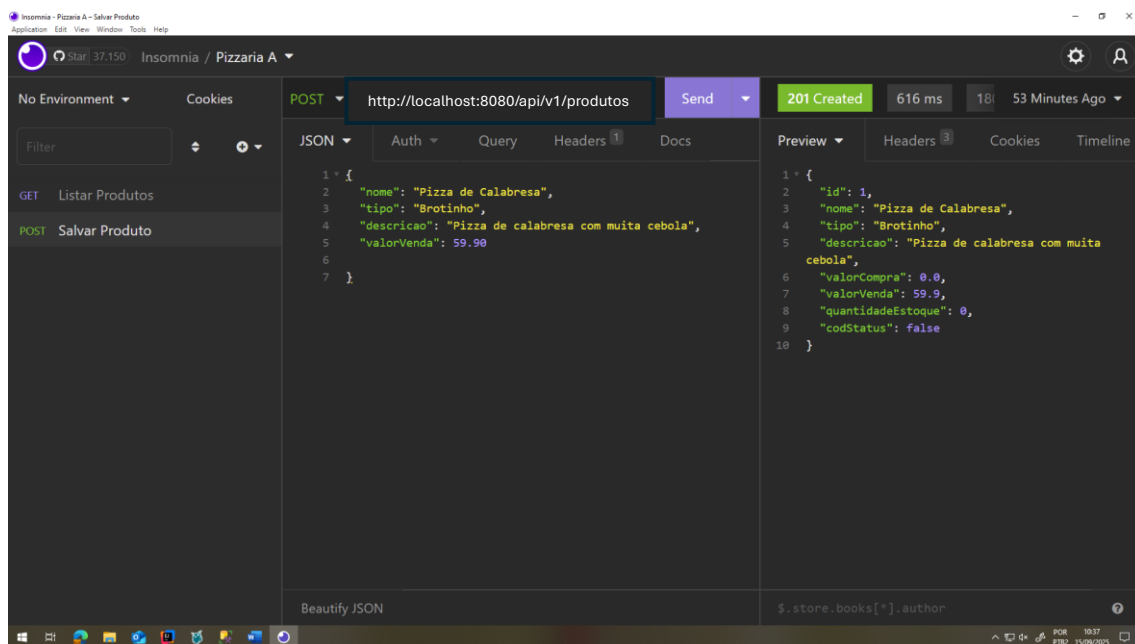
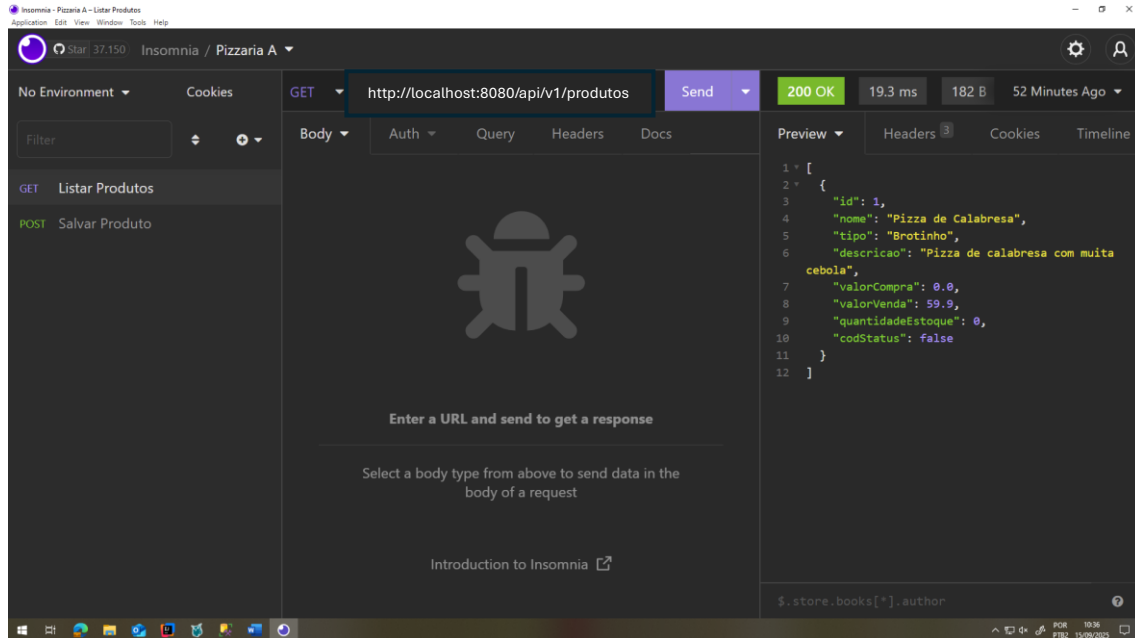


Clique HttpRequest



A seguir, temos 2 End-Points Criados:

- Listar Produto (GET)
- Salvar Produto (POST)



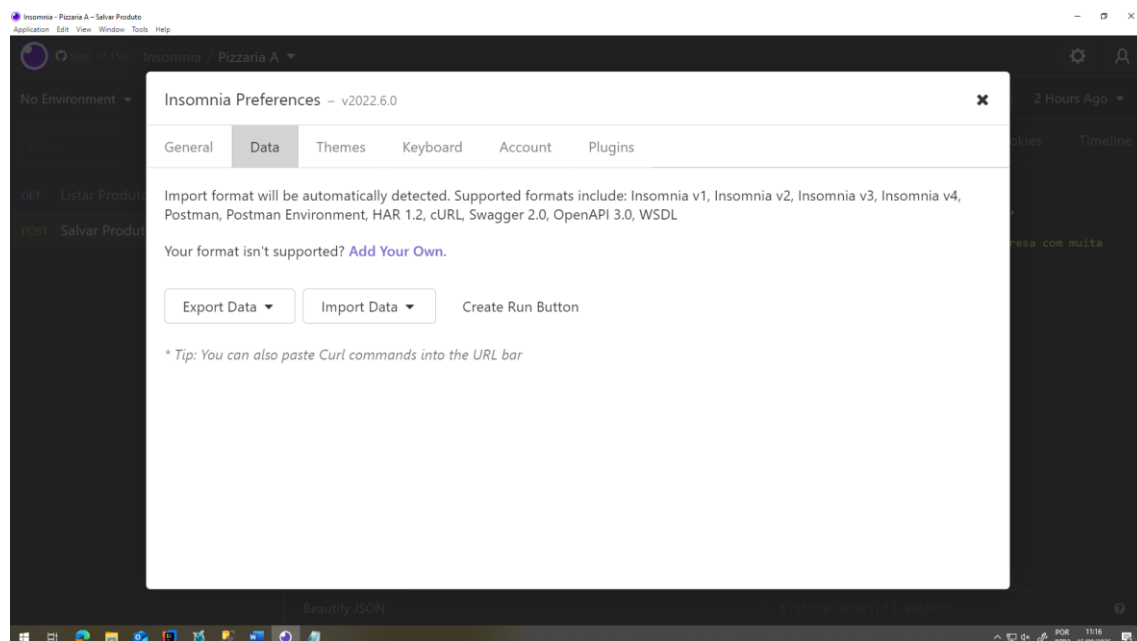
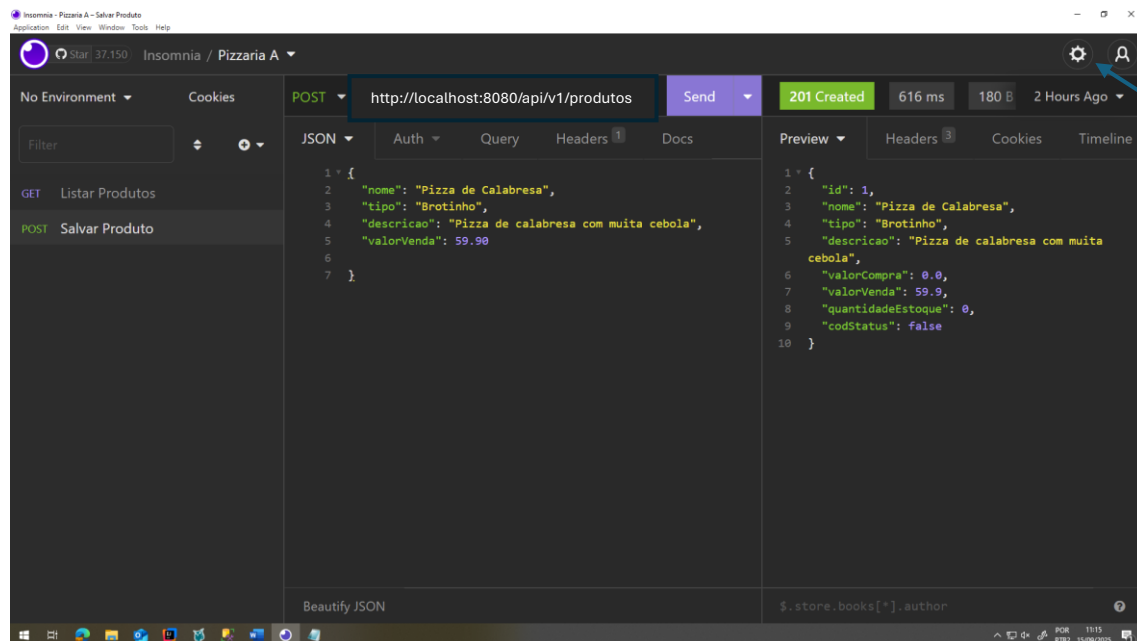
Para não perder o projeto feito no Insomnia, precisamos “Salvar”

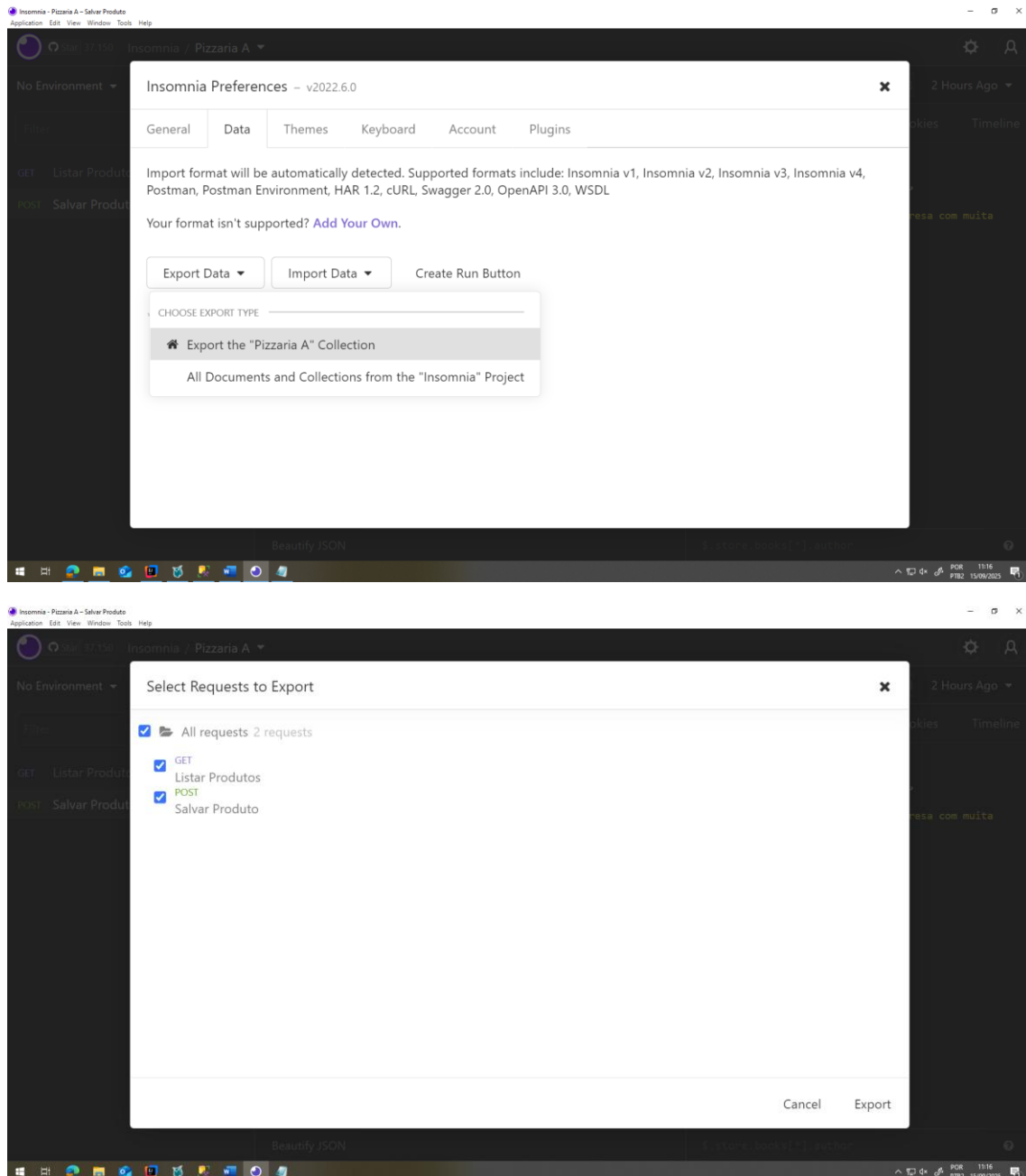
Clique:

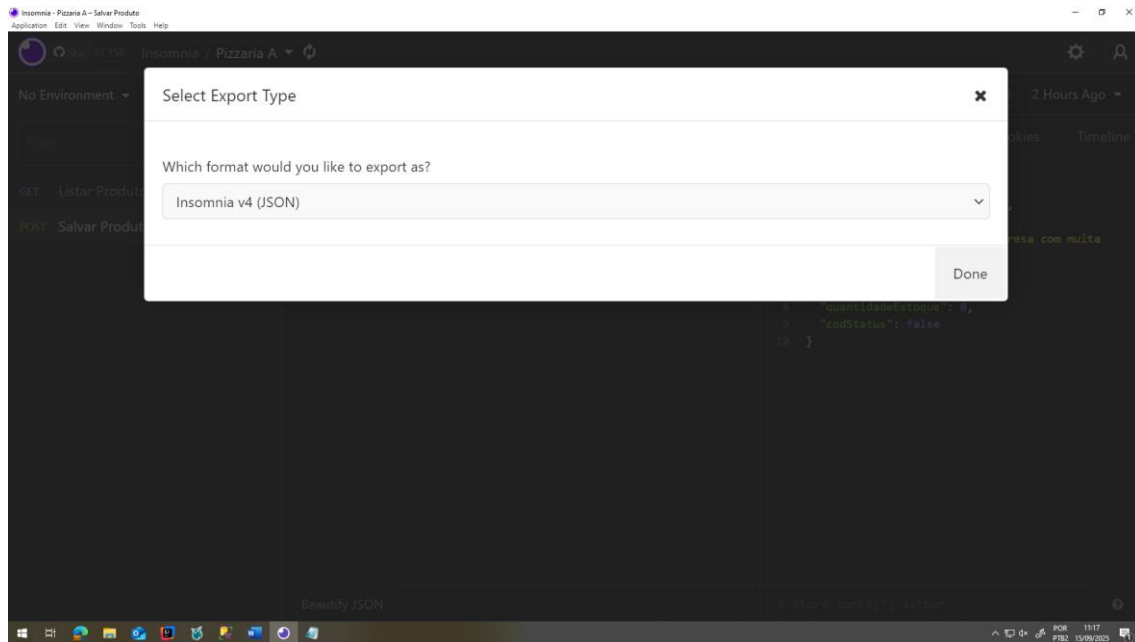
Preferences -> Data -> Export Data -> Export the “Pizzaria A” Collection -> Export -> Done

“Escolher o diretório onde você deseja salvar o arquivo”

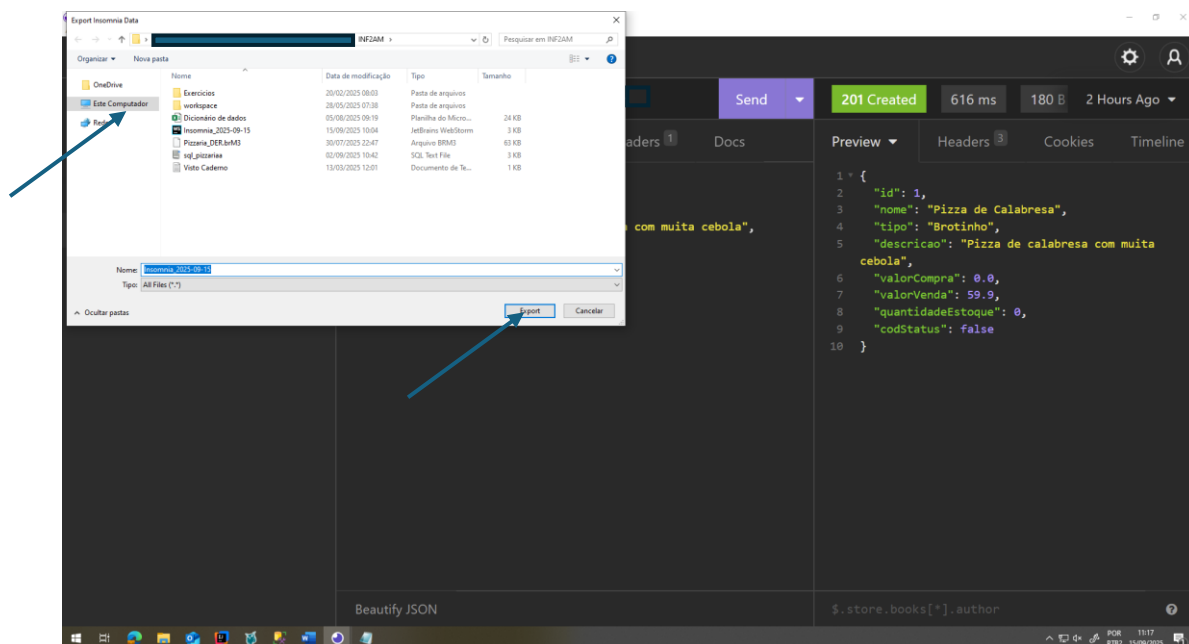
A seguir os prints passo – a – passo





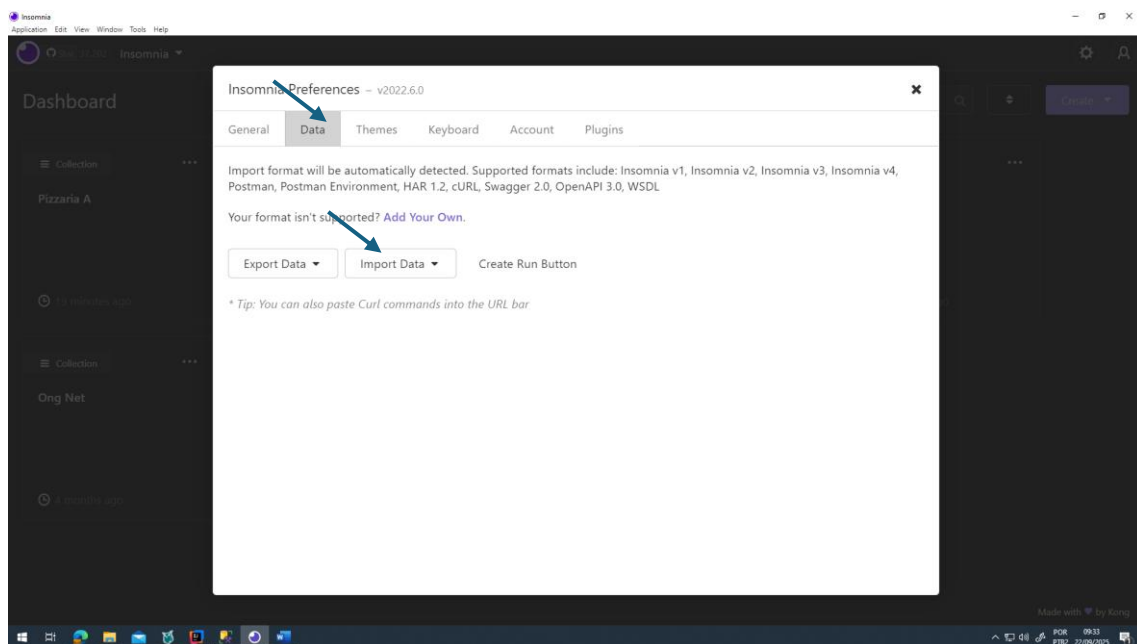
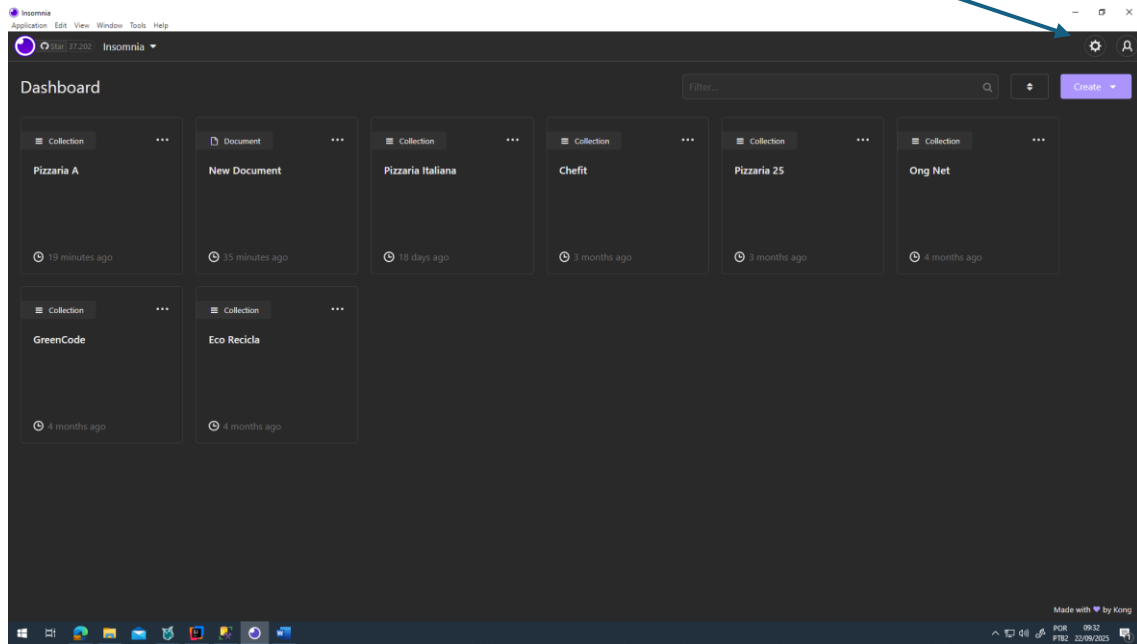


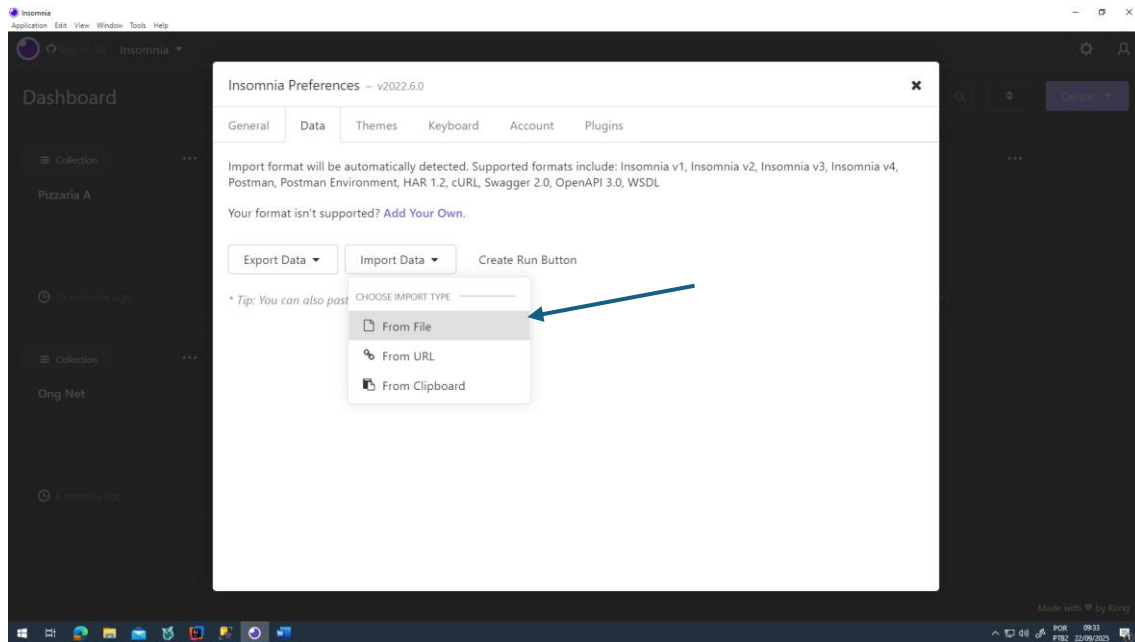
Escolher o diretório onde deseja salvar...



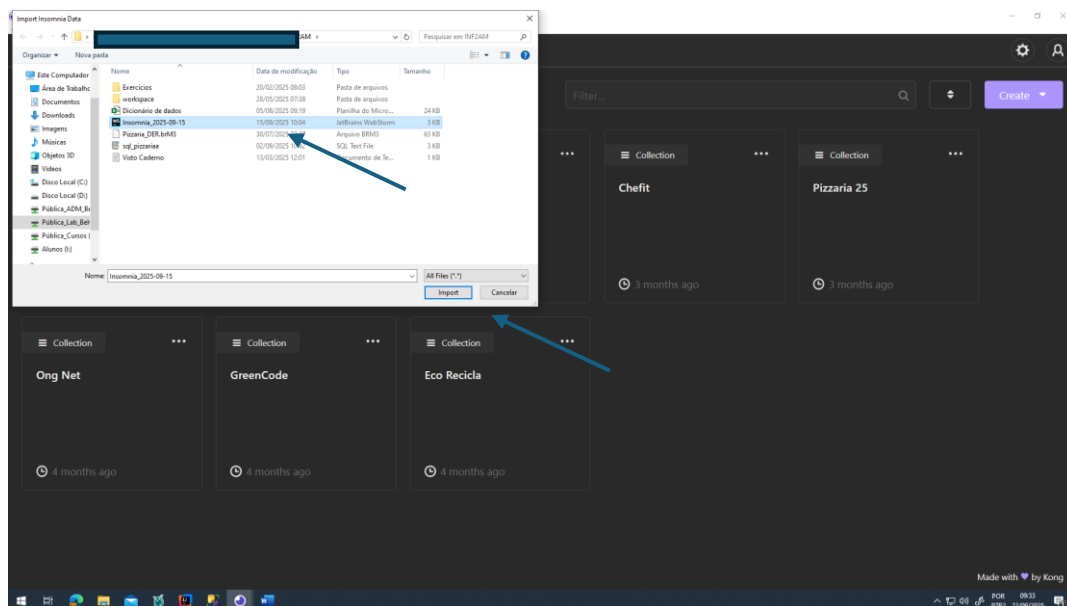
Importando os End-Points

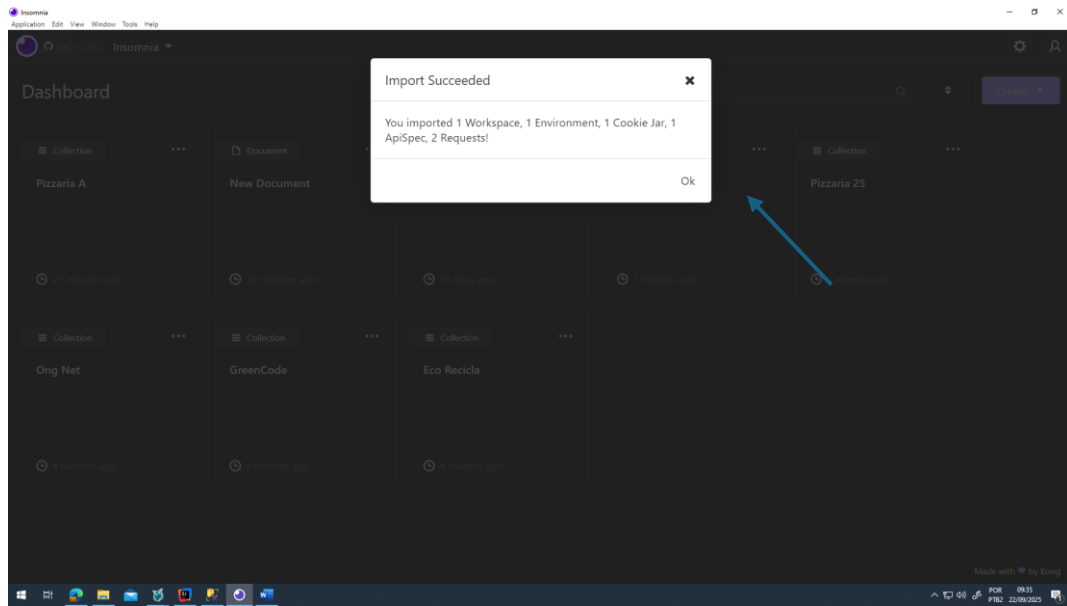
Sempre que quiser recuperar (abrir) os end-points salvos na pasta...



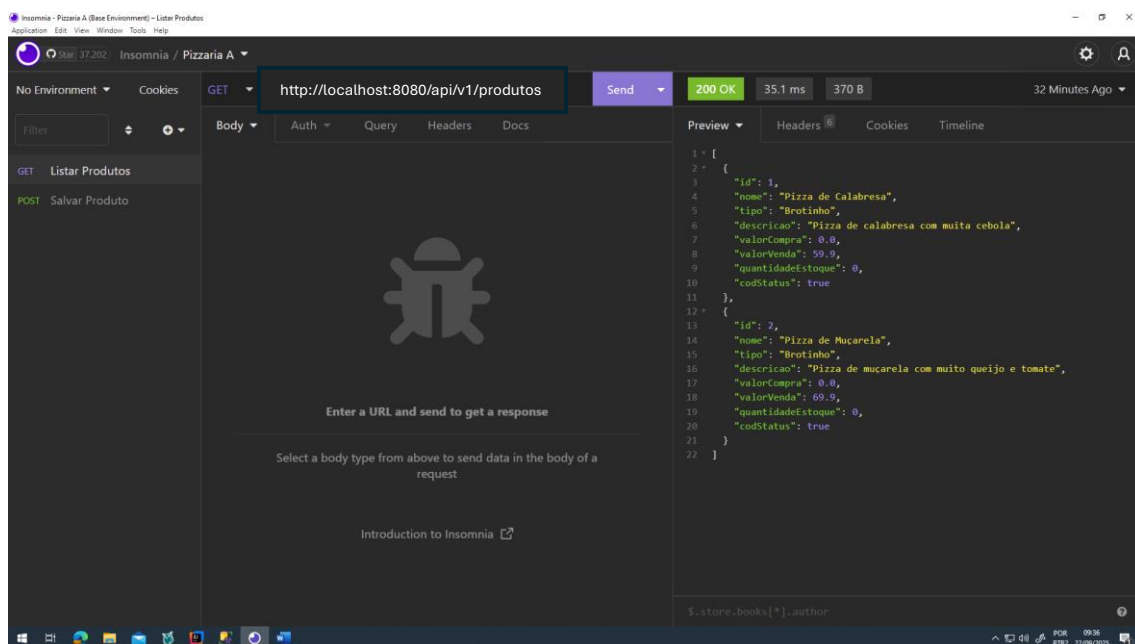
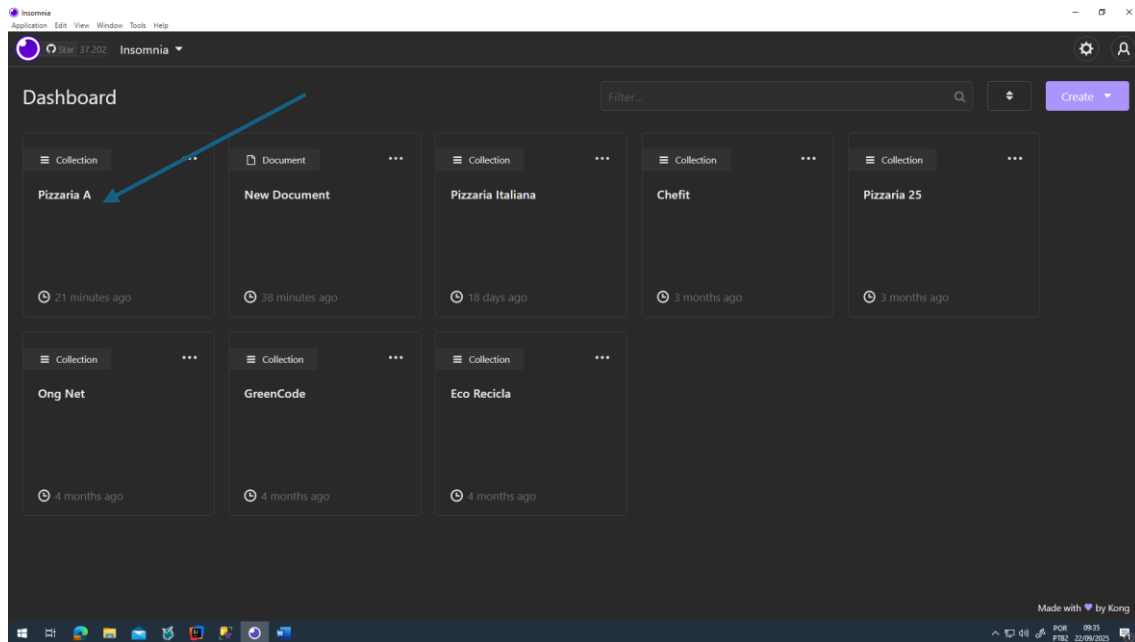


Procurar na pasta onde você salvou o arquivo





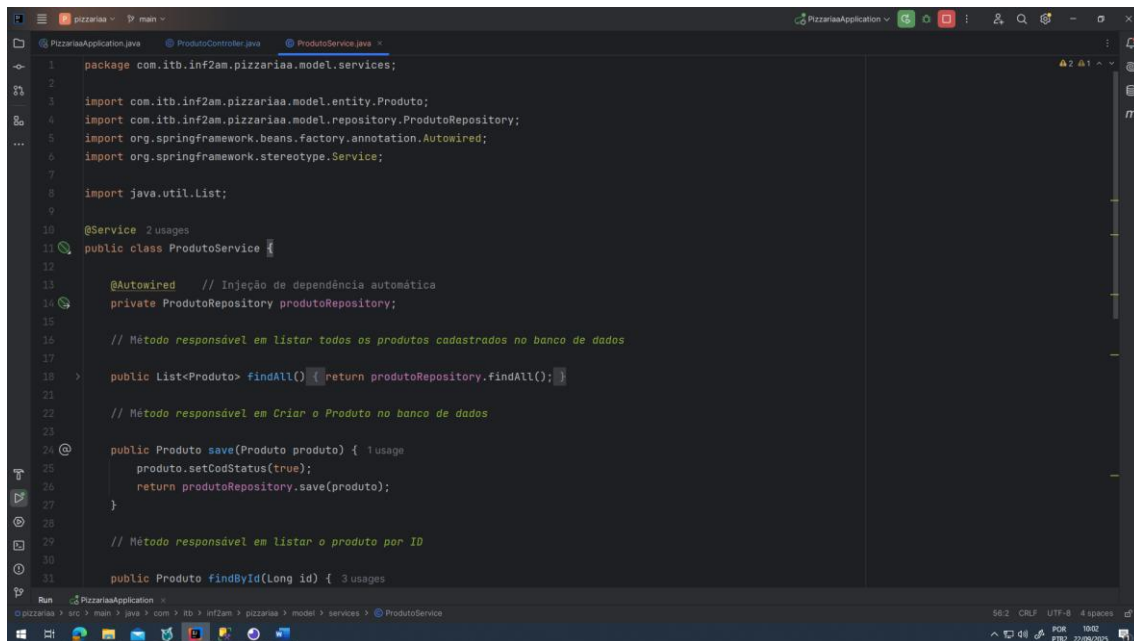
Escolher a referida Collection



Para os demais End-Points, precisamos atualizar nosso **ProdutoService**, vamos prosseguir na criação dos seguintes métodos:

- findById
- update
- delete

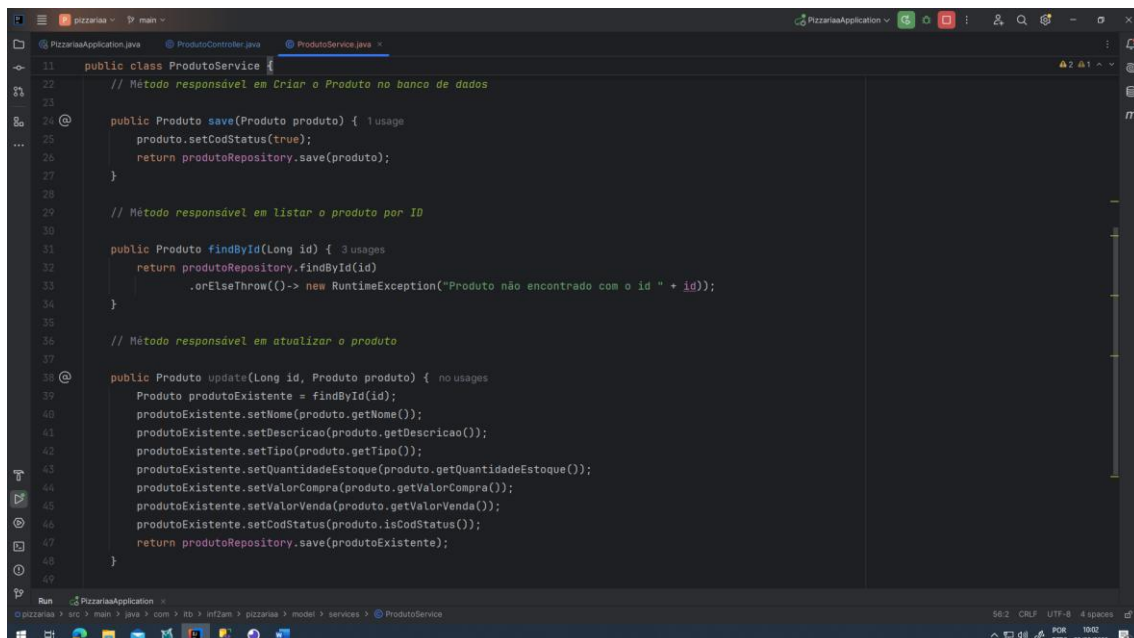
Veja: A classe **ProdutoService** completa:



```

1 package com.itb.inf2am.pizzariaa.model.services;
2
3 import com.itb.inf2am.pizzariaa.model.entity.Produto;
4 import com.itb.inf2am.pizzariaa.model.repository.ProdutoRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7
8 import java.util.List;
9
10 @Service
11 public class ProdutoService {
12
13     @Autowired // Injeção de dependência automática
14     private ProdutoRepository produtoRepository;
15
16     // Método responsável em listar todos os produtos cadastrados no banco de dados
17
18     public List<Produto> findAll() { return produtoRepository.findAll(); }
19
20     // Método responsável em Criar o Produto no banco de dados
21
22     @
23     public Produto save(Produto produto) { 1 usage
24         produto.setCodStatus(true);
25         return produtoRepository.save(produto);
26     }
27
28     // Método responsável em listar o produto por ID
29
30     public Produto findById(Long id) { 3 usages

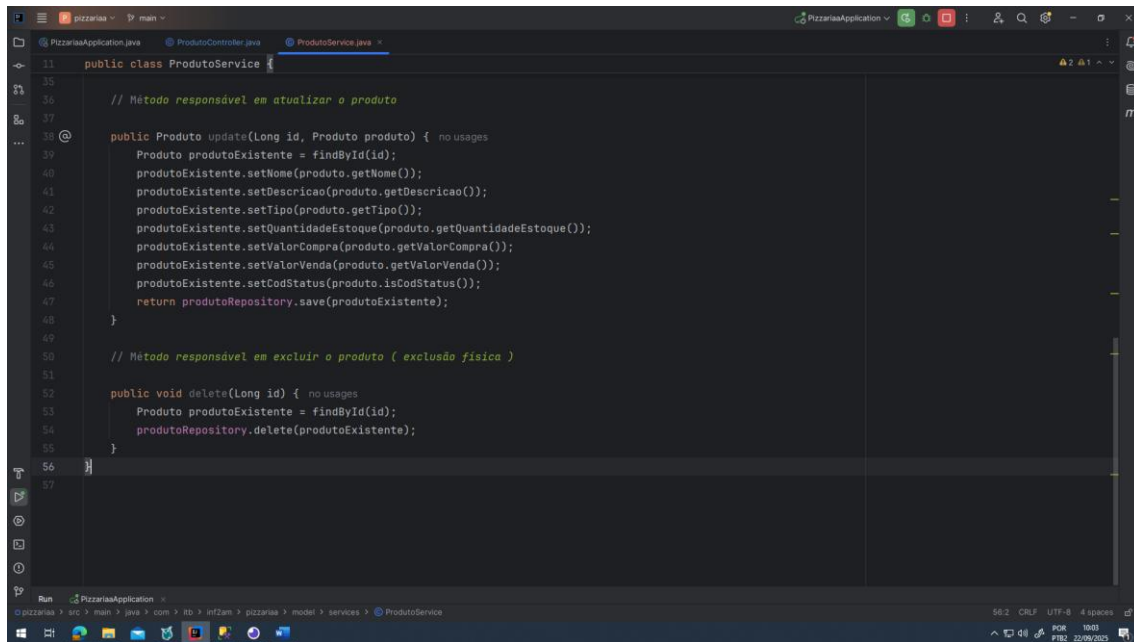
```



```

31     public Produto findById(Long id) { 3 usages
32         return produtoRepository.findById(id)
33             .orElseThrow(() -> new RuntimeException("Produto não encontrado com o id " + id));
34     }
35
36     // Método responsável em atualizar o produto
37
38     @
39     public Produto update(Long id, Produto produto) { no usages
40         Produto produtoExistente = findById(id);
41         produtoExistente.setNome(produto.getNome());
42         produtoExistente.setDescricao(produto.getDescricao());
43         produtoExistente.setTipo(produto.getTipo());
44         produtoExistente.setQuantidadeEstoque(produto.getQuantidadeEstoque());
45         produtoExistente.setValorCompra(produto.getValorCompra());
46         produtoExistente.setValorVenda(produto.getValorVenda());
47         produtoExistente.setCodStatus(produto.isCodStatus());
48         return produtoRepository.save(produtoExistente);
49     }

```



Após término, atualizar o git / github

Nota: Caso não tenha o repositório, fazer o exercício 3 da página 24

Abrir o Terminal ...

git add . <<ENTER>>

git commit -m "Serviços (CRUD) de Produto concluído" <<ENTER>>

git push <<ENTER>>

A seguir, o ProdutoController completo com todos os métodos do CRUD

- Listar todos Produtos
- Salvar Produto
- Listar Produto por Id
- Atualizar Produto
- Excluir Produto

```

1 package com.itb.inf2fm.pizzariaf.controller;
2
3 import com.itb.inf2fm.pizzariaf.model.entity.Produto;
4 import com.itb.inf2fm.pizzariaf.model.services.ProdutoService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11 import java.util.Map;
12
13 /*
14  ResponseEntity: Toda resposta HTTP (status, cabeçalhos e corpo), aqui teremos mais controle sobre o que é devolvido
15  para o cliente
16  1. Status HTTP: (200 ok, 201 CREATED, 404 NOT FOUND etc...)
17  2. Headers: ( cabeçalhos extras, como Location, Authorization etc...)
18  3. Body: _____ ( o objeto que será convertido em JSON/XML para o cliente )
19
20  @RequestBody : Corpo da requisição (Recebendo um objeto JSON )
21
22  @RestController
23
24  @RequestMapping("/api/v1/produtos")
25  )
26
27 public class ProdutoController {

```

```

17 2. Headers: ( cabeçalhos extras, como Location, Authorization etc...)
18 3. Body: _____ ( o objeto que será convertido em JSON/XML para o cliente )
19
20 @RequestBody : Corpo da requisição (Recebendo um objeto JSON )
21
22 @RestController
23
24 @RequestMapping("/api/v1/produtos")
25
26 public class ProdutoController {
27
28     @Autowired
29     private ProdutoService produtoService;
30
31     @GetMapping
32     public ResponseEntity<List<Produto>> findAll() {
33
34         return ResponseEntity.ok(produtoService.findAll());
35     }
36
37     @PostMapping
38     public ResponseEntity<Produto> save(@RequestBody Produto produto) {
39         Produto novoProduto = produtoService.save(produto);
40         return ResponseEntity.status(HttpStatus.CREATED).body(novoProduto);
41     }
42

```

```

26
42
43 @GetMapping("/{id}") new *
44 public ResponseEntity<Object> findById(@PathVariable String id) {
45     try {
46         return ResponseEntity.ok(produtosService.findById(Long.parseLong(id)));
47     } catch (NumberFormatException e) {
48         return ResponseEntity.badRequest().body (
49             Map.of(
50                 k1: "status", v1: 400,
51                 k2: "error", v2: "Bad Request",
52                 k3: "message", v3: "0 id informado não é válido: " + id
53             )
54         );
55     } catch (RuntimeException e) {
56         return ResponseEntity.status(404).body (
57             Map.of(
58                 k1: "status", v1: 404,
59                 k2: "error", v2: "Not Found",
60                 k3: "message", v3: "Produto não encontrado com o id: " + id
61             )
62         );
63     }
64 }
65
66

```

```

26
43
64 @PutMapping("/{id}") new *
67 public ResponseEntity<Object> atualizarProduto(@PathVariable String id, @RequestBody Produto produto) {
68     try {
69         return ResponseEntity.ok(produtosService.update(Long.parseLong(id), produto));
70     } catch (NumberFormatException e) {
71         return ResponseEntity.badRequest().body (
72             Map.of(
73                 k1: "status", v1: 400,
74                 k2: "error", v2: "Bad Request",
75                 k3: "message", v3: "0 id informado não é válido: " + id
76             )
77         );
78     } catch (RuntimeException e) {
79         return ResponseEntity.status(404).body (
80             Map.of(
81                 k1: "status", v1: 404,
82                 k2: "error", v2: "Not Found",
83                 k3: "message", v3: "Produto não encontrado com o id: " + id
84             )
85         );
86     }
87 }
88
89

```

```

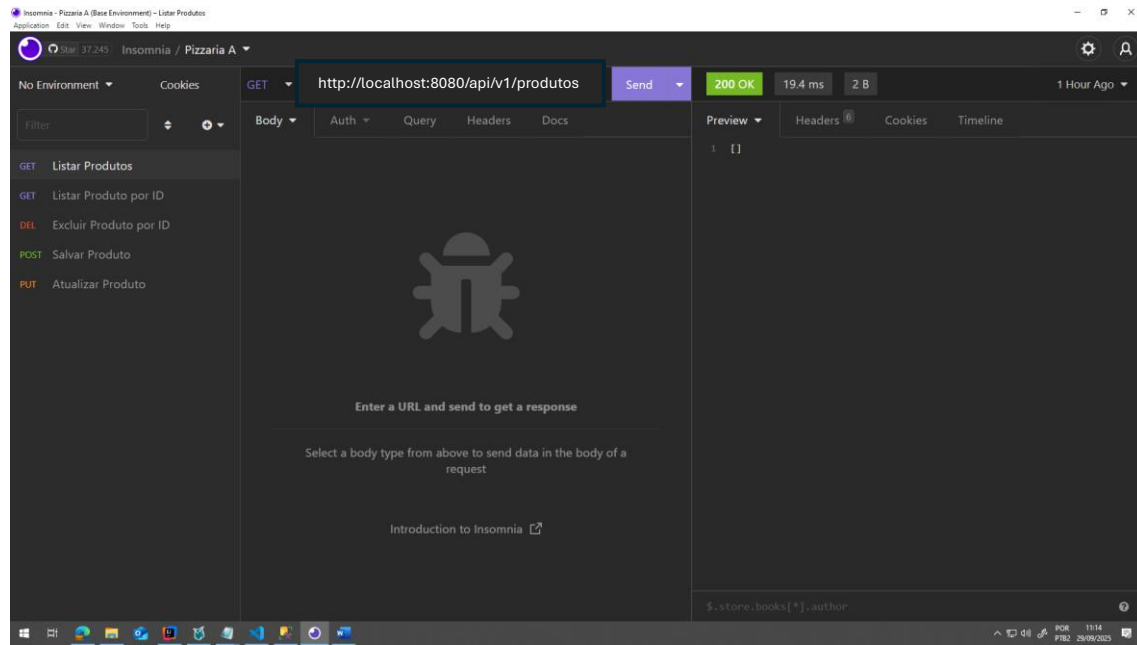
71
96 @DeleteMapping("/{id}") new *
97 public ResponseEntity<Object> excluirProduto(@PathVariable String id) {
98     try {
99         produtosService.delete(Long.parseLong(id));
100         return ResponseEntity.ok().body(
101             Map.of(
102                 k1: "status", v1: 200,
103                 k2: "message", v2: "Produto excluído com sucesso!"
104             )
105         );
106     } catch (NumberFormatException e) {
107         return ResponseEntity.badRequest().body(
108             Map.of(
109                 k1: "status", v1: 400,
110                 k2: "error", v2: "Bad Request",
111                 k3: "message", v3: "0 id informado não é válido: " + id
112             )
113         );
114     } catch (RuntimeException e) {
115         return ResponseEntity.status(404).body(
116             Map.of(
117                 k1: "status", v1: 404,
118                 k2: "error", v2: "Not Found",
119                 k3: "message", v3: "Produto não encontrado com o id: " + id
120             )
121         );
122     }
123 }
124
125

```

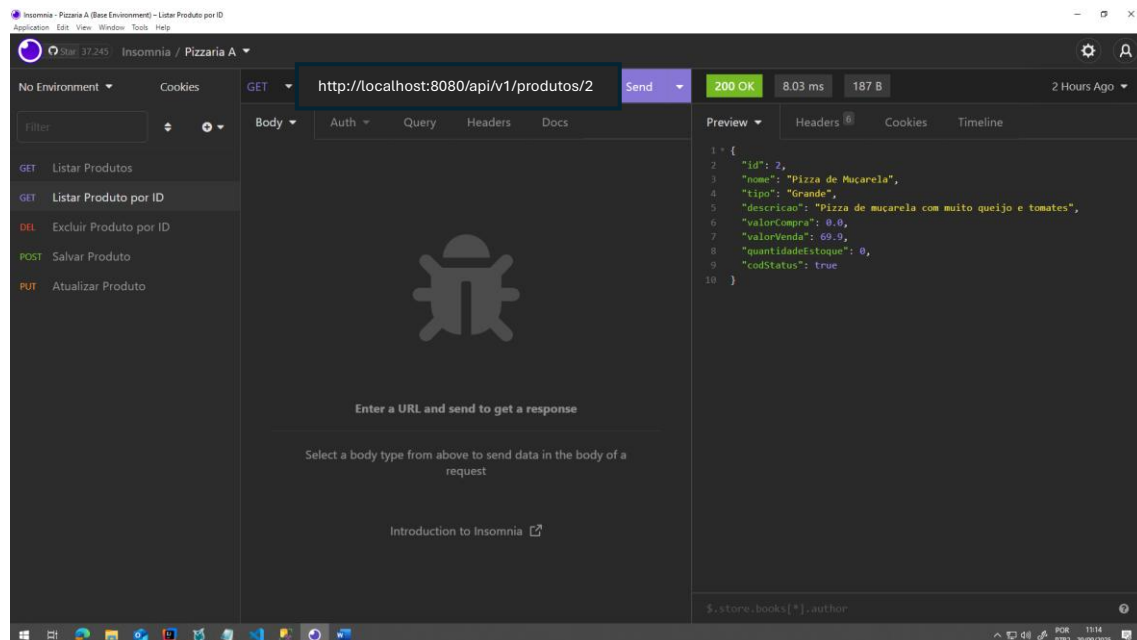
Prosseguindo temos todos os End Points no Insomnia

No VS-CODE: REST Cliente no arquivo (produtos.http)

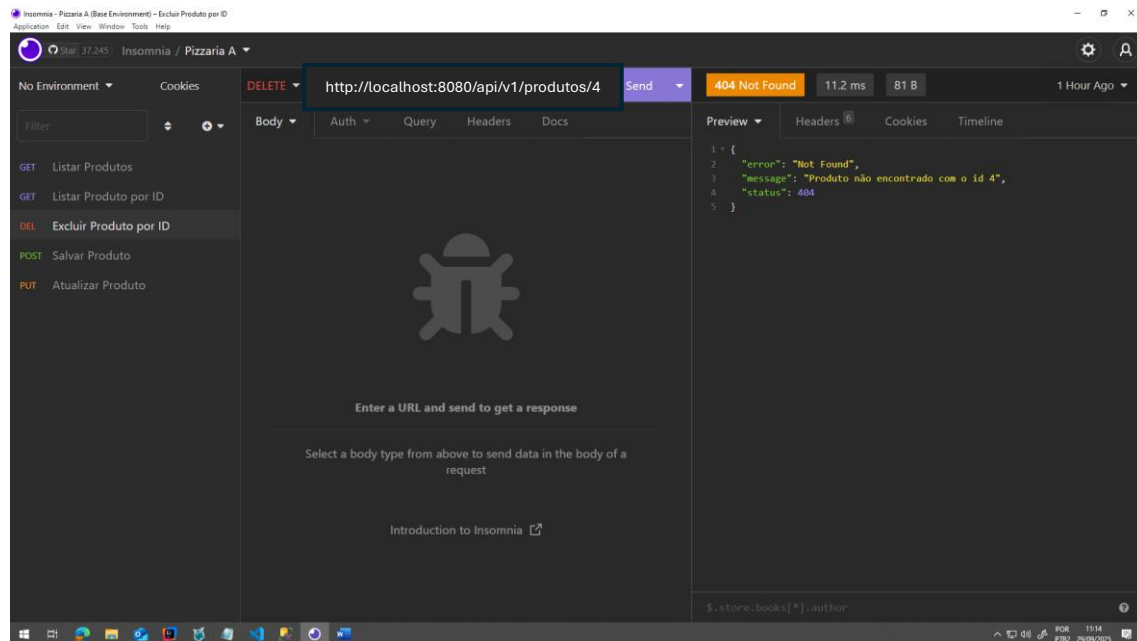
Listar Produtos



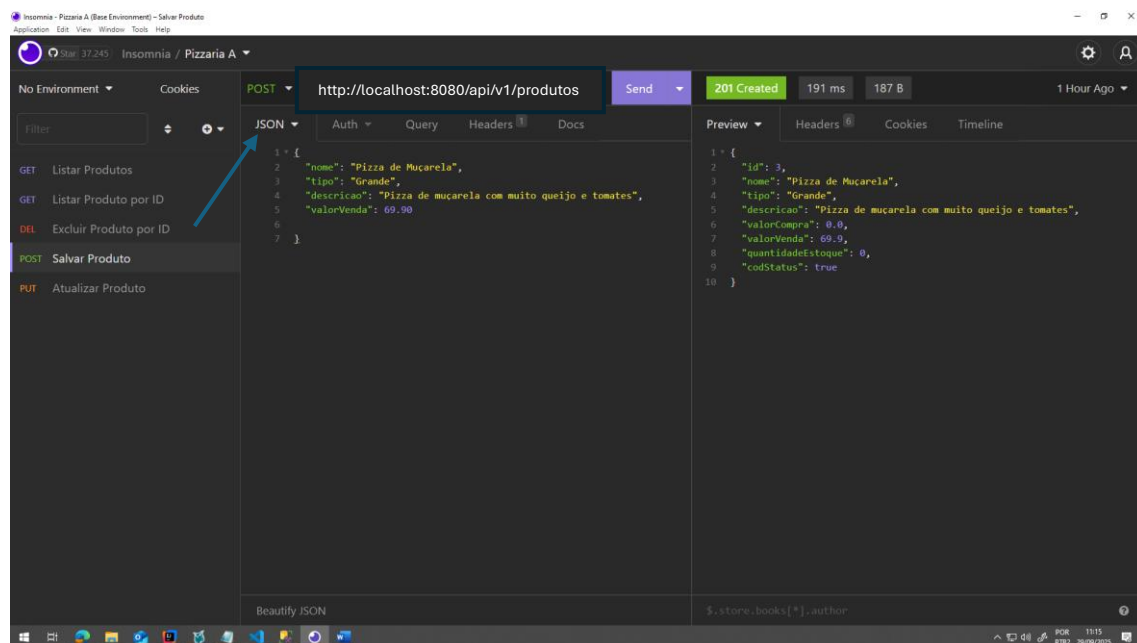
Listar Produto por ID



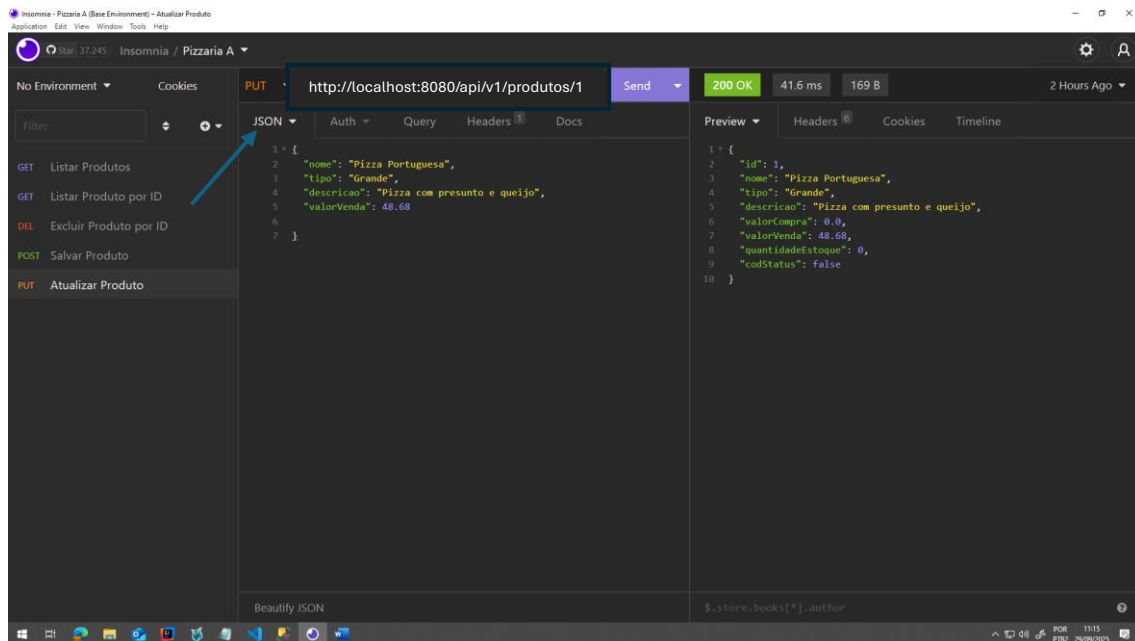
Excluir produto por ID



Salvar Produto



Atualizar Produto



Com todos os testes terminados, temos a necessidade de liberar o CORS para que o **FRONT-END** consiga conectar com a nossa API.

Em nosso caso, vamos liberar as portas 5173 (Vite – React) e a porta 8686 (Mobile Flutter)

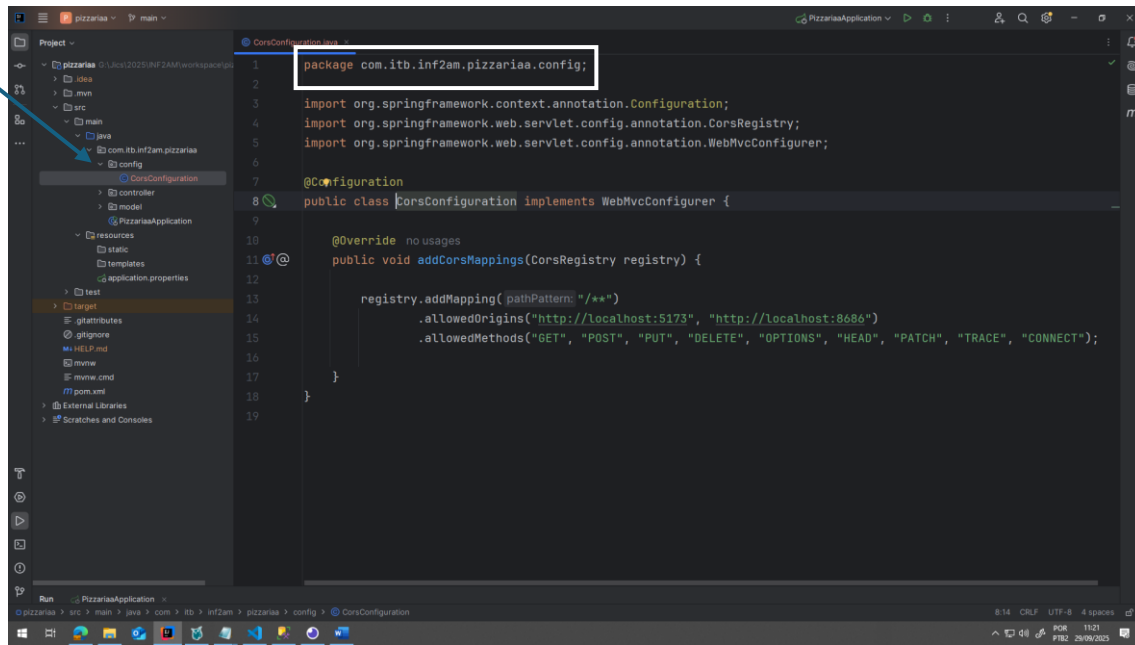
CORS

CORS - Cross-Origin Resource Sharing (Compartilhamento de recursos com origens diferentes) é um mecanismo que usa cabeçalhos adicionais **HTTP** para informar a um navegador que permita que um aplicativo Web seja executado em uma origem (domínio) com permissão para acessar recursos selecionados de um servidor em uma origem distinta. Um aplicativo Web executa uma requisição *cross-origin* HTTP ao solicitar um recurso que tenha uma origem diferente (domínio, protocolo e porta) da sua própria origem.

Criar o package “config” e dentro uma classe “CorsConfiguration”

Obs: Criar o packge “CORRETAMENTE” dentro do package principal.

Veja:



Após término, atualizar o git / github

Nota: Caso não tenha o repositório, fazer o exercício 3 da página 24

Abrir o Terminal ...

git add . <<ENTER>>

git commit -m "Controller (CRUD) de Produto concluído e Cors configurado" <<ENTER>>

git push <<ENTER>>

Projeto Completo no GitHub

- <https://github.com/rogercaetanospizzaria-crud.git>